Titan-Board-Mini SDK

Documentation

The Titan Board Mini, developed by RT-Thread, is built on Renesas' R7KA8P1 chip featuring a dual-core architecture of Cortex-M85 and Cortex-M33. It offers engineers a flexible and comprehensive development platform, empowering developers to gain deeper expertise in the embedded IoT field.

Version 1.2.0 | English | October 22, 2026

www.rt-thread.org

Copyright (c) 2006-2025, RT-Thread Development Team

Contents

- 1. Getting Started
  - 1.1. Template Project
- 2. Basics
  - 2.1. LED Blink Example
- 3. Drivers
  - 3.1. Titan Mini Driver All Example Guide
  - 3.2. Ethernet Example Guide
- 4. Multimedia Display
  - 4.1. MIPI CSI Camera Example Guide
  - 4.2. LVGL Example Guide
  - 4.3. MIPI NPU Face Detection
- 5. Multicore
  - 5.1. Titan_Mini_rpmsg_master Inter-Core Communication Example (M85)
  - 5.2. Titan_Mini_rpmsg_remote Inter-Core Communication Example (M33)
- 6. Peripherals
  - 6.1. PDM Microphone Real-time Loopback Example Guide
  - 6.2. IMU Sensor Example
  - 6.3. RGB LCD Example Guide
  - 6.4. USB PCDC Driver Example Guide
  - 6.5. WAV Audio Player Example Guide

1. Getting Started

1.1. Template Project

English | **中文**

Introduction

This is a template project for **Titan Board Mini**. It implements basic LED blinking functionality and can be used as a starting point for secondary development.

Features

This template provides:

- RT-Thread full system configuration
- Basic LED driver (GPIO)
- Serial debug output (UART)
- Project structure for secondary development

Project Structure

```
project/
├── board/              # Board configuration
├── libraries/          # RA8P1 HAL libraries
├── rt-thread/          # RT-Thread kernel
├── src/                # Application source code
├── Kconfig             # Menuconfig configuration
```

```
├── SConstruct          # Build script
└── ...
```

Quick Start

1. Open the project in RT-Thread Studio
2. Configure the project using menuconfig if needed
3. Build the project
4. Download and run

Secondary Development

This template can be used as a base for:

- Adding new peripherals
- Implementing custom applications
- Developing drivers
- Building complete applications

Further Reading

- RT-Thread Documentation

2. Basics

2.1. LED Blink Example

English | **中文**

Introduction

This example is the simplest and most basic example in the SDK, similar to the "Hello World" program that programmers encounter as their first program. The main function of this example is to make the onboard RGB LED blink periodically.

GPIO Introduction

1. Overview

GPIO (General Purpose Input/Output) is one of the most commonly used peripheral interfaces of an MCU. It can be configured under software control as either **input mode** or **output mode**:

- **Input mode**: Used to read external voltage levels, such as button inputs.
- **Output mode**: Used to control peripheral voltage levels, such as lighting up an LED or driving a buzzer.

2. RA8 Series GPIO Features

- **Rich pin resources**: Each port can be independently configured as input/output/multiplexed function.
- **Multi-function multiplexing**: In addition to GPIO functionality, the same pin can also serve as a signal line for peripherals such as UART, SPI, or I2C.

- **Level control**: In output mode, pins can drive either high or low levels; some pins support open-drain mode.
- **Direction control**: Input/output direction can be dynamically switched.

3. RT-Thread PIN Framework

RT-Thread provides a **PIN device driver framework**, which offers a unified interface to shield hardware differences:

- `rt_pin_mode()` - Set the working mode of a pin (input/output/pull-up/pull-down, etc.)
- `rt_pin_write()` - Output a voltage level (high/low)
- `rt_pin_read()` - Read an input voltage level

This way, developers don't need to directly manipulate registers; GPIO control can be achieved through RT-Thread's API.

In this example, the LED pin is configured as **output mode**, and the software continuously toggles between high and low levels to make the LED blink.

Hardware Description

Titan Board Mini provides three user LEDs: LED_R (RED), LED_B (BLUE), LED_G (GREEN). The LED corresponds to specific pins on the MCU. To turn on the LED, a low level signal is output from the MCU pin, and to turn it off, a high level signal is output.

Software Description

The source code for this example is located in `src/hal_entry.c`. The MCU pin definitions for the LEDs and the LED control logic can be found in the source file.

```
/* Configure LED pins */
rt_pin_mode(LED_PIN, PIN_MODE_OUTPUT);

/* Control LED */
rt_pin_write(LED_PIN, LED_ON);  // Turn on
rt_pin_write(LED_PIN, LED_OFF); // Turn off
```

Compilation & Download

1. Open the project in RT-Thread Studio
2. Build the project using the build button
3. Connect the debug probe to the development board
4. Download the firmware to the board

Run Effect

After pressing the reset button to restart the development board, observe the LED blinking effect. The RGB LEDs will change periodically.

Further Reading

- RT-Thread PIN Device

3. Drivers

3.1. Titan Mini Driver All Example Guide

Chinese | **English**

Introduction

`Titan_Mini_driver_all` is a peripheral driver example collection project for the Titan Mini Board. It centralizes verification of common peripherals, storage, and networking functions. The project exposes individual example entry points for each peripheral through MSH commands, making them convenient to debug independently.

Currently integrated examples:

- RTC
- GPIO / PWM / Key interrupt
- SD Card
- Flash file system
- SDRAM
- ADC
- SPI
- CANFD
- I2C
- Watchdog

FSP Configuration

This project is based on **Renesas FSP (Flexible Software Package) 6.4.0** and **RT-Thread Studio** for hardware abstraction layer configuration and code

generation. It works out of the box without reconfiguration, but follow the workflow below to modify peripheral pins or add new FSP Stacks.

FSP configuration tool download:

- setup_fsp_v6_4_0_rasc_v2025-12.exe

Version Requirements

Item	Version	Note
FSP version	6.4.0	Pinned by `#FSPVersion#` in `configuration.xml`
MCU	R7KA8P1KFLCAC (RA8P1)	Cortex-M85 + Cortex-M33 dual-core
IDE	RT-Thread Studio	Bundled with the FSP configuration plug-in

> ⚠️ *Always use FSP 6.4.0. Other versions may cause regenerated `ra/`, `ra_gen/`, `ra_cfg/` code to be incompatible with the driver layer.*

Project Directory Layout

```
Titan_Mini_driver_all/
├── configuration.xml      # FSP configuration (pins / clocks /
├── ra/                    # FSP-generated driver sources (r_gp
├── ra_cfg/                # FSP configuration headers (fsp_cfg
├── ra_gen/                # FSP auto-generated HAL data (hal_d
├── board/                 # BSP board-level initialization
├── src/                   # Application-layer example code (rt
└── rtconfig.h             # RT-Thread kernel configuration
```

Steps to Modify the FSP Configuration

1. **Open configuration**: In RT-Thread Studio, right-click the project → `Renesas FSP Configuration` (or double-click `configuration.xml`).

2. **Adjust Stacks**: In the `Stacks` view, add/remove driver Stacks (such as `r_gpt`, `r_adc`, `r_canfd`). The properties of each Stack must match the channel/pins described in the corresponding peripheral section.
3. **Configure pins**: In the `Pins` view, assign pin modes (GPIO/SPI/I2C/CAN, etc.). Ethernet-related pins must have their drive strength set to `H`.
4. **Generate code**: Click `Generate Project Content`. FSP will regenerate `ra_gen/hal_data.c`, `ra_gen/pin_data.c`, and the configuration headers under `ra_cfg/`.
5. **Sync RT-Thread Settings**: In `RT-Thread Settings`, tick the corresponding driver frameworks (PWM / ADC / CAN / SPI / I2C / WDT, etc.) so that the RT-Thread device framework registers the corresponding device nodes (such as `pwm6`, `adc0`, `canfd0`).
6. **Rebuild**: After FSP generates code, you must clean and then build to avoid stale `.o` caches causing link errors.

Common Issues

- `pwm6` / `adc0` **not found**: Usually because the corresponding driver framework is not ticked in RT-Thread Settings, or the FSP Stack is not added.
- `hal_data.h` **errors**: FSP version mismatch. Regenerate with 6.4.0.
- **ETH pin communication abnormal**: Confirm that all ETH pins have their drive strength set to `H` (see the Ethernet section).
- **Link errors after modifying configuration**: Run `Project → Clean`, then build.

Build and Run

1. Open the `project/Titan_Mini_driver_all` project in RT-Thread Studio.
2. Compile and download the firmware to the Titan Mini development board.
3. Enter the MSH command line through the serial terminal or USB PCDC terminal.

4. Enter `help` to see available commands. Example commands for each peripheral are described in the following sections.

After power-on, the terminal prints the following boot banner:

```
============================================================
           Titan Mini Board Factory Test System
============================================================
System initialized successfully!
```

RTC Example

Introduction

The RTC example is based on the **RT-Thread RTC device driver framework**. It verifies the proper operation of the RA8P1 on-chip real-time clock, supporting date/time set and read operations. When the `RT_USING_ALARM` component is enabled, it also verifies the alarm function. The RTC is driven by a 32.768 kHz crystal; when main power is removed, time can be backed up via the VBATT battery.

Hardware Parameters

Parameter	Specification	Note
Clock source	32.768 kHz crystal	Sub-clock oscillator, ±5 ppm accuracy
Calendar range	2000–2099	Automatic leap-year compensation
Time format	24-hour	HH:MM:SS
Operating voltage	1.62–3.6 V	Wide voltage range
Backup power	VBATT	Keeps time information after main power loss
Alarm triggers	sec / min / hour / day / week / month / year	Multiple programmable triggers

Usage Steps

1. Enter `rtc_sample` to set the RTC date to 2025-8-1, time to 15:00:00, and read and print the current time after 3 seconds.
2. If the project enables `RT_USING_ALARM`, enter `alarm_sample` to create a second-level alarm.

Key Code

The example source is in `src/rt_example_rtc.c`. The core logic is shown below.

Time Set and Read

```
void rtc_sample(void)
{
    rt_device_t device = rt_device_find(RTC_NAME);     // RTC_N
    if (device == RT_NULL)
    {
        rt_kprintf("find %s failed!\n", RTC_NAME);
        return;
    }

    if (rt_device_open(device, 0) != RT_EOK)
    {
        rt_kprintf("open %s failed!\n", RTC_NAME);
        return;
    }

    set_date(2025, 8, 1);                              // set d
    set_time(15, 0, 0);                                // set t

    rt_thread_mdelay(3000);
    time_t now = 0;
    get_timestamp(&now);
    rt_kprintf("now: %.*s", 25, ctime(&now));
}
MSH_CMD_EXPORT(rtc_sample, rtc sample);
```

Alarm Creation (depends on `RT_USING_ALARM`)

```
static void user_alarm_callback(rt_alarm_t alarm, time_t timest
{
    rt_kprintf("user alarm callback function.\n");
```

```
}

void alarm_sample(void)
{
    struct rt_alarm_setup setup;
    static rt_alarm_t alarm = RT_NULL;
    time_t now = get_timestamp(RT_NULL) + 1;            // trig
    struct tm p_tm;
    gmtime_r(&now, &p_tm);

    setup.flag = RT_ALARM_SECOND;                       // seco
    setup.wktime = *(struct tm *)&p_tm;                 // trig

    alarm = rt_alarm_create(user_alarm_callback, &setup);
    rt_alarm_start(alarm);
}
```

> *If `RT_USING_ALARM` is not enabled, `alarm_sample` prints `alarm sample is unavailable: RT_USING_ALARM is not enabled.`.*

Result

- `rtc_sample` prints the current RTC time after setting it.
- `alarm_sample` prints `user alarm callback function.` when triggered.

Terminal output after running `alarm_sample`:

```
 \ | /
- RT -     Thread Operating System
 / | \     5.1.0 build Mar 26 2026 09:38:12
 2006 - 2024 Copyright by RT-Thread team

Hello RT-Thread!
============================================================
This example project is a rtc and alarm routine!
============================================================
set RTC date to 2025-8-1
msh >now: Fri Aug  1 15:00:03 2025
al
alarm_sample
msh >alarm_sample
msh >[I/rtc] user alarm callback function.
[I/rtc] user alarm callback function.
```

GPIO / PWM / Key Interrupt Example

Introduction

These examples are based on the **RT-Thread PWM device driver framework** and the **PIN device interrupt interface**. They verify the RA8P1 GPT general-purpose PWM timer and GPIO external interrupt functionality. Two entry points are provided:

- `pwm_sample`: manually configure the period and duty cycle of `pwm6` for oscilloscope observation.
- `key_irq_sample`: configure the key falling-edge interrupt; the green LED toggles on each trigger.

Hardware Parameters

Parameter	Specification	Note
PWM device	`pwm6` (GPT6)	32-bit general-purpose PWM timer
PWM counter	32-bit	Range 0 – 4294967295
PWM output pin	P601 (GTIOC6A)	Oscilloscope probe point
LED pin	`BSP_IO_PORT_01_PIN_08`	Green LED
Key pin	`BSP_IO_PORT_02_PIN_01`	Internal pull-up, falling-edge trigger
GPT modes	Periodic / One-shot / PWM	Square / Sawtooth / Triangle wave

Usage Steps

1. Enter `pwm_sample <period> <pulse>`, e.g. `pwm_sample 500000 250000` (units in ns → 2 kHz / 50% duty). Connect an oscilloscope to P601 to observe the waveform.
2. Enter `key_irq_sample`, then press the key to toggle the green LED.

Key Code

The PWM example is in `src/rt_example_gpt.c`, and the key interrupt example is in `src/rt_example_key_irq.c`.

PWM Configuration (manual period and pulse)

```
#define PWM_DEV_NAME        "pwm6"
#define PWM_DEV_CHANNEL     0

static int pwm_sample(int argc, char *argv[])
{
    rt_uint32_t period = (rt_uint32_t) atoi(argv[1]);
    rt_uint32_t pulse  = (rt_uint32_t) atoi(argv[2]);

    if ((period == 0) || (pulse > period))
    {
        rt_kprintf("invalid parameters, ensure period > 0 and p
        return -RT_ERROR;
    }

    struct rt_device_pwm *pwm_dev = (struct rt_device_pwm *) rt
    rt_pwm_set(pwm_dev, PWM_DEV_CHANNEL, period, pulse);
    rt_pwm_enable(pwm_dev, PWM_DEV_CHANNEL);

    rt_kprintf("pwm started on %s channel %d, period=%u pulse=%
               PWM_DEV_NAME, PWM_DEV_CHANNEL, period, pulse);
    return RT_EOK;
}
MSH_CMD_EXPORT(pwm_sample, configure and start pwm output: pwm_
```

Key Interrupt (toggle LED on falling edge)

```
#define LED_PIN_G   BSP_IO_PORT_01_PIN_08
#define KEY_PIN     BSP_IO_PORT_02_PIN_01

static volatile rt_bool_t g_key_flag = RT_FALSE;

static void key_irq_callback(void *args)
{
    rt_pin_write(LED_PIN_G, g_key_flag ? PIN_HIGH : PIN_LOW);
    g_key_flag = g_key_flag ? RT_FALSE : RT_TRUE;
}

void key_irq_sample(void)
{
```

```
    rt_pin_mode(LED_PIN_G, PIN_MODE_OUTPUT);
    rt_pin_mode(KEY_PIN, PIN_MODE_INPUT_PULLUP);
    rt_pin_attach_irq(KEY_PIN, PIN_IRQ_MODE_FALLING, key_irq_ca
    rt_pin_irq_enable(KEY_PIN, PIN_IRQ_ENABLE);
}
MSH_CMD_EXPORT(key_irq_sample, key interrupt sample);
```

Parameter Conversion

Parameters of `pwm_sample` are in **nanoseconds**:

```
Frequency = 1e9 / period
Duty cycle = pulse / period × 100%

Example pwm_sample 500000 250000:
  Frequency = 1e9 / 500000 = 2000 Hz (2 kHz)
  Duty cycle = 250000 / 500000 = 50%
```

Result

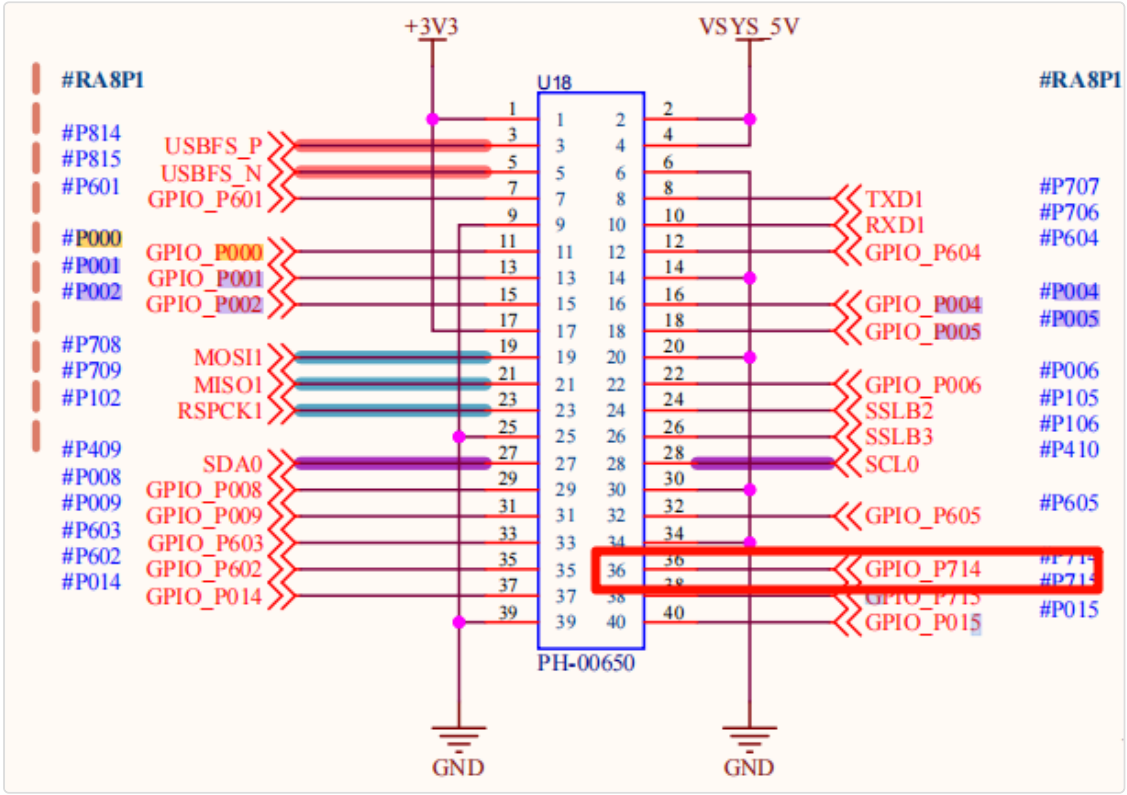
- On success, `pwm_sample` prints the device, channel, period, and pulse. The waveform can be observed on P601.
- After `key_irq_sample` starts, every falling edge of the key toggles the green LED once.

Terminal output of `pwm_sample 500000 250000`:

```
 \ | /
- RT -     Thread Operating System
 / | \     5.1.0 build Mar 25 2026 17:53:14
 2006 - 2024 Copyright by RT-Thread team

Hello RT-Thread!
============================================================
This example project is a Titan Board Mini template routine!
============================================================
[I/pwm] Tips
[I/pwm] Please enter one of the above commands to run the corresponding sample.
[I/pwm]    • hwtimer_sample : Demonstration of Hardware Timer usage.
[I/pwm]    • pwm_sample     : Demonstration of PWM (Pulse Width Modulation) usage
.
msh >pwm
pwm_sample
pwm
msh >pwm_s
pwm_sample
msh >pwm_sample
[I/pwm] Usage: pwm_sample <period> <pulse>
[I/pwm] Example: pwm_sample 500000 250000
msh >pwm_sample 500000 250000
[I/pwm] PWM started on device: pwm12, channel: 0
[I/pwm] Period: 500000 ns, Pulse: 250000 ns
[I/pwm] Please connect the 'P714' to a logic analyzer or oscilloscope for wavefo
rm observation.
msh >
```

The 2 kHz / 50% PWM waveform observed on P601 with an oscilloscope:



SD Card Example

Introduction

The SD card example is based on the **RT-Thread DFS file system framework** and the **RA8 SDHI hardware module**. It verifies that the `/sdcard` mount point can be properly created, written, read, and used to delete test files. SDHI communicates with SD/SDHC/SDXC cards via the SD bus, supporting 1-bit/4-bit data lines and DMA transfer.

Hardware Parameters

Parameter	Specification	Note
Supported card types	SDSC / SDHC / SDXC	Compatible with SD v1.x / v2.x
Bus width	1-bit / 4-bit	Project default: 1-bit
Block size	512 Byte	Standard block size
Maximum clock	50 MHz SDCLK	Depends on MCU clock configuration
Operating voltage	3.3 V	Some Micro SD cards support 1.8 V
Error check	CRC7 (command) / CRC16 (data)	Hardware auto check

Usage Steps

1. Confirm the SD card is inserted and automatically mounted at `/sdcard` (handled by `BSP_USING_FS_AUTO_MOUNT` + `BSP_USING_SDCARD_FATFS`).
2. Enter `sdcard_sample` to perform one file read/write verification.

Key Code

The example source is in `src/rt_example_sdcard.c`, using the standard POSIX `fopen/fputs/fgets/unlink` API.

```
void sdcard_sample(void)
{
    const char *test_file = "/sdcard/test_sdcard.txt";
    const char *test_data = "SD Card test data - Titan Mini Boa
    char read_buf[64] = {0};

    /* 1. Write test data */
    FILE *fp = fopen(test_file, "w");
    if (fp == RT_NULL)
    {
        rt_kprintf("failed to create test file: %s\n", test_fil
        return;
    }
    fputs(test_data, fp);
    fclose(fp);

    /* 2. Read back and verify */
    fp = fopen(test_file, "r");
    if (fgets(read_buf, sizeof(read_buf), fp) == RT_NULL)
    {
        rt_kprintf("failed to read test data\n");
        fclose(fp);
        return;
    }
    fclose(fp);

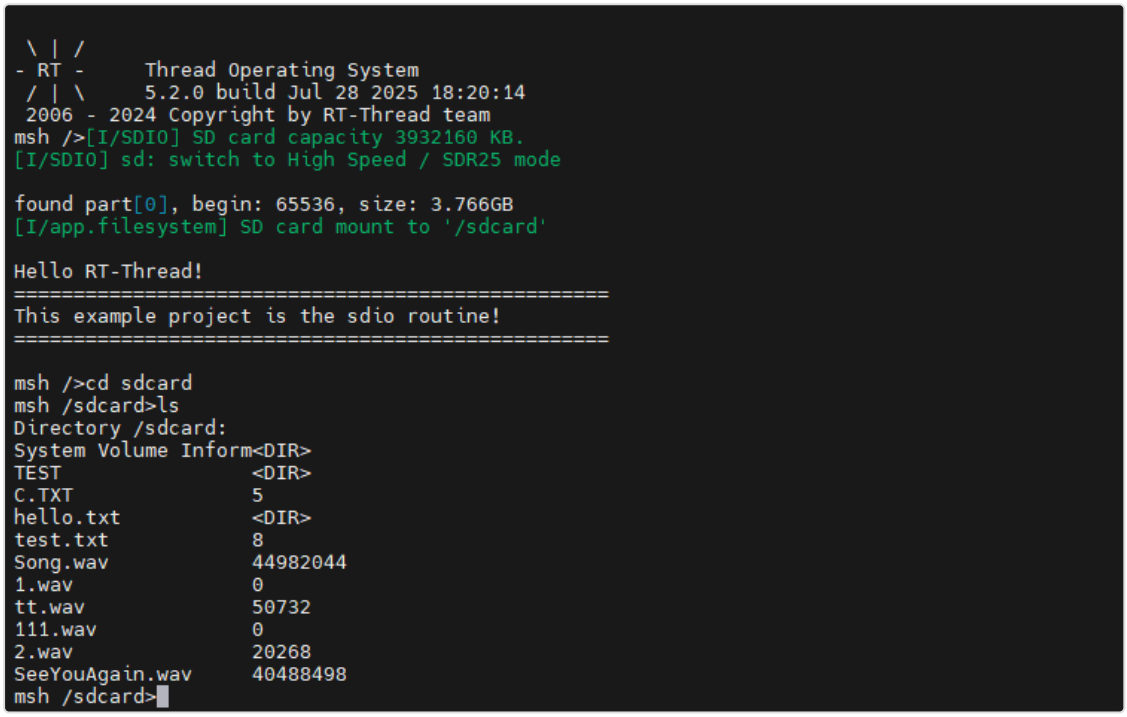
    /* 3. Compare content */
    if (strcmp(read_buf, test_data) != 0)
    {
        rt_kprintf("sdcard data mismatch!\n");
        return;
    }

    /* 4. Cleanup */
    unlink(test_file);
    rt_kprintf("sdcard sample passed\n");
}
MSH_CMD_EXPORT(sdcard_sample, sdcard file read write sample);
```

Result

- On success, the example writes, reads, verifies, and deletes the test file, then prints `sdcard sample passed`.
- On mount failure, write failure, or data mismatch, the corresponding cause is printed (`failed to create/write/read` or `data mismatch`).

Terminal output after the SD card is mounted:



Flash File System Example

Introduction

The Flash example is based on the **RT-Thread DFS + FAL + LittleFS** three-layer architecture. It verifies basic read/write capability on the on-board QSPI Flash (W25Q64) at the `/fal` mount point. The bottom layer uses the FAL abstraction layer to manage Flash partitions, and the file system uses power-loss-safe LittleFS.

Hardware Parameters

Parameter	Specification	Note
Flash model	W25Q64	On-board QSPI NOR Flash
Capacity	8 MB (64 Mbit)	4 KB sector / 64 KB block / 256 B page
Interface	QSPI (Quad SPI)	4-bit data line, up to 133 MHz
Operating voltage	2.7 V – 3.6 V	—
Endurance	~100k erase/program per sector	~20-year data retention at room temperature
Filesystem partition	`filesystem` (1 MB)	Defined by the FAL partition table

Software Stack

```
Application (flash_sample)
    ↓
DFS (DFS framework, provides fopen/fread/POSIX API)
    ↓
LittleFS (power-loss safe + wear leveling)
    ↓
FAL (Flash abstraction layer, manages partitions)
    ↓
W25Q64 QSPI driver → OSPI_B hardware
```

Usage Steps

1. Confirm that FAL and LittleFS are enabled and automatically mounted at `/fal` (handled by `BSP_USING_FLASH_FS_AUTO_MOUNT`; on first mount failure it will automatically `dfs_mkfs("lfs", ...)` to format).
2. Enter `flash_sample` to perform a small-file read/write verification.
3. Enter `flash_speed_test` to perform a 64 KB write/read/delete bandwidth test.

Key Code

The example source is in `src/rt_example_flash.c`. It provides two commands: `flash_sample` (functional verification) and `flash_speed_test` (bandwidth test).

Functional Verification (flash_sample)

```
void flash_sample(void)
{
    const char *test_file = "/fal/test_flash.txt";
    const char *test_data = "Flash test data - Titan Mini Board
    char read_buf[64] = {0};

    /* 1. Write test data */
    FILE *fp = fopen(test_file, "w");
    if (fp == RT_NULL)
    {
        rt_kprintf("failed to create test file: %s\n", test_fil
        return;
    }
    fputs(test_data, fp);
    fclose(fp);

    /* 2. Read back and verify */
    fp = fopen(test_file, "r");
    if (fgets(read_buf, sizeof(read_buf), fp) == RT_NULL)
    {
        rt_kprintf("failed to read test data\n");
        fclose(fp);
        return;
    }
    fclose(fp);

    /* 3. Compare content */
    if (strcmp(read_buf, test_data) != 0)
    {
        rt_kprintf("flash data mismatch!\n");
        return;
    }

    /* 4. Cleanup */
    unlink(test_file);
    rt_kprintf("flash sample passed\n");
}
MSH_CMD_EXPORT(flash_sample, flash file read write sample);
```

Bandwidth Test (flash_speed_test)

The DWT cycle counter is used to time the write, read, and delete operations on a 64 KB file and convert them to KB/s bandwidth. The read phase also performs data verification.

```
#define FLASH_SPEED_TEST_SIZE   (64 * 1024)   /* 64 KB test fil
#define FLASH_SPEED_BUF_SIZE    4096          /* single read/wr

void flash_speed_test(void)
{
    /* 1. Sequentially write 64 KB (4 KB per iteration), DWT-ti
    dwt_init();
    FILE *fp = fopen("/fal/speed_test.bin", "wb");
    uint32_t start = dwt_get_cycle();
    for (offset = 0; offset < total; offset += FLASH_SPEED_BUF_
        fwrite(wr_buf, 1, FLASH_SPEED_BUF_SIZE, fp);
    fflush(fp); fclose(fp);
    flash_speed_report("Write", dwt_get_cycle() - start, total)

    /* 2. Sequential read + data verification, DWT-timed */
    fp = fopen("/fal/speed_test.bin", "rb");
    start = dwt_get_cycle();
    for (offset = 0; offset < total; offset += FLASH_SPEED_BUF_
    {
        fread(rd_buf, 1, FLASH_SPEED_BUF_SIZE, fp);
        if (memcmp(rd_buf, wr_buf, FLASH_SPEED_BUF_SIZE) != 0)
    }
    flash_speed_report("Read", dwt_get_cycle() - start, total);
    fclose(fp);

    /* 3. Delete the file (corresponds to Flash erase), DWT-tim
    start = dwt_get_cycle();
    unlink("/fal/speed_test.bin");
    flash_speed_report("Erase", dwt_get_cycle() - start, total)
}
MSH_CMD_EXPORT(flash_speed_test, flash read/write speed test);
```

Bandwidth conversion formula (same as SDRAM):

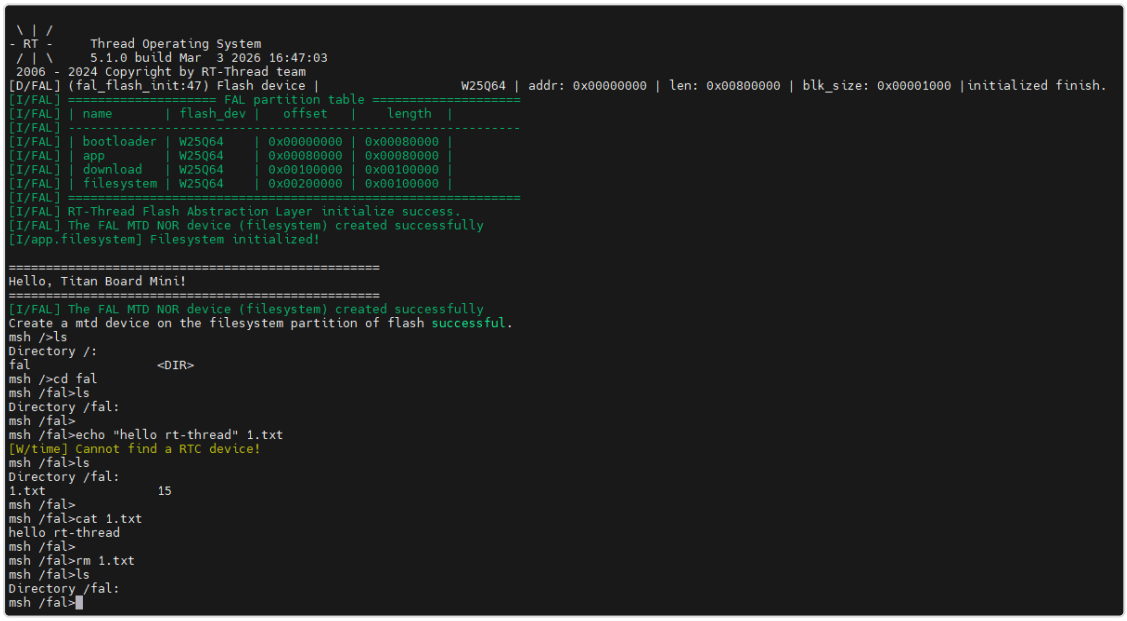
```
Elapsed (s) = DWT delta / SystemCoreClock
Bandwidth (KB/s) = (test bytes / 1024) / elapsed
```

Result

- `flash_sample` prints `flash sample passed` on success; on failure it prints `failed to create/write/read` or `flash data mismatch!`.
- `flash_speed_test` reports `Write`, `Read`, and `Erase` bandwidth (KB/s) and elapsed time (ms), and verifies data during the read phase.

> *Flash must be erased before writing and must be aligned to sector/page boundaries. LittleFS automatically handles wear leveling and power-loss protection; the application layer simply uses POSIX interfaces.*

Terminal output after the Flash file system is mounted:



SDRAM Example

Introduction

The SDRAM example is based on the **RA8P1 SDRAMC controller**. It verifies that the external 64 MB SDRAM is readable/writable and uses the DWT cycle counter to report write, read, and `memcpy` bandwidth.

Hardware Parameters

Parameter	Specification	Note
Capacity	64 MB (256 Mbit)	4 banks × 4 M × 32 bit
Data width	32 bit	High-throughput data transfer
Base address	`0x68000000`	Access range 0x68000000 – 0x68FFFFFF
Operating voltage	3.3 V	—
CAS latency	CL3	Programmable timing
Refresh	Auto refresh	Built into SDRAMC, ensures data integrity

Usage Steps

1. Enter `sdram_speed_test` to perform one SDRAM speed test.

Key Code

The example source is in `src/rt_example_sdram.c`. It uses the DWT cycle counter for nanosecond-level timing.

Memory Mapping and Timing Helpers

```
#define SDRAM_TEST_WORDS    (8 * 1024 * 1024)             // 8 M
#define SDRAM_BASE_ADDR     (0x68000000)

static volatile uint32_t *sdram_buf = (uint32_t *) SDRAM_BASE_A

static void dwt_init(void)
{
    CoreDebug->DEMCR |= CoreDebug_DEMCR_TRCENA_Msk;
    DWT->CYCCNT = 0;
    DWT->CTRL  |= DWT_CTRL_CYCCNTENA_Msk;
}
```

Result

- The terminal prints the test capacity, CPU frequency, and `Write`, `Read`, and `memcpy` bandwidth results (MB/s and ms).
- If a read-back verification mismatch occurs, it prints the error address and the expected/actual values (`expect_low16` / `actual_low16`).

`sdram_speed_test` output:

```
msh >[D/drv_sdram] sdram init success, mapped at 0x68C00000, size is 20971520 by
tes, data width is 32

 \ | /
- RT -     Thread Operating System
 / | \     5.1.0 build Mar 25 2026 17:28:15
 2006 - 2024 Copyright by RT-Thread team

Hello RT-Thread!
============================================================
This example project is a Titan Board Mini template routine!
============================================================
msh >sd
sdram_speed_test
msh >sdram_speed_test

========== SDRAM 32-BIT SPEED TEST ==========
CPU Freq: 1000000000 Hz
Test Size: 32 MB
Write:  233.44 MB/s  (137.081 ms)
Read:   166.99 MB/s  (191.626 ms)
memcpy: 82.48 MB/s  (387.954 ms)
=============================================
```

ADC Example

Introduction

The ADC example is based on the **RT-Thread ADC device driver framework**. It verifies that channel 0 of `adc0` on the RA8P1 can sample correctly and converts the raw value to a voltage. The example also adds **invalid-value detection** and **near-full-scale warning** logic to avoid mistaking inputs above the reference voltage for valid samples.

Hardware Parameters

Parameter	Specification	Note
Resolution	16 bit	Raw value range 0 – 65535

Parameter	Specification	Note
Reference voltage	3.3 V	Expressed in code as `330` (3.30 V × 100, fixed-point)
Sample channel	`adc0` channel 0	Single-ended input, range 0 V – 3.3 V
Trigger	Software trigger	Periodic sampling via `rt_adc_read`

> ⚠️ ***Important***: *The input voltage must be below the reference voltage Vref. Applying 3.3 V directly may trigger a full-scale warning or overflow message.*

Usage Steps

1. Connect the analog voltage under test to the corresponding ADC channel (input should be below Vref).
2. Enter `adc_sample` to start the continuous sampling thread, which outputs the raw value and converted voltage once per second.

Key Code

The example source is in `src/rt_example_adc.c`. The core logic is shown below.

Voltage Conversion Constants

```
#define ADC_DEV_NAME        "adc0"
#define ADC_DEV_CHANNEL     0
#define REFER_VOLTAGE       330              // 3.30 V × 100, f
#define CONVERT_BITS        (1 << 16)        // 16-bit resoluti
#define ADC_INVALID_VALUE   0xFFFF           // ADC overrange m
#define ADC_WARN_THRESHOLD  (CONVERT_BITS * 95 / 100)  // 95% f
```

Sampling Thread Main Loop

```
static void adc_vol_sample(void *parameter)
{
    rt_adc_device_t adc_dev = (rt_adc_device_t)rt_device_find(A
    if (adc_dev == RT_NULL)
    {
        rt_kprintf("adc sample run failed! can't find %s device
        return;
    }

    if (rt_adc_enable(adc_dev, ADC_DEV_CHANNEL) != RT_EOK)
    {
        rt_kprintf("adc enable failed! channel = %d\n", ADC_DEV
        return;
    }

    while (1)
    {
        rt_uint32_t value = rt_adc_read(adc_dev, ADC_DEV_CHANNE

        if (value == ADC_INVALID_VALUE)
        {
            rt_kprintf("adc overrange: input is too high, pleas
        }
        else
        {
            rt_uint32_t vol = value * REFER_VOLTAGE / CONVERT_B
            rt_kprintf("the value is :%d\n", value);

            if (value >= ADC_WARN_THRESHOLD)
                rt_kprintf("warning: near full scale, the volta
            else
                rt_kprintf("the voltage is :%d.%02d\n", vol / 1
        }

        rt_thread_mdelay(1000);
    }
}
```

Command Export

```
void adc_sample(void)
{
    rt_thread_t adc = rt_thread_create("adc", adc_vol_sample, R
    if (adc == RT_NULL)
    {
        rt_kprintf("create adc thread failed!\n");
```

```
        return;
    }
    rt_thread_startup(adc);
}
MSH_CMD_EXPORT(adc_sample, adc sample demo);
```

Voltage Conversion

Fixed-point arithmetic is used to avoid floating-point overhead:

```
Voltage (mV) = ADC reading × Reference voltage (mV) / Conversio
             = value × 3300 / 65536

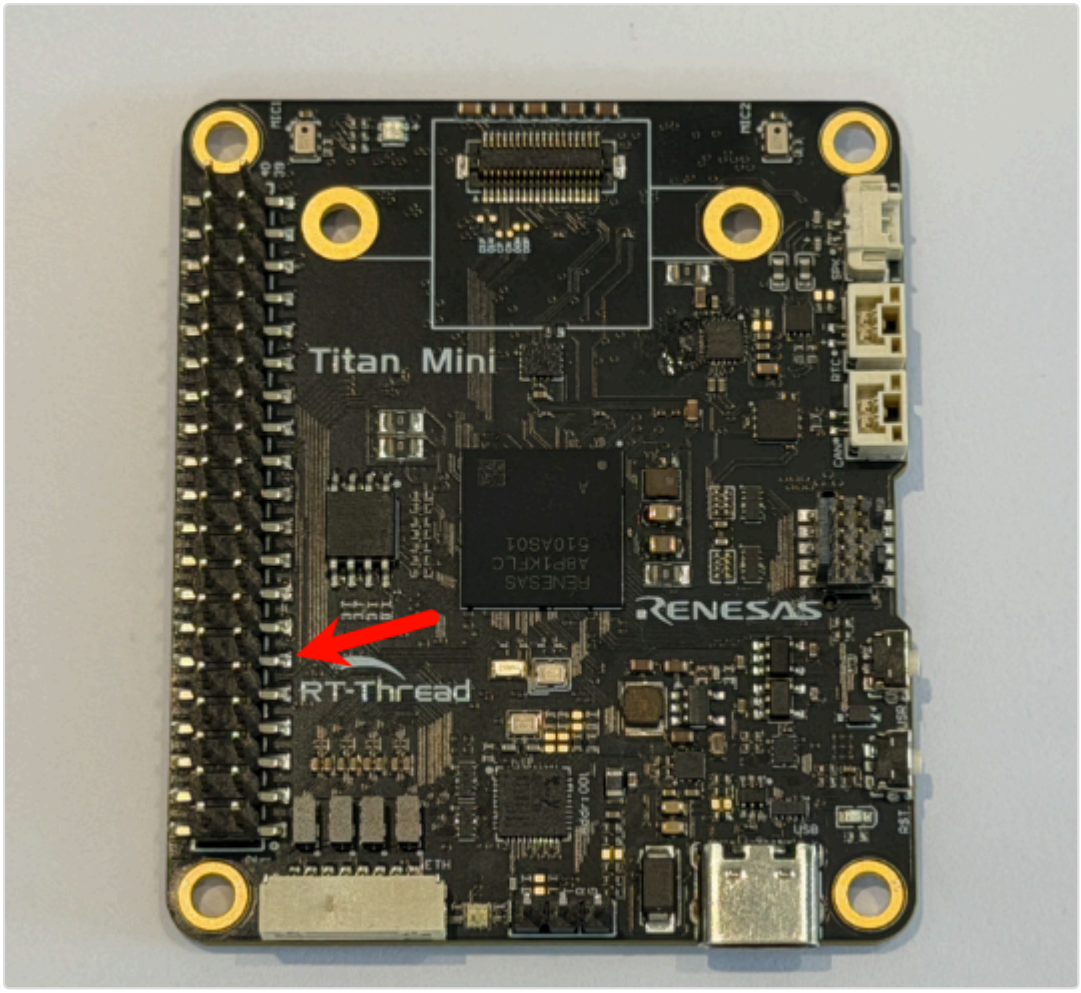
In code: vol = value * 330 / 65536   (unit is 0.01 V)
Display: vol / 100  -> integer part (V)
         vol % 100  -> fractional part (0.01 V)
```

For example, when `value = 32768`, `vol ≈ 165`, the output is `the voltage is :1.65`.

Result

- `adc_sample` periodically prints the raw sample value and the converted voltage.
- When the reading is `0xFFFF`, it reports `adc overrange` — the input is over range.
- When the reading approaches full scale (≥ 95%), it outputs `warning: near full scale`.

When testing, you can use a jumper wire to connect 3.3 V to the ADC pin for verification:



Terminal sampling output after running `adc_sample`:

```
 \ | /
- RT -     Thread Operating System
 / | \     5.1.0 build Mar 25 2026 15:46:49
 2006 - 2024 Copyright by RT-Thread team
[D/drv.adc] adc0 init success

Hello RT-Thread!
============================================================
This example project is a Titan Board Mini template routine!
============================================================
msh >adc_sample
the value is :0
the voltage is :0.00
msh >the value is :12472
the voltage is :0.62
the value is :12464
the voltage is :0.62
the value is :65452
the voltage is :3.29
the value is :65408
the voltage is :3.29
the value is :65412
the voltage is :3.29
the value is :65400
the voltage is :3.29
the value is :65444
the voltage is :3.29
the value is :12468
the voltage is :0.62
the value is :12488
the voltage is :0.62
the value is :12472
the voltage is :0.62
the value is :12468
the voltage is :0.62
the value is :12464
the voltage is :0.62
```

SPI Example

Introduction

The SPI example is based on the **RT-Thread SPI device driver framework**. It performs a loopback test on the `spi1` bus, defaulting to 1 MHz, Mode 0, 8-bit data width. MOSI and MISO must be shorted before running the test.

Hardware Parameters

Parameter	Specification	Note
SPI bus	`spi1`	RA8 hardware SPI

Parameter	Specification	Note
Attached device	`spi10`	Created via `rt_hw_spi_device_attach`
Data width	8 bit	Configurable 1–16 bit
Maximum clock	25 MHz (depends on system clock)	Example uses 1 MHz
Mode	Mode 0 (CPOL=0, CPHA=0)	Master + MSB first
Chip select	Software (`RT_SPI_NO_CS`)	No CS needed for loopback
MOSI/MISO	P708 / P709	Raspberry Pi expansion header

Usage Steps

1. Short P708 (MOSI) and P709 (MISO) as required by the wiring.
2. Enter `spi_loop_test` to run the basic SPI loopback example.

Key Code

The example source is in `src/rt_example_spi.c`.

```
#define SPI_BUS_NAME        "spi1"
#define SPI_NAME            "spi10"

void spi_loop_test(void)
{
    static uint8_t sendbuf[1024];
    static uint8_t readbuf[1024];

    /* 1. Prepare test data 0x00 ~ 0xFF cyclic */
    for (int i = 0; i < (int) sizeof(sendbuf); i++)
        sendbuf[i] = (uint8_t) i;

    /* 2. Attach SPI device and configure */
    rt_hw_spi_device_attach(SPI_BUS_NAME, SPI_NAME, RT_NULL);
    struct rt_spi_configuration cfg = {
        .data_width = 8,
```

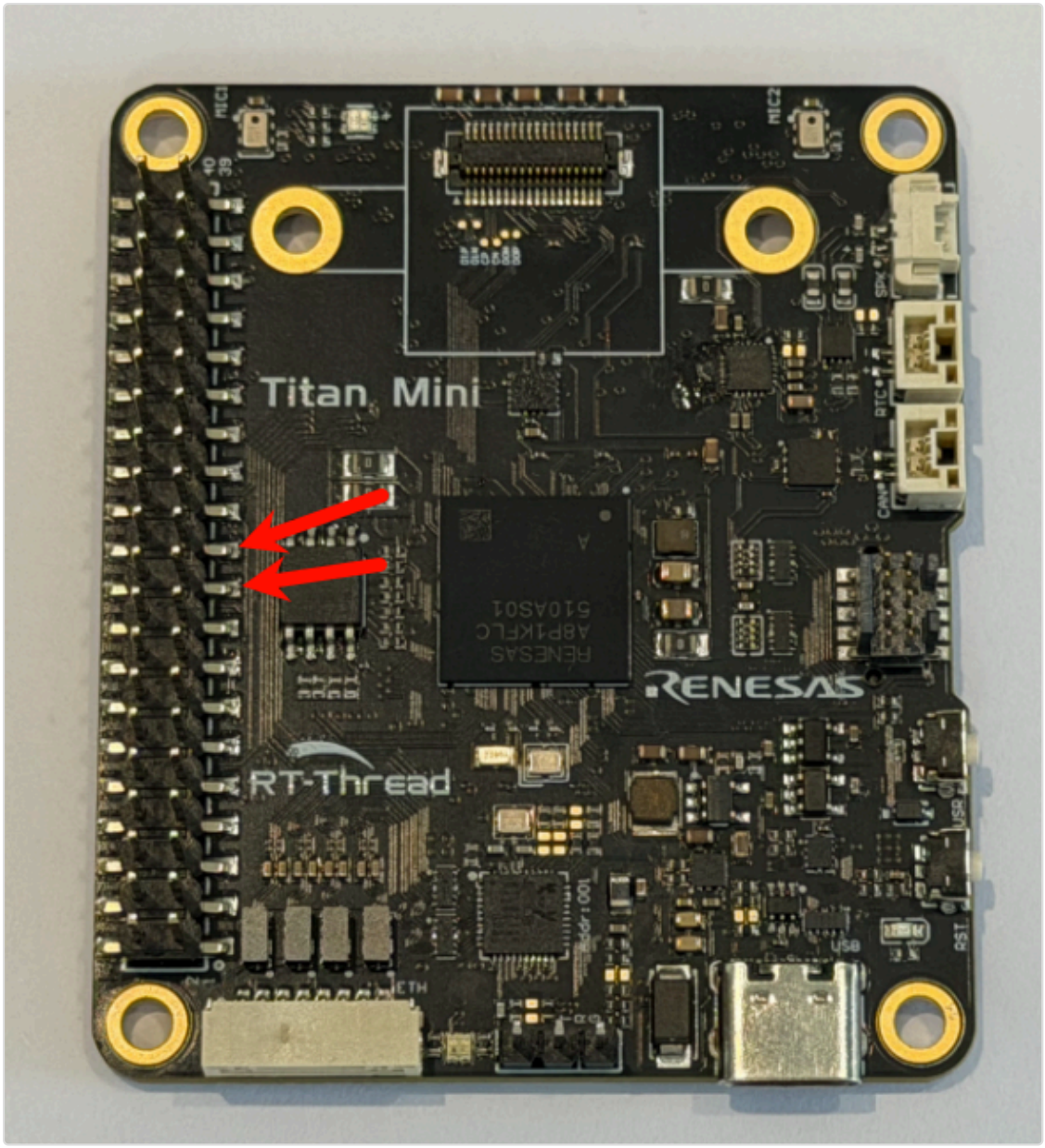
```
        .mode = RT_SPI_MASTER | RT_SPI_MODE_0 | RT_SPI_MSB | RT
        .max_hz = 1 * 1000 * 1000,                 // 1 MHz
    };
    struct rt_spi_device *spi_dev = (struct rt_spi_device *) rt
    rt_spi_configure(spi_dev, &cfg);

    /* 3. Full-duplex transfer + byte-by-byte compare */
    rt_spi_transfer(spi_dev, sendbuf, readbuf, sizeof(sendbuf))
    for (int i = 0; i < (int) sizeof(readbuf); i++)
    {
        if (readbuf[i] != sendbuf[i])
        {
            rt_kprintf("SPI test fail!!!\n");
            break;
        }
    }
    rt_kprintf("\n\nSPI test end\n");
}
MSH_CMD_EXPORT(spi_loop_test, test spi1);
```

Result

- `spi_loop_test` prints the contents of the send and receive buffers. When all bytes match, it outputs `SPI test end`.

Wiring point for shorting P708 (MOSI) and P709 (MISO) to form the loopback:



Terminal output after running `spi_loop_test`:

```
 \ | /
- RT -     Thread Operating System
 / | \     5.1.0 build Mar  3 2026 17:52:47
 2006 - 2024 Copyright by RT-Thread team

Hello RT-Thread!
=====================================================
This example project is a Titan Board Mini template routine!
=====================================================
msh >spi_loop_test
spi10 send:
00 01 02 03 04 05 06 07 08 09 0a 0b 0c 0d 0e 0f 10 11 12 13 14 15 16 17 18 19 1a 1b 1c 1d 1e 1f 20 21 22 23 24 25 26 27 28 29 2a 2b 2c 2d 2e 2f 30 31 32 33 34 35 36 37 38 39 3a 3b 3c 3d 3e 3f 40 41 42 43 44 45 46
 47 48 49 4a 4b 4c 4d 4e 4f 50 51 52 53 54 55 56 57 58 59 5a 5b 5c 5d 5e 5f 60 61 62 63 64 65 66 67 68 69 6a 6b 6c 6d 6e 6f 70 71 72 73 74 75 76 77 78 79 7a 7b 7c 7d 7e 7f 80 81 82 83 84 85 86 87 88 89 8a 8b 8c 8
d 8e 8f 90 91 92 93 94 95 96 97 98 99 9a 9b 9c 9d 9e 9f a0 a1 a2 a3 a4 a5 a6 a7 a8 a9 aa ab ac ad ae af b0 b1 b2 b3 b4 b5 b6 b7 b8 b9 ba bb bc bd be bf c0 c1 c2 c3 c4 c5 c6 c7 c8 c9 ca cb cc cd ce cf d0 d1 d2 d3
d4 d5 d6 d7 d8 d9 da db dc dd de df e0 e1 e2 e3 e4 e5 e6 e7 e8 e9 ea eb ec ed ee ef f0 f1 f2 f3 f4 f5 f6 f7 f8 f9 fa fb fc fd fe ff 00 01 02 03 04 05 06 07 08 09 0a 0b 0c 0d 0e 0f 10 11 12 13 14 15 16 17 18 19 1a
 1b 1c 1d 1e 1f 20 21 22 23 24 25 26 27 28 29 2a 2b 2c 2d 2e 2f 30 31 32 33 34 35 36 37 38 39 3a 3b 3c 3d 3e 3f 40 41 42 43 44 45 46 47 48 49 4a 4b 4c 4d 4e 4f 50 51 52 53 54 55 56 57 58 59 5a 5b 5c 5d 5e 5f 60 6
1 62 63 64 65 66 67 68 69 6a 6b 6c 6d 6e 6f 70 71 72 73 74 75 76 77 78 79 7a 7b 7c 7d 7e 7f 80 81 82 83 84 85 86 87 88 89 8a 8b 8c 8d 8e 8f 90 91 92 93 94 95 96 97 98 99 9a 9b 9c 9d 9e 9f a0 a1 a2 a3 a4 a5 a6 a7
 a8 a9 aa ab ac ad ae af b0 b1 b2 b3 b4 b5 b6 b7 b8 b9 ba bb bc bd be bf c0 c1 c2 c3 c4 c5 c6 c7 c8 c9 ca cb cc cd ce cf d0 d1 d2 d3 d4 d5 d6 d7 d8 d9 da db dc dd de df e0 e1 e2 e3 e4 e5 e6 e7 e8 e9 ea eb ec ee
 ef f0 f1 f2 f3 f4 f5 f6 f7 f8 f9 fa fb fc fd fe ff 00 01 02 03 04 05 06 07 08 09 0a 0b 0c 0d 0e 0f 10 11 12 13 14 15 16 17 18 19 1a 1b 1c 1d 1e 1f 20 21 22 23 24 25 26 27 28 29 2a 2b 2c 2d 2e 2f 30 31 32 33 34 3
5 36 37 38 39 3a 3b 3c 3d 3e 3f 40 41 42 43 44 45 46 47 48 49 4a 4b 4c 4d 4e 4f 50 51 52 53 54 55 56 57 58 59 5a 5b 5c 5d 5e 5f 60 61 62 63 64 65 66 67 68 69 6a 6b 6c 6d 6e 6f 70 71 72 73 74 75 76 77 78 79 7a 7b
7c 7d 7e 7f 80 81 82 83 84 85 86 87 88 89 8a 8b 8c 8d 8e 8f 90 91 92 93 94 95 96 97 98 99 9a 9b 9c 9d 9e 9f a0 a1 a2 a3 a4 a5 a6 a7 a8 a9 aa ab ac ad ae af b0 b1 b2 b3 b4 b5 b6 b7 b8 b9 ba bb bc bd be bf c0 c1 c2
 c3 c4 c5 c6 c7 c8 c9 ca cb cc cd ce cf d0 d1 d2 d3 d4 d5 d6 d7 d8 d9 da db dc dd de df e0 e1 e2 e3 e4 e5 e6 e7 e8 e9 ea eb ec ed ee ef f0 f1 f2 f3 f4 f5 f6 f7 f8 f9 fa fb fc fd fe ff 00 01 02 03 04 05 06 07 08 0
9 0a 0b 0c 0d 0e 0f 10 11 12 13 14 15 16 17 18 19 1a 1b 1c 1d 1e 1f 20 21 22 23 24 25 26 27 28 29 2a 2b 2c 2d 2e 2f 30 31 32 33 34 35 36 37 38 39 3a 3b 3c 3d 3e 3f 40 41 42 43 44 45 46 47 48 49 4a 4b 4c 4d 4e 4f
50 51 52 53 54 55 56 57 58 59 5a 5b 5c 5d 5e 5f 60 61 62 63 64 65 66 67 68 69 6a 6b 6c 6d 6e 6f 70 71 72 73 74 75 76 77 78 79 7a 7b 7c 7d 7e 7f 80 81 82 83 84 85 86 87 88 89 8a 8b 8c 8d 8e 8f 90 91 92 93 94 95 96
 97 98 99 9a 9b 9c 9d 9e 9f a0 a1 a2 a3 a4 a5 a6 a7 a8 a9 aa ab ac ad ae af b0 b1 b2 b3 b4 b5 b6 b7 b8 b9 ba bb bc bd be bf c0 c1 c2 c3 c4 c5 c6 c7 c8 c9 ca cb cc cd ce cf d0 d1 d2 d3 d4 d5 d6 d7 d8 d9 da db dc d
d de df e0 e1 e2 e3 e4 e5 e6 e7 e8 e9 ea eb ec ed ee ef f0 f1 f2 f3 f4 f5 f6 f7 f8 f9 fa fb fc fd fe ff

spi10 rcv:
00 01 02 03 04 05 06 07 08 09 0a 0b 0c 0d 0e 0f 10 11 12 13 14 15 16 17 18 19 1a 1b 1c 1d 1e 1f 20 21 22 23 24 25 26 27 28 29 2a 2b 2c 2d 2e 2f 30 31 32 33 34 35 36 37 38 39 3a 3b 3c 3d 3e 3f 40 41 42 43 44 45 46
 47 48 49 4a 4b 4c 4d 4e 4f 50 51 52 53 54 55 56 57 58 59 5a 5b 5c 5d 5e 5f 60 61 62 63 64 65 66 67 68 69 6a 6b 6c 6d 6e 6f 70 71 72 73 74 75 76 77 78 79 7a 7b 7c 7d 7e 7f 80 81 82 83 84 85 86 87 88 89 8a 8b 8c 8
d 8e 8f 90 91 92 93 94 95 96 97 98 99 9a 9b 9c 9d 9e 9f a0 a1 a2 a3 a4 a5 a6 a7 a8 a9 aa ab ac ad ae af b0 b1 b2 b3 b4 b5 b6 b7 b8 b9 ba bb bc bd be bf c0 c1 c2 c3 c4 c5 c6 c7 c8 c9 ca cb cc cd ce cf d0 d1 d2 d3
d4 d5 d6 d7 d8 d9 da db dc dd de df e0 e1 e2 e3 e4 e5 e6 e7 e8 e9 ea eb ec ed ee ef f0 f1 f2 f3 f4 f5 f6 f7 f8 f9 fa fb fc fd fe ff 00 01 02 03 04 05 06 07 08 09 0a 0b 0c 0d 0e 0f 10 11 12 13 14 15 16 17 18 19 1a
 1b 1c 1d 1e 1f 20 21 22 23 24 25 26 27 28 29 2a 2b 2c 2d 2e 2f 30 31 32 33 34 35 36 37 38 39 3a 3b 3c 3d 3e 3f 40 41 42 43 44 45 46 47 48 49 4a 4b 4c 4d 4e 4f 50 51 52 53 54 55 56 57 58 59 5a 5b 5c 5d 5e 5f 60 6
1 62 63 64 65 66 67 68 69 6a 6b 6c 6d 6e 6f 70 71 72 73 74 75 76 77 78 79 7a 7b 7c 7d 7e 7f 80 81 82 83 84 85 86 87 88 89 8a 8b 8c 8d 8e 8f 90 91 92 93 94 95 96 97 98 99 9a 9b 9c 9d 9e 9f a0 a1 a2 a3 a4 a5 a6 a7
 a8 a9 aa ab ac ad ae af b0 b1 b2 b3 b4 b5 b6 b7 b8 b9 ba bb bc bd be bf c0 c1 c2 c3 c4 c5 c6 c7 c8 c9 ca cb cc cd ce cf d0 d1 d2 d3 d4 d5 d6 d7 d8 d9 da db dc dd de df e0 e1 e2 e3 e4 e5 e6 e7 e8 e9 ea eb ec ee
 ef f0 f1 f2 f3 f4 f5 f6 f7 f8 f9 fa fb fc fd fe ff 00 01 02 03 04 05 06 07 08 09 0a 0b 0c 0d 0e 0f 10 11 12 13 14 15 16 17 18 19 1a 1b 1c 1d 1e 1f 20 21 22 23 24 25 26 27 28 29 2a 2b 2c 2d 2e 2f 30 31 32 33 34 3
5 36 37 38 39 3a 3b 3c 3d 3e 3f 40 41 42 43 44 45 46 47 48 49 4a 4b 4c 4d 4e 4f 50 51 52 53 54 55 56 57 58 59 5a 5b 5c 5d 5e 5f 60 61 62 63 64 65 66 67 68 69 6a 6b 6c 6d 6e 6f 70 71 72 73 74 75 76 77 78 79 7a 7b
7c 7d 7e 7f 80 81 82 83 84 85 86 87 88 89 8a 8b 8c 8d 8e 8f 90 91 92 93 94 95 96 97 98 99 9a 9b 9c 9d 9e 9f a0 a1 a2 a3 a4 a5 a6 a7 a8 a9 aa ab ac ad ae af b0 b1 b2 b3 b4 b5 b6 b7 b8 b9 ba bb bc bd be bf c0 c1 c2
 c3 c4 c5 c6 c7 c8 c9 ca cb cc cd ce cf d0 d1 d2 d3 d4 d5 d6 d7 d8 d9 da db dc dd de df e0 e1 e2 e3 e4 e5 e6 e7 e8 e9 ea eb ec ed ee ef f0 f1 f2 f3 f4 f5 f6 f7 f8 f9 fa fb fc fd fe ff 00 01 02 03 04 05 06 07 08 0
9 0a 0b 0c 0d 0e 0f 10 11 12 13 14 15 16 17 18 19 1a 1b 1c 1d 1e 1f 20 21 22 23 24 25 26 27 28 29 2a 2b 2c 2d 2e 2f 30 31 32 33 34 35 36 37 38 39 3a 3b 3c 3d 3e 3f 40 41 42 43 44 45 46 47 48 49 4a 4b 4c 4d 4e 4f
50 51 52 53 54 55 56 57 58 59 5a 5b 5c 5d 5e 5f 60 61 62 63 64 65 66 67 68 69 6a 6b 6c 6d 6e 6f 70 71 72 73 74 75 76 77 78 79 7a 7b 7c 7d 7e 7f 80 81 82 83 84 85 86 87 88 89 8a 8b 8c 8d 8e 8f 90 91 92 93 94 95 96
 97 98 99 9a 9b 9c 9d 9e 9f a0 a1 a2 a3 a4 a5 a6 a7 a8 a9 aa ab ac ad ae af b0 b1 b2 b3 b4 b5 b6 b7 b8 b9 ba bb bc bd be bf c0 c1 c2 c3 c4 c5 c6 c7 c8 c9 ca cb cc cd ce cf d0 d1 d2 d3 d4 d5 d6 d7 d8 d9 da db dc d
d de df e0 e1 e2 e3 e4 e5 e6 e7 e8 e9 ea eb ec ed ee ef f0 f1 f2 f3 f4 f5 f6 f7 f8 f9 fa fb fc fd fe ff

SPI test end
msh >
```

CANFD Example

Introduction

The CANFD example is based on the **RT-Thread CAN device driver framework** and the **RA8P1 CANFD controller**. It uses an interrupt-driven + semaphore mechanism to implement asynchronous transmit/receive. The `canfd_test` command spawns independent TX/RX threads that continuously send and receive CAN frames.

Hardware Parameters

Parameter	Specification	Note
CANFD channel	`canfd0`	RA8P1 has 2 independent CANFD channels
Arbitration bitrate	Up to 1 Mbps	Classic CAN compatible
Data bitrate	Up to 8 Mbps	CANFD high-speed segment
Frame ID	`0x78` (standard frame)	8-byte data `0x00~0x07`
TX buffers	16 per channel	—
RX FIFOs	2 per channel	Interrupt + semaphore notification
Filters	16 rules per channel	—
Error check	CRC17 / CRC21	Enhanced CRC

Usage Steps

1. Enter `canfd_test` to start the TX/RX threads (after startup, one 8-byte frame with ID `0x78` is sent every second).
2. To interface with an external node, prepare a bus loopback or a peer CAN device to observe received data.

Key Code

The example source is in `src/rt_example_canfd.c`. It uses interrupt reception + a semaphore to wake the receive thread.

RX Callback and RX Thread

```
static struct rt_semaphore g_can_rx_sem;

static rt_err_t can_rx_indicate(rt_device_t dev, rt_size_t size
{
    rt_sem_release(&g_can_rx_sem);          // release semaphor
    return RT_EOK;
}

static void can_rx_thread(void *parameter)
{
    struct rt_can_msg rxmsg = {0};
    rt_device_set_rx_indicate(g_can_dev, can_rx_indicate);

    while (1)
    {
        rxmsg.hdrindex = -1;
        rt_sem_take(&g_can_rx_sem, RT_WAITING_FOREVER);

        if (rt_device_read(g_can_dev, 0, &rxmsg, sizeof(rxmsg))
        {
            rt_kprintf("ID:%x message:", rxmsg.id);
            for (rt_uint8_t i = 0; i < rxmsg.len; i++)
                rt_kprintf("%d ", rxmsg.data[i]);
            rt_kprintf("\n");
        }
    }
}
```

TX Thread and Initialization

```
static void can_tx_thread(void *parameter)
{
    struct rt_can_msg txmsg = {0};
    txmsg.id  = 0x78;
    txmsg.ide = RT_CAN_STDID;       // standard frame
    txmsg.rtr = RT_CAN_DTR;         // data frame
    txmsg.len = 8;

    while (1)
```

```
    {
        for (rt_uint8_t i = 0; i < txmsg.len; i++)
            txmsg.data[i] = i;

        rt_device_write(g_can_dev, 0, &txmsg, sizeof(txmsg));
        rt_thread_mdelay(1000);
    }
}

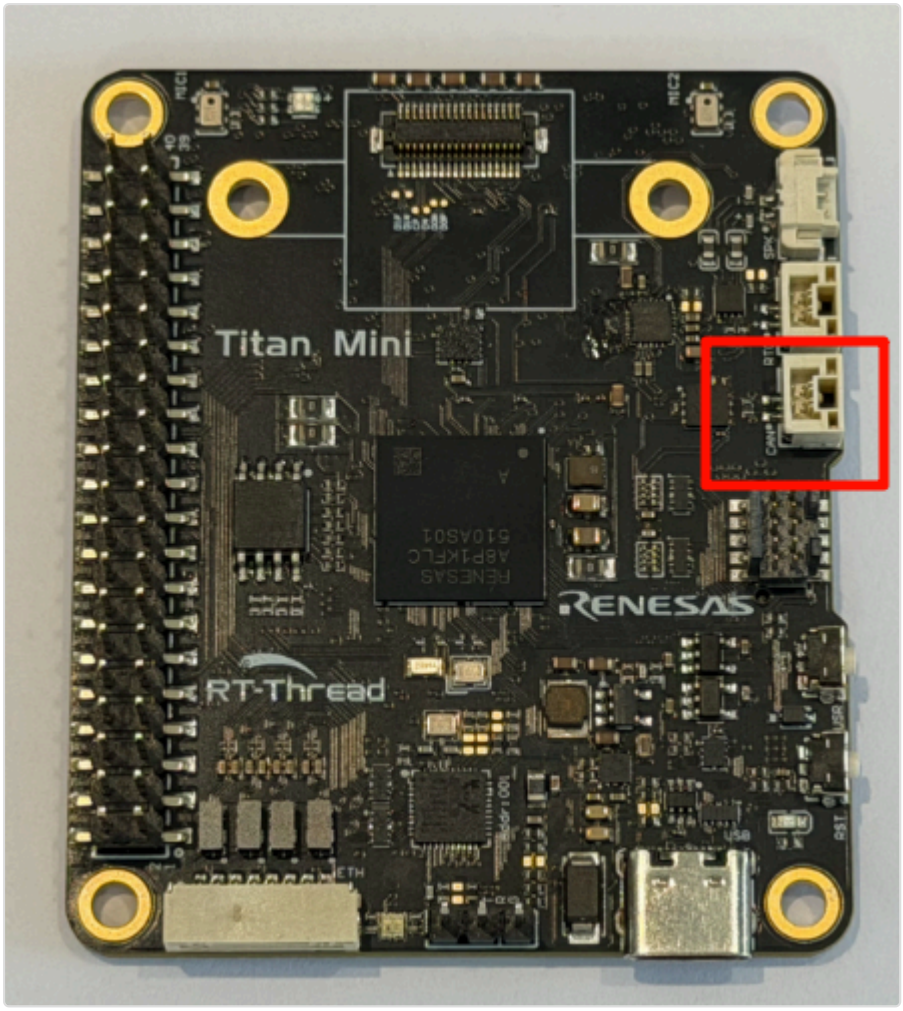
int canfd_test(void)
{
    rt_sem_init(&g_can_rx_sem, "canrx", 0, RT_IPC_FLAG_FIFO);
    g_can_dev = rt_device_find("canfd0");
    rt_device_open(g_can_dev, RT_DEVICE_FLAG_INT_TX | RT_DEVICE

    rt_thread_startup(rt_thread_create("can_rx", can_rx_thread,
    rt_thread_startup(rt_thread_create("can_tx", can_tx_thread,
    return RT_EOK;
}
MSH_CMD_EXPORT(canfd_test, canfd test);
```

Result

- After `canfd_test` starts, it periodically prints `canfd tx ok`, and prints `ID:<id> message:<data...>` when a frame is received.

CAN connector location on the board (note: CAN_H / CAN_L must not be swapped):



Terminal TX/RX output after running `canfd_test`:

```
msh />canfd_test
cacanfd tx ok
nfd tx ok
arted
msh />canfd tx ok
canfd tx ok
canfd tx ok
canfd tx ok
canfd tx ok
canfd tx ok
canfd tx ok
canfd tx ok
ID:78 message:0 1 2 3 4 5 6 7
canfd tx ok
ID:78 message:0 1 2 3 4 5 6 7
canfd tx ok
ID:78 message:0 1 2 3 4 5 6 7
canfd tx ok
ID:78 message:0 1 2 3 4 5 6 7
canfd tx ok
ID:78 message:0 1 2 3 4 5 6 7
canfd tx ok
```

I2C Example

Introduction

The I2C example is based on the **RT-Thread I2C device driver framework**. It reads the on-board **LSM6DS3TR-C** 6-axis IMU sensor (accelerometer + gyroscope + temperature) via the on-board software I2C bus `i2c1`, and prints the parsed physical quantities along with a static attitude estimate — a complete I2C sensor read-out example.

Hardware Parameters

Parameter	Specification	Note
I2C bus	`i2c1`	On-board software I2C (SCL=PIN1292, SDA=PIN1291)
Target device	LSM6DS3TR-C	On-board 6-axis IMU
Device address	`0x6A`	7-bit address, WHO_AM_I=0x6A
Accel full scale	±2g	ODR 12.5 Hz, 1 LSB ≈ 0.061 mg
Gyro full scale	±2000 dps	ODR 12.5 Hz, 1 LSB ≈ 70 mdps
Temperature precision	0.0625 ℃ / LSB	Offset 25 ℃

Usage Steps

1. Confirm that the on-board LSM6DS3TR-C IMU is wired to `i2c1` (SDA/SCL/GND/VCC; the dev board is shipped pre-connected).
2. Enter `i2c_sample`. It will automatically perform: device probe → register configuration → one-shot data acquisition → parsing and printing.

Key Code

The example source is in `src/rt_example_i2c.c`. The core flow is: first verify WHO_AM_I, then configure the CTRL1_XL / CTRL2_G / CTRL3_C

registers, and finally read STATUS_REG to check data-ready and combine 6 bytes of acceleration / 6 bytes of angular rate / 2 bytes of temperature in little-endian order.

```
#define I2C_BUS_NAME            "i2c1"
#define LSM6DS3TR_C_I2C_ADDR    0x6A
#define LSM6DS3TR_C_WHO_AM_I    0x0F

/* Register read: write register address first, then read N byt
static rt_err_t imu_reg_read(rt_uint8_t reg, rt_uint8_t *buf, r
{
    struct rt_i2c_msg msgs[2];
    msgs[0].addr  = LSM6DS3TR_C_I2C_ADDR;
    msgs[0].flags = RT_I2C_WR;
    msgs[0].buf   = &reg;
    msgs[0].len   = 1;
    msgs[1].addr  = LSM6DS3TR_C_I2C_ADDR;
    msgs[1].flags = RT_I2C_RD;
    msgs[1].buf   = buf;
    msgs[1].len   = len;
    if (rt_i2c_transfer(i2c_bus, msgs, 2) != 2)
        return -RT_ERROR;
    return RT_EOK;
}

void i2c_sample(void)
{
    if (imu_init() != RT_EOK)              /* WHO_AM_I check +
        return;
    imu_read_and_print();                  /* read accel/gyro/t
}
MSH_CMD_EXPORT(i2c_sample, read LSM6DS3TR-C IMU sensor via i2c1
```

Result

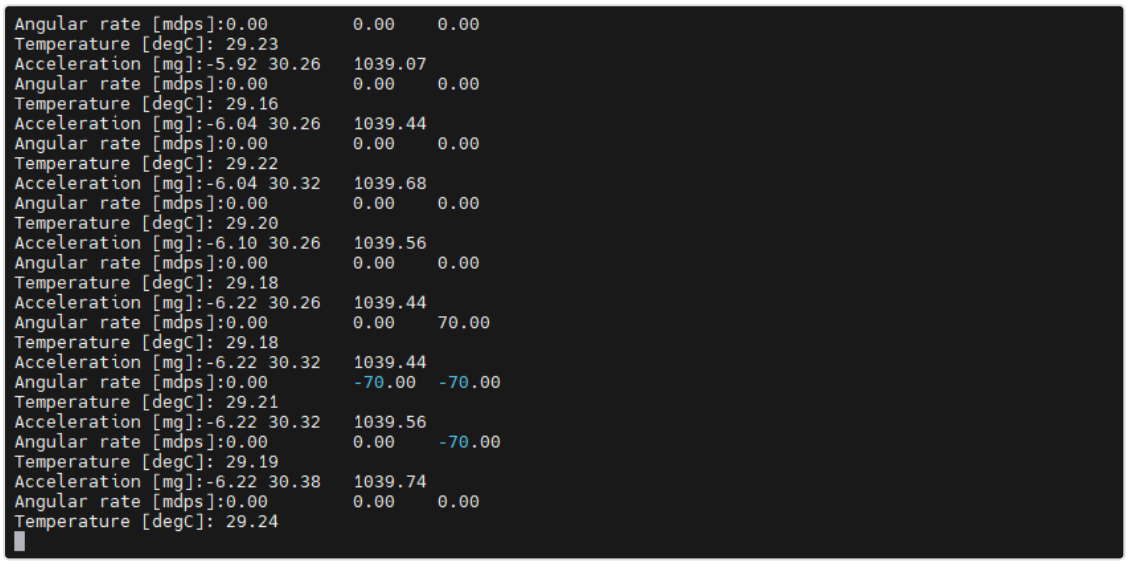
On success, a complete IMU sample is printed, including acceleration (mg), angular rate (dps), temperature (℃), and the pitch / roll estimated from the gravity direction:

```
[I/i2c.imu] LSM6DS3TR-C detected, WHO_AM_I=0x6A
[I/i2c.imu] IMU configured: ODR=12.5Hz, accel ±2g, gyro ±2000dp
------ LSM6DS3TR-C IMU Sample ------
Accel  (mg) : X=   -8.5  Y=   12.3  Z= 1003.4  | |a|= 1003.5
Gyro   (dps): X=   0.12  Y=   -0.05  Z=   0.08  | |w|=   0.15
Temp   (C)  :   28.50
```

```
Tilt   (deg): pitch=  -0.49  roll=   0.70  (static estimate)
-------------------------------------
```

On failure, possible messages:

- `cannot find i2c bus i2c1`: software I2C bus not registered (check `RT_USING_SOFT_I2C1`).
- `IMU not found, WHO_AM_I=0xXX`: sensor did not respond or address mismatch; check wiring / power.



Watchdog Example

Introduction

The Watchdog example is based on the **RT-Thread watchdog device framework** and the **RA8P1 WDT hardware**. It verifies that the `wdt` device can start correctly, and uses an independent thread to simulate the scenario of "feeding the dog normally 10 times, then stopping" to demonstrate the watchdog timeout reset mechanism.

Hardware Parameters

Parameter	Specification	Note
WDT device	`wdt`	RA8P1 on-chip independent watchdog

Parameter	Specification	Note
Independent operation	Yes	Still counts when CPU is halted
Timeout	128/512/.../16384 cycles	Configurable via FSP
Clock divider	1/4/16/.../8192	Multiple options
Reset mode	Reset mode / NMI mode	Reset mode auto restarts
Feed method	`RT_DEVICE_CTRL_WDT_KEEPALIVE`	Software control command
Feed interval	1000 ms	Defined by `WDT_FEED_INTERVAL`

Usage Steps

1. Enter `wdt_sample` to start the watchdog example.
2. Observe the first 10 feed logs (once per second).
3. From the 11th iteration feeding stops. Keep the terminal connected and observe the system reset.

Key Code

The example source is in `src/rt_example_wdt.c`.

Feed Thread (feed for the first 10 times, then stop)

```
#define WDT_FEED_INTERVAL   1000

static void feed_dog_entry(void *parameter)
{
    int count = 0;
    while (1)
    {
        if (count < 10)
        {
            rt_device_control(g_wdt_dev, RT_DEVICE_CTRL_WDT_KEE
```

```
            rt_kprintf("[FeedDog] Feeding watchdog... %d\n", co
        }
        else
        {
            rt_kprintf("[FeedDog] Simulate exception! Stop feed
        }
        count++;
        rt_thread_mdelay(WDT_FEED_INTERVAL);
    }
}
```

Start Watchdog and Feed Thread

```
int wdt_sample(void)
{
    g_wdt_dev = rt_device_find("wdt");
    if (g_wdt_dev == RT_NULL)
    {
        rt_kprintf("cannot find wdt device!\n");
        return -RT_ERROR;
    }

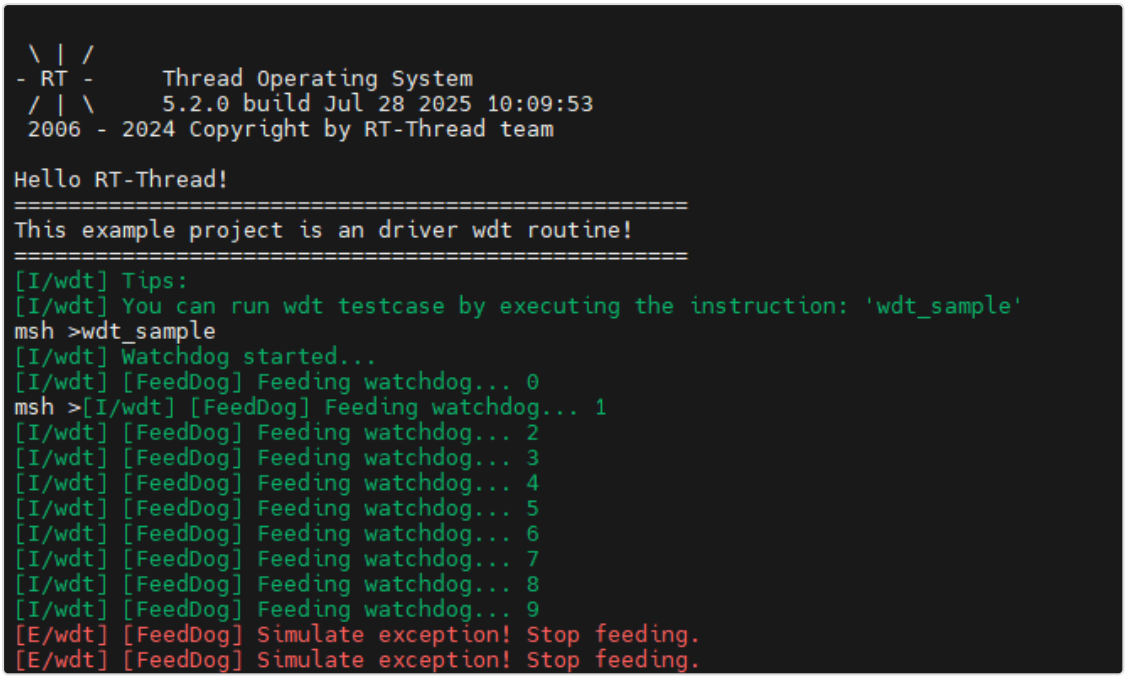
    if (rt_device_control(g_wdt_dev, RT_DEVICE_CTRL_WDT_START,
    {
        rt_kprintf("start watchdog failed!\n");
        return -RT_ERROR;
    }

    g_feed_thread = rt_thread_create("feed_dog", feed_dog_entry
    rt_thread_startup(g_feed_thread);
    rt_kprintf("watchdog sample started\n");
    return RT_EOK;
}
MSH_CMD_EXPORT(wdt_sample, wdt_sample);
```

Result

- On startup it prints `watchdog sample started`.
- For the first 10 iterations it prints `[FeedDog] Feeding watchdog... <n>` once per second.
- After that it keeps printing `[FeedDog] Simulate exception! Stop feeding.`. If the WDT is configured in reset mode, the system will automatically reset after timeout.

Feed and stop-feed output after running `wdt_sample`:



3.2. Ethernet Example Guide

Chinese | **English**

Introduction

This example demonstrates how to use the **Ethernet interface** on **Titan Board Mini**, combined with the **RT-Thread Ethernet driver framework** to implement network communication functions.

Key features include:

- Initialize RA8 series Ethernet hardware
- Configure IP address, subnet mask, and gateway
- Send and receive Ethernet frames
- Integrate RT-Thread `netdev` framework for unified network device management
- Support DMA and interrupts for high-speed data transfer

Ethernet Overview

1. General Introduction

Ethernet is the most widely used Local Area Network (LAN) technology, proposed by Xerox PARC in the 1970s and later standardized by IEEE 802.3. Ethernet has the following characteristics:

- **Data transmission method**: Frame-based packet switching, physically transmitted through twisted pair, fiber optic, or wireless media.
- **Topology**: Traditional Ethernet used bus or star topology, while modern Ethernet mainly adopts star and tree topologies.
- **Protocol layer**: Belongs to the data link layer (Layer 2) and physical layer (Layer 1) technologies of the OSI model.

2. Ethernet Frame Structure

Ethernet uses **frames** as the unit of data transmission. An Ethernet frame consists of the following fields:

Field	Length	Description
Preamble	7 bytes	Used for frame synchronization
Start Frame Delimiter (SFD)	1 byte	Frame start marker, value is 10101011
Destination MAC Address	6 bytes	Receiver hardware address
Source MAC Address	6 bytes	Transmitter hardware address
Type/Length	2 bytes	Upper layer protocol type or frame length
Data Payload	46~1500 bytes	Upper layer data (e.g., IP packet)
CRC Checksum	4 bytes	Cyclic redundancy check for frame integrity

> ***Minimum frame length**: 64 bytes*
> ***Maximum frame length**: 1518 bytes (without VLAN tag)*

RA8 Series Ethernet Features

RA8 series MCUs (such as RA8P1) integrate high-performance **Ethernet MAC**, supporting RGMII, RMII, and MII interfaces, providing stable and reliable high-speed network communication capabilities. The MAC can work with external PHY and is compatible with the LwIP TCP/IP protocol stack.

1. Network Interface Features

1. **Interface Types**

- **RMII (Reduced Media Independent Interface)**: Pin-saving, supports 10/100 Mbps

- **MII (Media Independent Interface)**: Standard interface, supports 10/100 Mbps
- **RGMII (Reduced Gigabit MII)**: Supports 10/100/1000 Mbps, high-speed interface for Gigabit Ethernet

2. **PHY Connection**

- External PHY connected via MDC/MDIO interface
- Supports auto-negotiation of speed and duplex mode
- Can read and write PHY registers for configuration and status monitoring

2. MAC Features

1. **Duplex and Speed Support**

- Full/half duplex
- Supports 10/100/1000 Mbps (RGMII)
- Supports auto-negotiation or forced configuration

2. **Frame Processing**

- VLAN tag support (IEEE 802.1Q, optional)
- Supports multicast and broadcast frame filtering
- Hardware CRC generation and verification

3. **MAC Address Management**

- Supports single or multiple MAC addresses
- Can be dynamically configured via FSP or software

3. DMA and Buffer Features

1. **Independent TX/RX DMA Engines**

- Supports simultaneous transmission and reception
- Reduces CPU usage and improves throughput

2. **Descriptor Queues**

- Configurable number of TX/RX buffers
- Supports chained DMA for efficient large data transfer

3. **Multi-buffer Management**

- Supports ring buffers for continuous transmission
- Reduces packet loss

4. **Hardware Acceleration**

- Frame filtering, length check, and CRC verification

4. Interrupt Mechanism

1. **Interrupt Types**

- Receive complete (RX)
- Transmit complete (TX)
- Error interrupts (CRC error, buffer overflow)

2. **Interrupt Configuration**

- Priority configurable via FSP
- Supports RT-Thread ISR integration

3. **Optimization**

- RX/TX interrupts can work with DMA
- Selective interrupt enabling for improved performance

5. PHY Management

1. **MDIO Interface**

- Can read/write PHY registers for configuration, reset, and status monitoring

2. **Auto-negotiation**

- Supports speed (10/100/1000 Mbps) and duplex mode auto-negotiation

3. **Link Monitoring**

- Detect link status (Up/Down)
- CRC error and collision detection

6. Protocol and Stack Support

1. **TCP/IP Protocol Stack Integration**

- Compatible with LwIP
- Supports TCP/UDP/ICMP, DHCP client/server, ARP

2. **Application Layer Support**

- Supports Telnet, HTTP, MQTT and other applications
- Multi-thread safe, supports concurrent access

7. Performance and Reliability

1. **Throughput Optimization**

- DMA + interrupts reduce CPU usage
- Adjustable TX/RX buffer sizes

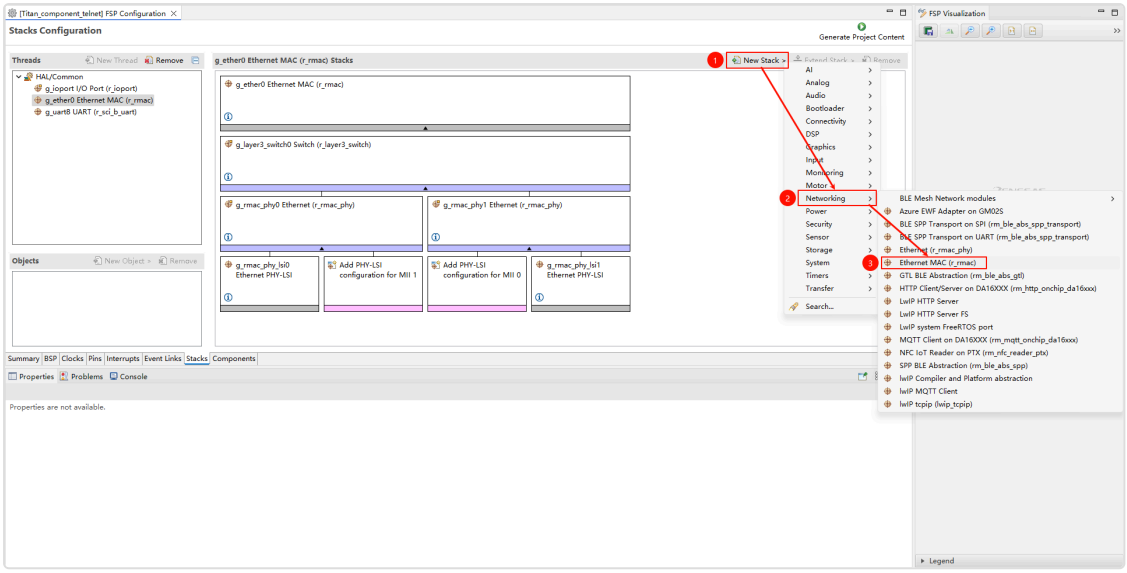
2. **Reliability Features**

- Hardware CRC checksum
- VLAN and multicast filtering reduce interference
- Link detection and auto-reconnection

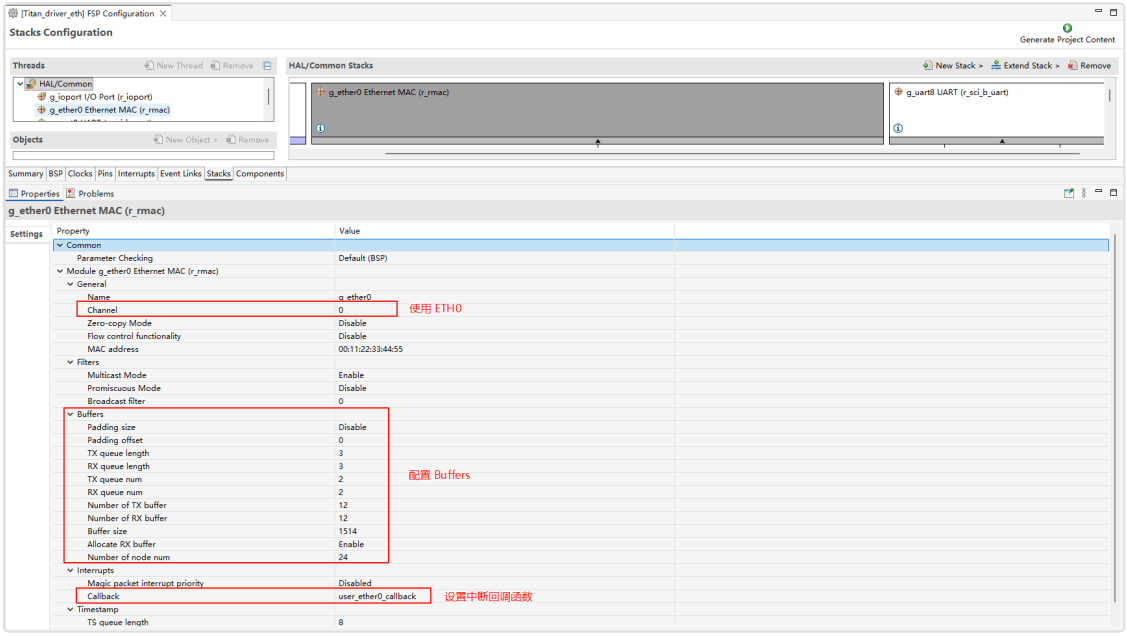
FSP Configuration

> ***Note:*** *This project uses FSP version 6.4.0. Please use FSP 6.4.0 when configuring FSP features.*

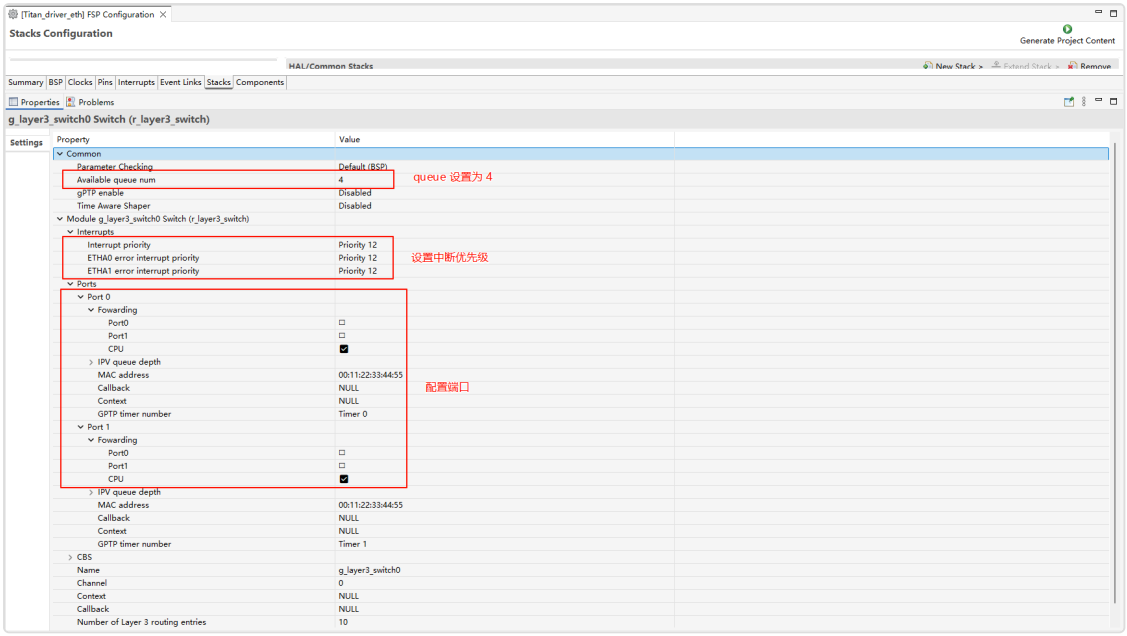
- Create new r_rmac stack:



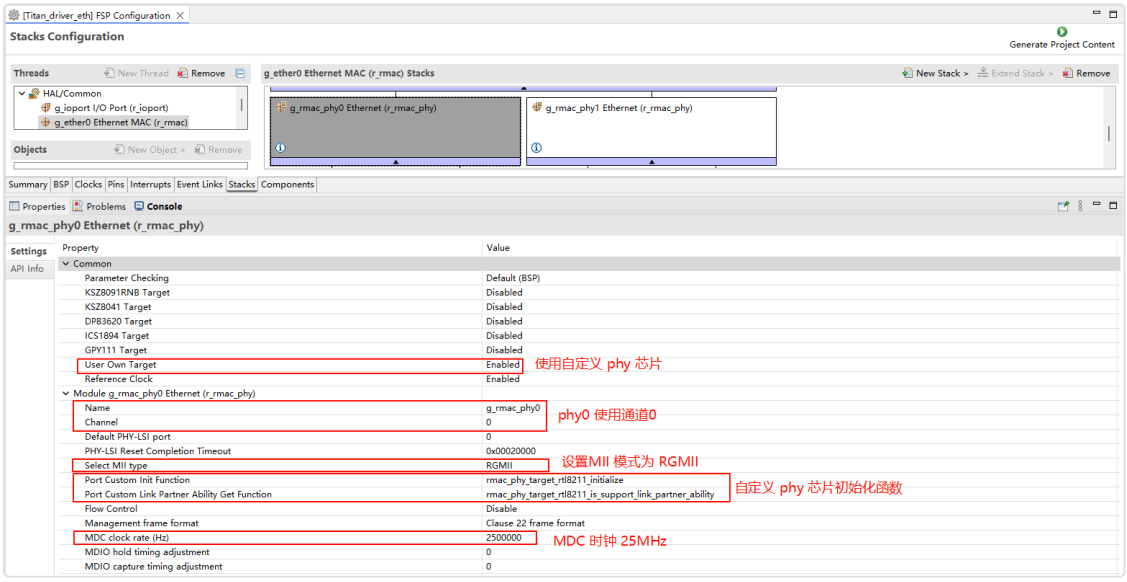
- Configure r_mac stack:



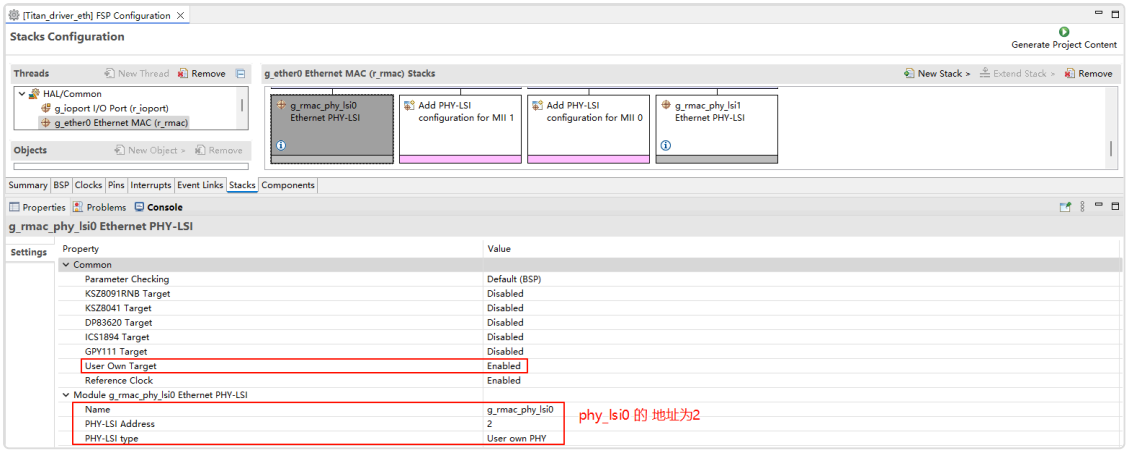
- Configure r_layer3_switch:



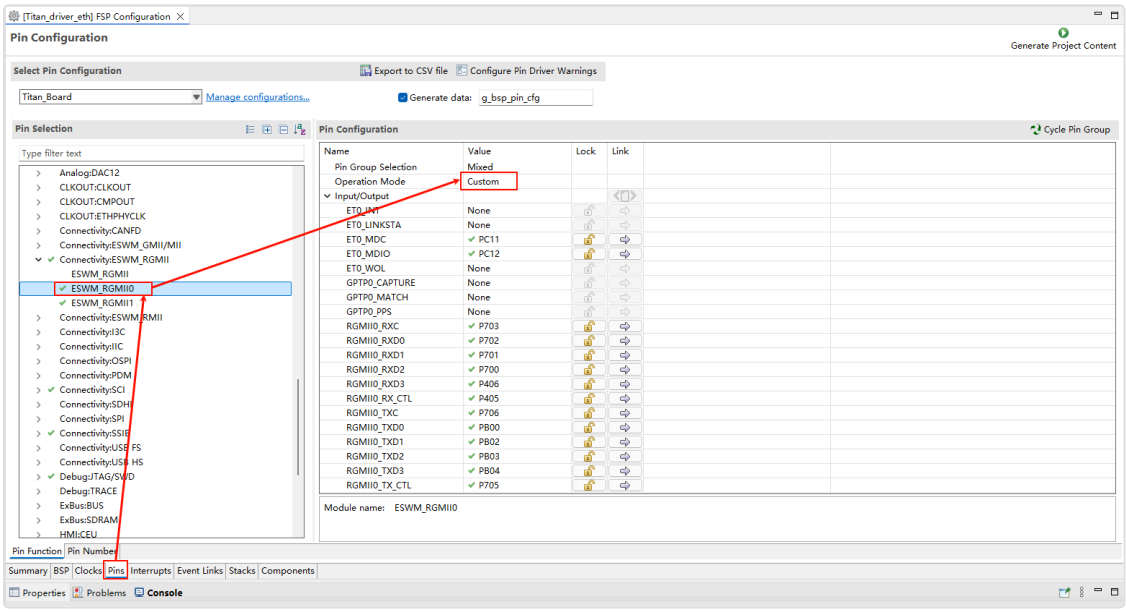
- Configure r_rmac_phy:



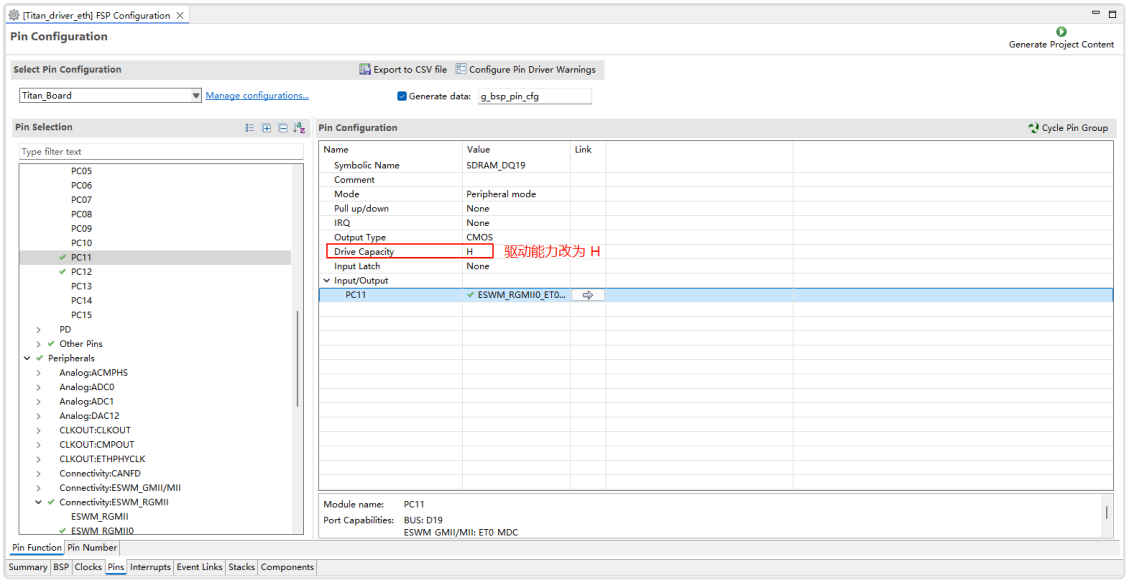
- Configure g_rmac_phy_lsi0:



- ETH0 pin configuration:

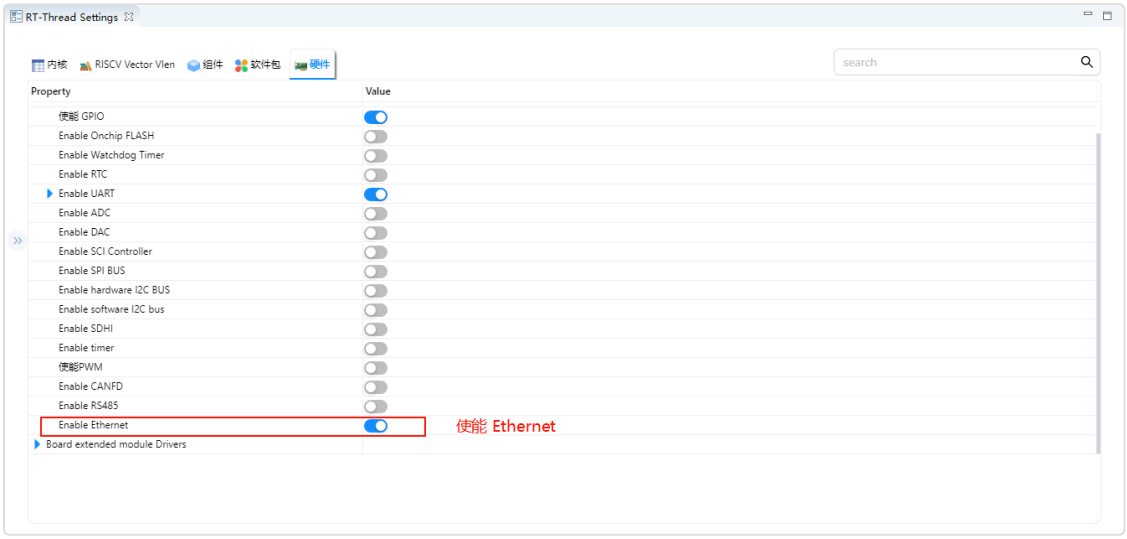


- **Note**: All ETH-related pins need to have their drive strength changed to H.



RT-Thread Settings Configuration

- Enable Ethernet in RT-Thread Settings.



Software Description

LwIP Performance Tuning Recommendations

The LwIP defaults in the upstream `rt-thread/components/net/lwip/Kconfig` are conservative (PBUF count 16, TCP send buffer 8196, etc.), which cannot fully exploit the throughput of the RA8P1 Gigabit Ethernet + DMA. After enabling `RT_USING_LWIP` (e.g. by

ticking `BSP_USING_ETH`), it is recommended to manually tune the following values:

Option	Default	Note
`RT_LWIP_PBUF_NUM`	256	PBUF pool count (upstream default 16)
`RT_LWIP_RAW_PCB_NUM`	4	RAW socket count
`RT_LWIP_UDP_PCB_NUM`	24	UDP PCB count (upstream default 4)
`RT_LWIP_TCP_PCB_NUM`	24	TCP PCB count (upstream default 4)
`RT_LWIP_TCP_SEG_NUM`	512	TCP segment count (upstream default 40)
`RT_LWIP_TCP_SND_BUF`	65535	TCP send buffer (upstream default 8196)
`RT_LWIP_TCP_WND`	65535	TCP window size (upstream default 8196)
`RT_LWIP_TCPTHREAD_PRIORITY`	6	lwIP main thread priority (upstream default 10; lower number = higher priority)
`RT_LWIP_TCPTHREAD_MBOX_SIZE`	144	lwIP main thread mailbox size (upstream default 8)
`RT_LWIP_TCPTHREAD_STACKSIZE`	2048	lwIP main thread stack (upstream default 1024)
`LWIP_NO_TX_THREAD`	y	No standalone TX thread (send path invoked directly to reduce context switches)
`RT_LWIP_ETHTHREAD_PRIORITY`	5	Ethernet thread priority (upstream default 12)

Option	Default	Note
`RT_LWIP_ETHTHREAD_STACKSIZE`	2048	Ethernet thread stack (upstream default 1024)
`RT_LWIP_ETHTHREAD_MBOX_SIZE`	144	Ethernet thread mailbox size (upstream default 8)

To apply these recommended values, open `RT-Thread Settings → RT-Thread Components → Network → lwIP` in RT-Thread Studio, or edit the project's `.config` directly.

> ⚠️ ***Memory footprint note**: These options significantly affect RAM usage. `RT_LWIP_PBUF_NUM=256` combined with `TCP_SND_BUF/WND=65535` reserves a substantial amount of heap memory at the default `RT_LWIP_PBUF_POOL_BUFSIZE`. Make sure your linker script reserves a large enough heap (≥ 256 KB recommended).*

RTL8211 PHY Initialization

The Ethernet PHY chip initialization function is in `./board/ports/drv_rtl8211.c`:

```
void rmac_phy_target_rtl8211_initialize (rmac_phy_instance_ctrl
{
#define RTL_8211F_PAGE_SELECT 0x1F
#define RTL_8211F_EEELCR_ADDR 0x11
#define RTL_8211F_LED_PAGE 0xD04
#define RTL_8211F_LCR_ADDR 0x10

    uint32_t val1, val2 = 0;

    /* switch to led page */
    R_RMAC_PHY_Write(phydev, RTL_8211F_PAGE_SELECT, RTL_8211F_L

    /* set led1(green) Link 10/100/1000M, and set led2(yellow)
    R_RMAC_PHY_Read(phydev, RTL_8211F_LCR_ADDR, &val1);
    val1 |= (1 << 5);
    val1 |= (1 << 8);
    val1 &= (~(1 << 9));
    val1 |= (1 << 10);
```

```
    val1 |= (1 << 11);
    R_RMAC_PHY_Write(phydev, RTL_8211F_LCR_ADDR, val1);

    /* set led1(green) EEE LED function disabled so it can keep
    R_RMAC_PHY_Read(phydev, RTL_8211F_EEELCR_ADDR, &val2);
    val2 &= (~(1 << 2));
    R_RMAC_PHY_Write(phydev, RTL_8211F_EEELCR_ADDR, val2);

    /* switch back to page0 */
    R_RMAC_PHY_Write(phydev, RTL_8211F_PAGE_SELECT, 0xa42);
}

bool rmac_phy_target_rtl8211_is_support_link_partner_ability (r
                                                              ui
{
    FSP_PARAMETER_NOT_USED(p_instance_ctrl);
    FSP_PARAMETER_NOT_USED(line_speed_duplex);

    /* This PHY-LSI supports half and full duplex mode. */
    return true;
}
```

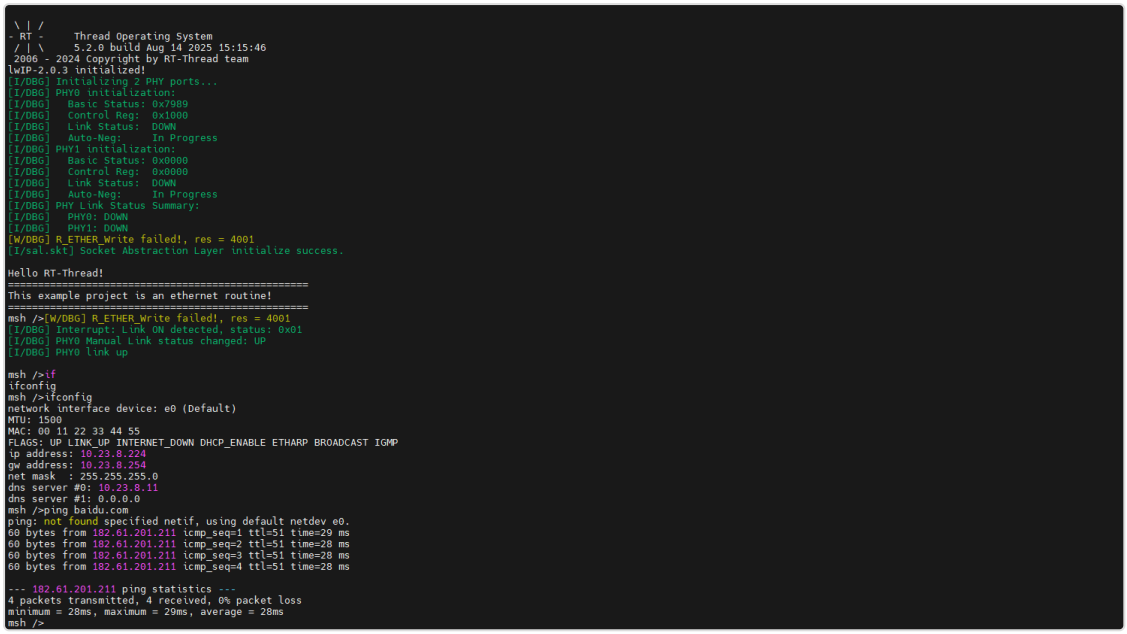
Build & Download

- RT-Thread Studio: Download the Titan Board resource package from the package manager in RT-Thread Studio, then create a new project and build it.

After compilation is complete, connect the development board's USB-DBG interface to the PC, then download the firmware to the development board.

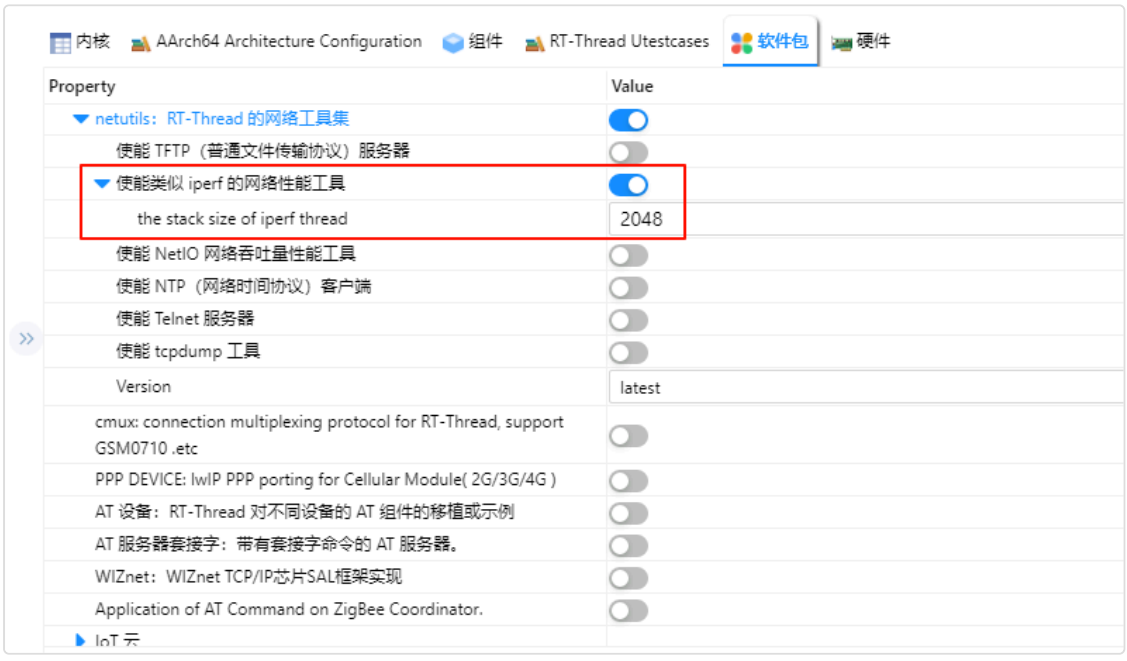
Running Effect

Insert the network cable into the Ethernet port, press the reset button to restart the development board. After waiting for PHY0 link up, enter `ifconfig` to view the IP address obtained by the development board, then enter `ping baidu.com` command for connectivity testing.



iperf Test

Open RT-Thread Settings, add the netutils package and enable the iperf tool.

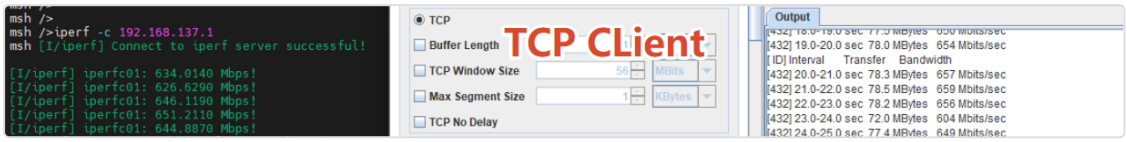


After compilation and download, enter `iperf -c host_IP -p 5001` in the serial terminal for iperf testing.

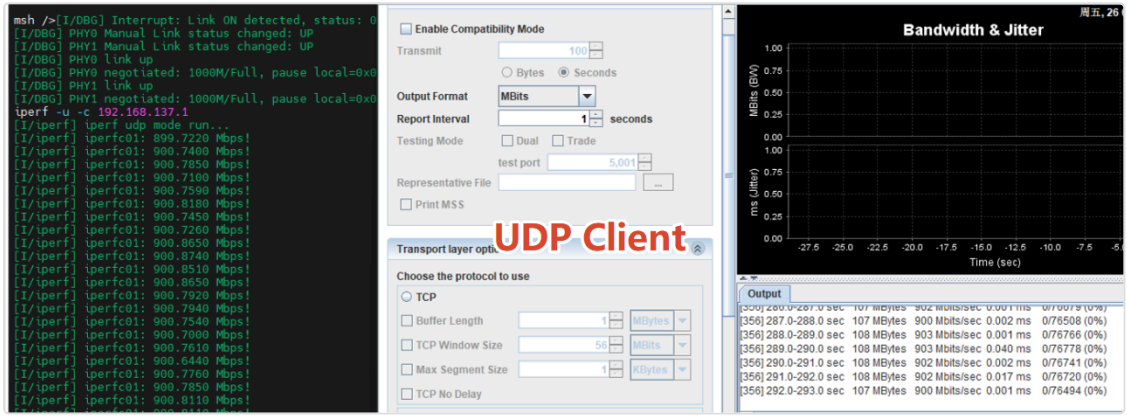
Network Application Examples

With the netutils package, TCP/UDP client and server send/receive tests can be performed.

TCP Client Test: The board acts as a client, actively connecting to a remote host to send and receive data.



UDP Client Test: The board acts as a client, sending to or receiving datagrams from a remote host.



4. Multimedia Display

4.1. MIPI CSI Camera Example Guide

English|Chinese

Introduction

This example demonstrates how to use the **MIPI CSI (Camera Serial Interface)** on the **Titan Board** to connect an **OV5640 camera**, and display the captured images on an **RGB565 LCD screen** via the **RT-Thread LCD framework**.

Note: The current camera example can only be opened with the FSP 6.0.0 configuration tool.

Key functionalities include:

- Initialize the MIPI CSI camera interface to capture real-time video streams
- Configure OV5640 camera parameters (resolution, frame rate, output format)
- Display captured images using the RT-Thread LCD driver
- Support image format conversion (YUV422 → RGB565)

RA8 Series MIPI CSI Features

The RA8 series MCU integrates a **MIPI CSI hardware module** for high-speed, low-power camera data acquisition, suitable for HD video and real-time image processing.

1. Hardware Interface Features

1. **Interface Type**

- **MIPI CSI-2** D-PHY interface for high-bandwidth serial data transmission

- Supports 1–4 data lanes
- Synchronization is handled by MIPI D-PHY; no additional HSYNC/VSYNC required

2. **Data Rate and Resolution**

- Supports up to 1.5–2.5 Gbps per lane (depending on MCU series and PHY configuration)
- Can drive common camera resolutions: VGA, QVGA, SXGA, UXGA, 1080p, etc.

3. **Camera Compatibility**

- Compatible with OV5640, IMX219, and other common CMOS cameras
- Supports camera register configuration and auto-initialization

2. Image Formats and Processing Capabilities

1. **Supported Image Formats**

- **RAW10 / RAW12** (for algorithm development and image processing)
- **YUV422** (for video display)
- **RGB565** (suitable for LCD display)

2. **Image Processing Features**

- **Color space conversion** (YUV → RGB)
- **Image cropping** (ROI capture)
- **Mirror and flip**
- **Hardware accelerated scaling**

3. **Hardware Acceleration**

- MIPI CSI includes built-in DMA to reduce CPU load
- Supports high-speed format conversion and scaling

3. DMA Support and Buffering

1. **High-Speed DMA Transfer**

- Works with MCU DMAC to write images directly to memory or LCD buffer
- Reduces CPU intervention and increases frame rate

2. **Multi-Buffer Mechanism**

- Supports double buffering or ring buffers for continuous video capture
- Prevents frame loss and display latency

3. **Flexible DMA Configuration**

- Configurable buffer start address and size
- Supports interrupt callback handling

4. Interrupt Mechanism

1. **Interrupt Types**

- **Frame End Interrupt**: triggered at the end of each frame capture
- **Line End Interrupt (optional)**
- **Error Interrupt**: buffer overflow, PHY errors, etc.

2. **Interrupt Features**

- Supports RT-Thread ISR callbacks
- Can work with DMA to enable real-time display

5. Timing and Synchronization Features

1. **Synchronization Method**

- CSI relies on MIPI D-PHY for clock and data synchronization
- No additional HSYNC/VSYNC required

2. **Pixel Clock and Data Alignment**

- Supports pixel-aligned or byte-aligned data
- Automatically handles RAW/YUV/RGB data alignment

6. Performance Optimization

1. **High Throughput**

- DMA + double buffering enables continuous video capture
- Low CPU usage suitable for real-time applications

2. **Reliability**

- PHY errors or data loss interrupts can trigger exception handling
- Supports buffer overflow detection

3. **Flexibility**

- Supports multiple resolutions and formats
- ROI capture and hardware scaling improve display efficiency

7. Application Scenarios

- Real-time video display on LCD
- Video capture and processing algorithm verification
- Embedded vision applications such as surveillance, gesture recognition, and robotics vision

RA8 Series MCU GLCDC (Graphics LCD Controller) Features

The RA8 series MCU (e.g., RA8P1) integrates a **GLCDC hardware module** for driving TFT/LCD displays, enabling high-speed graphics rendering and video display, supporting multiple resolutions, color formats, and display modes.

1. Hardware Features

1. **Resolution Support**

- Can drive common resolutions from QVGA (320×240) to WQVGA/XGA
- Limited by on-chip RAM and display interface bandwidth

2. **Color Support**

- Supports 1/4/8/16/24/32-bit color depth
- Common formats: RGB565, RGB888
- Supports palette mode (CLUT)
- Hardware color conversion available

3. **Interface Types**

- Parallel RGB (TFT LCD interface)

- Supports 16/18/24-bit data bus
- Direct connection to external LCD panels
- Programmable timing: HSYNC, VSYNC, DE, PCLK, RGB output

2. Layers and Display Modes

1. **Layer Support**

- Single-layer mode (single screen display)
- Multi-layer mode (palette or hardware alpha blending)
- Supports transparent/semi-transparent overlay

2. **Display Modes**

- RGB mode (direct color output)
- CLUT/Palette mode (indexed color via lookup table)
- Configurable scan direction (horizontal/vertical)

3. DMA and Frame Buffer

1. **Frame Buffer Access**

- GLCDC can directly access on-chip or external SRAM frame buffer
- Supports single or double buffering
- Supports ring buffer for continuous refresh

2. **DMA Support**

- Works with MCU DMAC to reduce CPU usage
- Can transfer images directly from memory to LCD
- Supports line, block, or full-frame transfer

4. Hardware Graphics Functions

1. **Window Cropping and Scaling**

- Can specify display window area
- Supports simple horizontal/vertical scaling

2. **Hardware Graphics Acceleration**

- Rectangle fill, color replacement
- Image transparency processing
- Can be combined with CEU or DMA for video display

3. **Color Format Conversion**

- YCbCr → RGB
- RGB888 → RGB565
- Hardware accelerated to reduce CPU load

5. Interrupt Mechanism

1. **Interrupt Types**

- Frame End
- Line End (optional)
- Access error/overflow

2. **Interrupt Application**

- Integrates with RT-Thread ISR
- Can trigger buffer update or double-buffer swap on frame completion
- Enables smooth animations and video display

6. Performance Optimization

1. **Double Buffer Mechanism**

- Reduces flicker
- CPU can render next frame in background
- GLCDC hardware automatically switches display buffer

2. **Frame Rate Control**

- Programmable clock and line/frame synchronization
- Supports common refresh rates (30fps, 60fps)

3. **CPU Offload**

- Many graphics operations performed in hardware
- DMA + GLCDC combination enables efficient image display

RT-Thread Settings Configuration

- Enable MIPI CSI camera using i2c0; enable RGB565 LCD using pwm7 for backlight output.



Compilation & Download

- **RT-Thread Studio**: In RT-Thread Studio's package manager, download the Titan Board resource package, create a new project, and compile it.

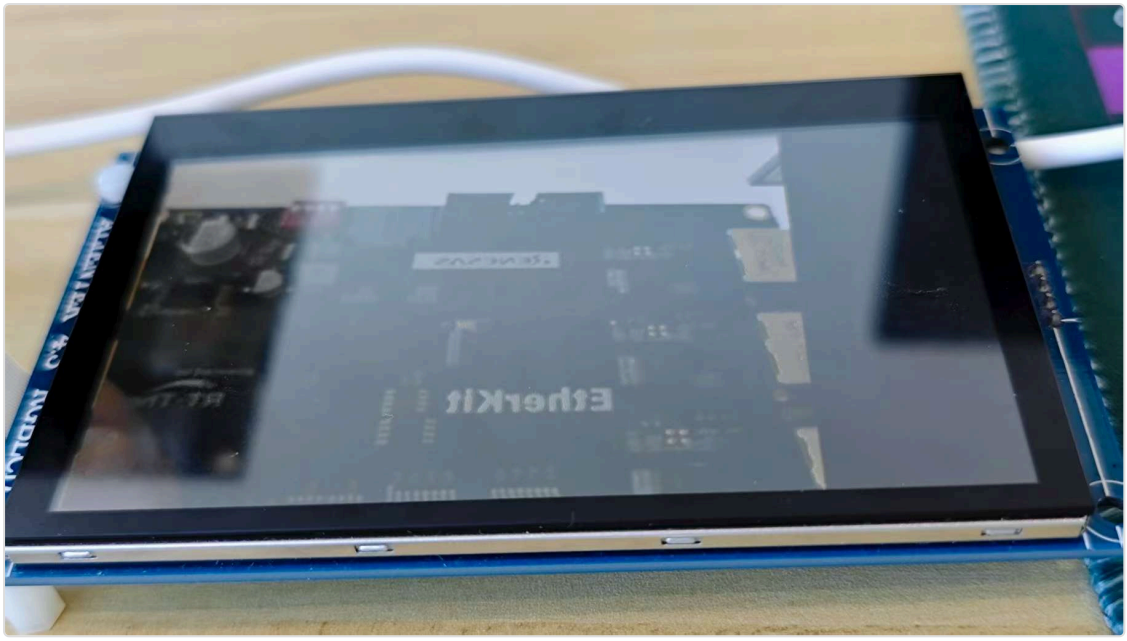
After compilation, connect the development board's USB-DBG interface to the PC and download the firmware to the development board.

Run Effect

After resetting the Titan Board, the terminal will output the following message:

```
 \ | /
- RT -     Thread Operating System
 / | \     5.2.0 build Aug 15 2025 15:02:50
 2006 - 2024 Copyright by RT-Thread team
[D/drv_lcd]
lcd framebuffer address:0x22020d40
msh >
Hello Titan Board!
================================================
This example project is an mipi-csi camera display routine!
================================================
[I/camera.i2c] I2C bus initialized: i2c0
The vin driver uses hardware mailboxes for the buffer.
```

Here is the image displayed on the LCD screen:


EtherKit

4.2. LVGL Example Guide

English | **Chinese**

Instruction

This example is based on the **LVGL** graphics library and demonstrates a sample graphical user interface (GUI) project designed for embedded devices. It integrates both display and input drivers, showcasing the complete workflow from low-level driver adaptation to high-level interface construction. Through this example, developers can not only learn how to port and configure LVGL in an embedded environment but also gain practical knowledge of key techniques such as interface layout, event handling, and theme customization, thereby laying a solid foundation for building real product interfaces.

LVGL Instruction

LVGL (Light and Versatile Graphics Library) is an open-source embedded GUI (Graphical User Interface) development framework. It was designed with the goal of providing smooth and modern graphical interfaces for resource-constrained embedded devices. To achieve this, it has been highly optimized in terms of runtime efficiency, memory usage, and portability. Whether running on simple low-end MCUs or more powerful MPU platforms, LVGL can operate efficiently while offering a wide range of graphical widgets and interactive features.

Key Features

1. **Lightweight**

   The greatest advantage of LVGL is its lightweight nature, requiring very few resources. It can still run smoothly on microcontrollers with only a few dozen KB of memory. The framework itself has minimal memory overhead, and its rendering algorithms are carefully optimized to ensure low CPU usage while delivering good visual performance on limited hardware. This makes LVGL particularly suitable for resource-sensitive scenarios such as low-power devices, wearables, and home appliance control panels.

2. **Cross-Platform**

   LVGL stands out for its cross-platform capability. It can run on multiple operating systems including FreeRTOS, RT-Thread, Zephyr, and Linux, or even directly on bare-metal systems. Developers only need to provide display and input driver interfaces for LVGL, and porting can be completed quickly, enabling the same UI code to be reused across different hardware platforms. This flexibility significantly reduces development costs and makes LVGL a universal solution for embedded GUIs.

3. **Rich Widgets**

   LVGL comes with a wide variety of built-in GUI widgets such as buttons, labels, sliders, progress bars, checkboxes, lists, tables, and charts. These components cover nearly all common HMI (Human-Machine Interface) requirements, sparing developers from designing and implementing them from scratch and thus shortening the development cycle. LVGL also allows extending existing widgets to create custom components, making it adaptable for both simple numeric displays and complex graphical control interfaces.

4. **Advanced Rendering Capabilities**

   In terms of visual presentation, LVGL offers a rich set of rendering features. It supports anti-aliasing, transparency, gradients, shadows, borders, and rounded corners, enabling modern and visually appealing interfaces. Additionally, LVGL includes a built-in animation system with multiple easing functions, making smooth movements, transitions, and dynamic effects easy to implement. These capabilities enhance not only functionality but also the overall user experience.

5. **Input Device Support**

   LVGL supports multiple input device types, including touchscreens, capacitive panels, mice, keyboards, and rotary encoders, with even multi-touch interaction available. A unified input interface layer is provided, so developers only need to implement low-level driver adaptation to pass input events into LVGL’s event system. This allows them to focus on interface logic instead of spending excessive time on input handling.

6. **Internationalization and Multilanguage Support**

   LVGL has excellent internationalization support. It uses UTF-8 encoding, which can handle nearly all character sets. It also supports bidirectional text rendering, ensuring correct display of languages such as Arabic and Hebrew that are written right-to-left. This makes LVGL suitable for globally

targeted products, allowing users in different countries and regions to enjoy a localized and seamless experience.

7. **Extensibility**
   LVGL provides a flexible theming and styling system, enabling developers to easily customize the appearance of widgets to achieve various UI styles. By switching themes, the overall look and feel of the interface can be changed quickly. Furthermore, LVGL can integrate with third-party graphics libraries, file systems, and image decoders to expand its functionality. For instance, developers can use a file system to load external fonts and images, or integrate JPEG/PNG decoders to display complex images.

Application Scenarios

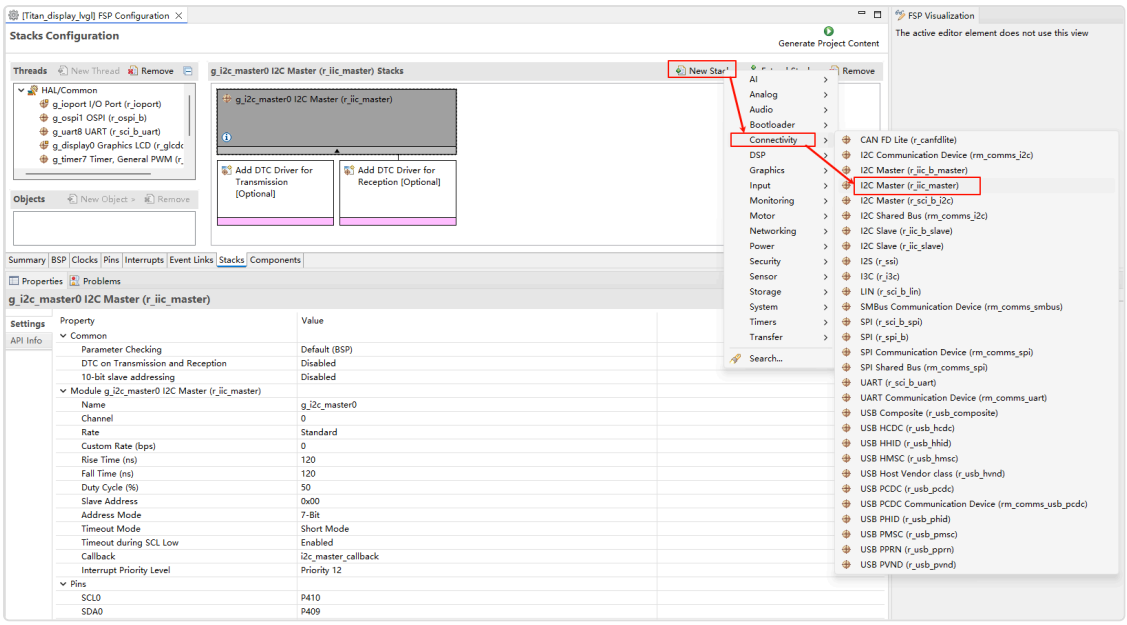
LVGL has a very broad range of real-world applications. In consumer electronics, it is commonly used in smart home control panels, appliance displays, smartwatches, and fitness devices, all of which require user-friendly interfaces on constrained hardware. In industrial control, LVGL is applied to HMIs (Human-Machine Interfaces) and various instruments, helping users intuitively monitor and control equipment. In automotive electronics, LVGL can power infotainment systems, passenger entertainment displays, and even instrument clusters. In the medical field, it is well-suited for small-scale displays, such as handheld diagnostic tools and portable monitoring devices.

Ecosystem and Community

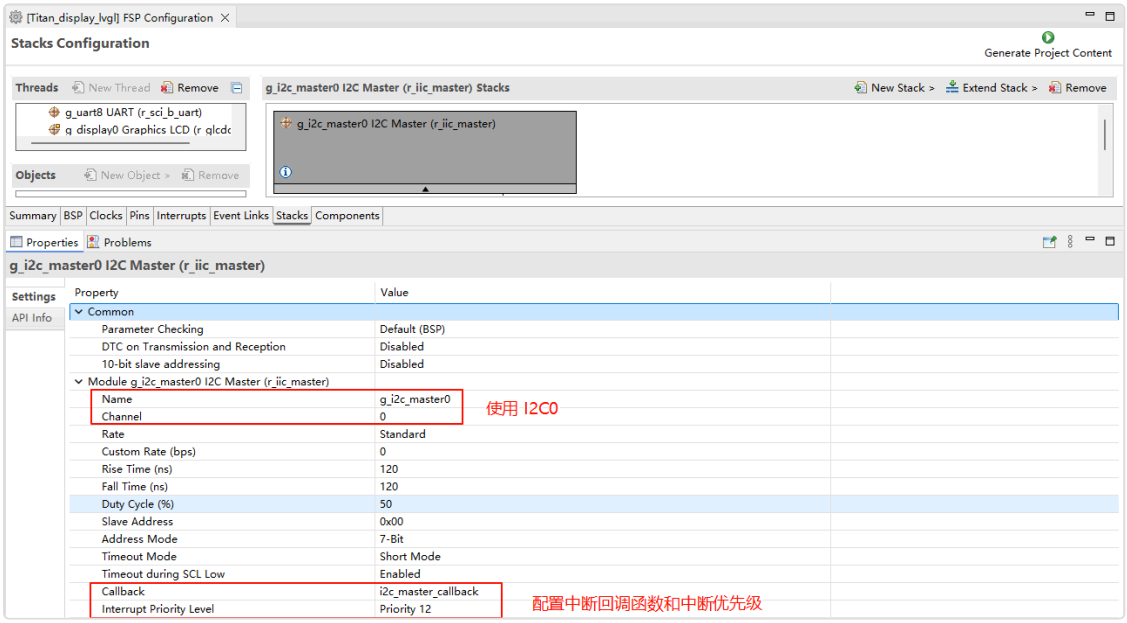
As an open-source project under the **MIT License**, LVGL is equally suitable for personal learning and free use in commercial products. Around LVGL, an active ecosystem has formed. Official tools such as **SquareLine Studio** provide visual drag-and-drop design with exportable LVGL code, greatly improving development efficiency. The **LVGL Simulator** enables developers to debug interfaces directly on a PC without repeatedly flashing target hardware. A vibrant global developer community contributes numerous open-source widgets, themes, and porting examples, offering valuable resources and support to both beginners and enterprise developers.

I2C0 Configuration

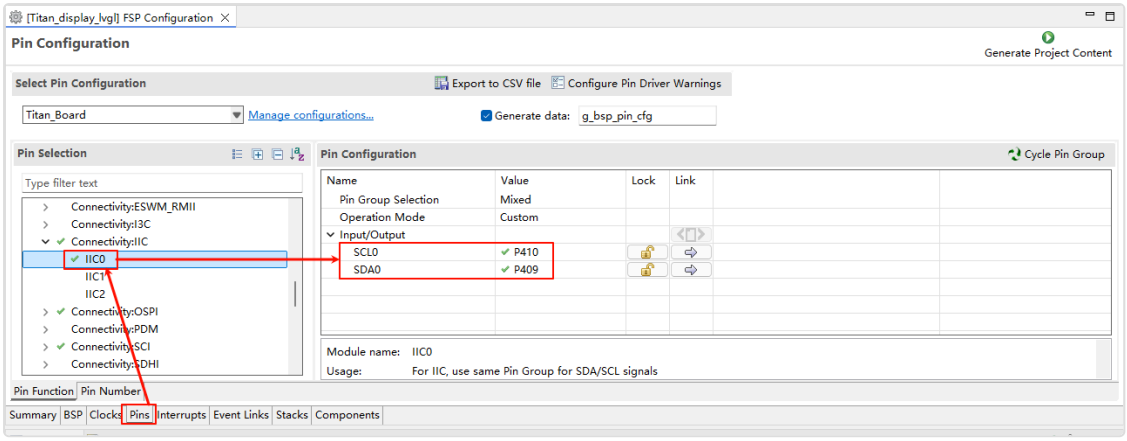
- Create a `r_iic_master` stack:



- Configure I2C0:

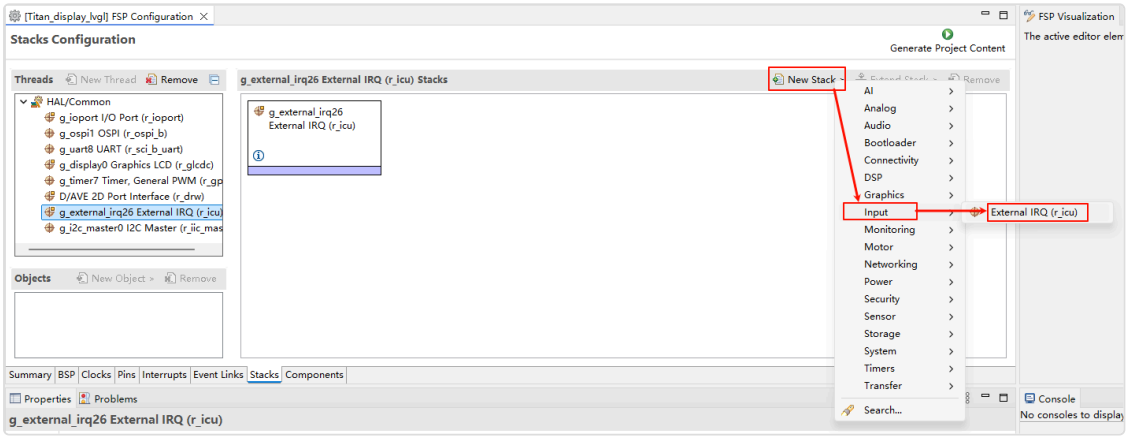


- Configure I2C0 pins:

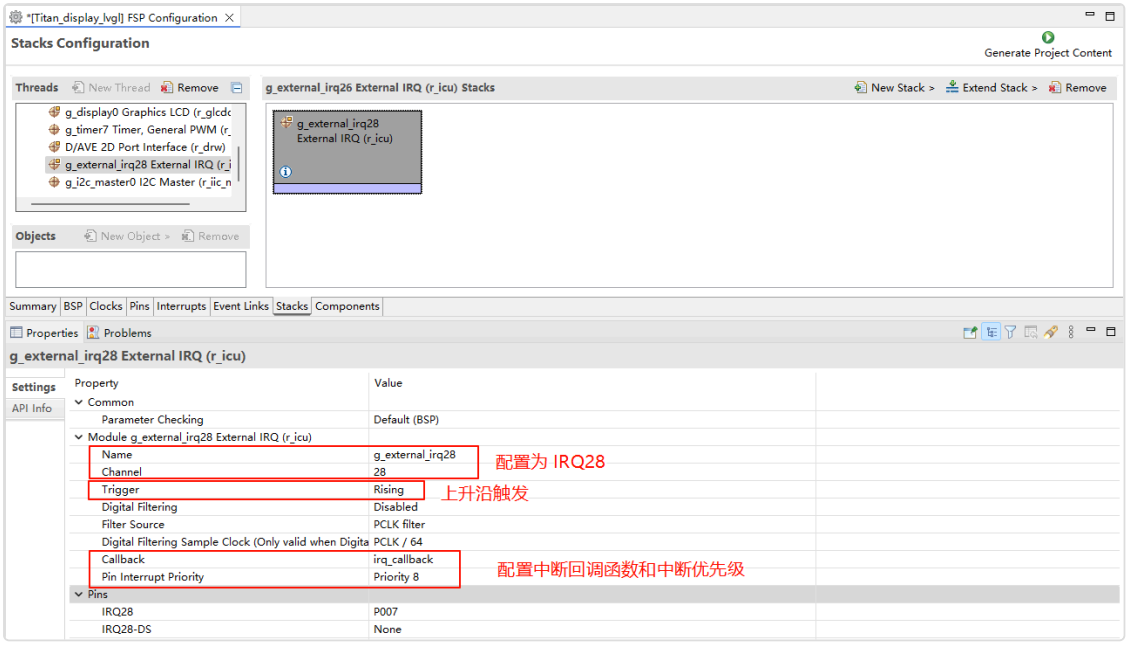


Touch Interrupt Configuration

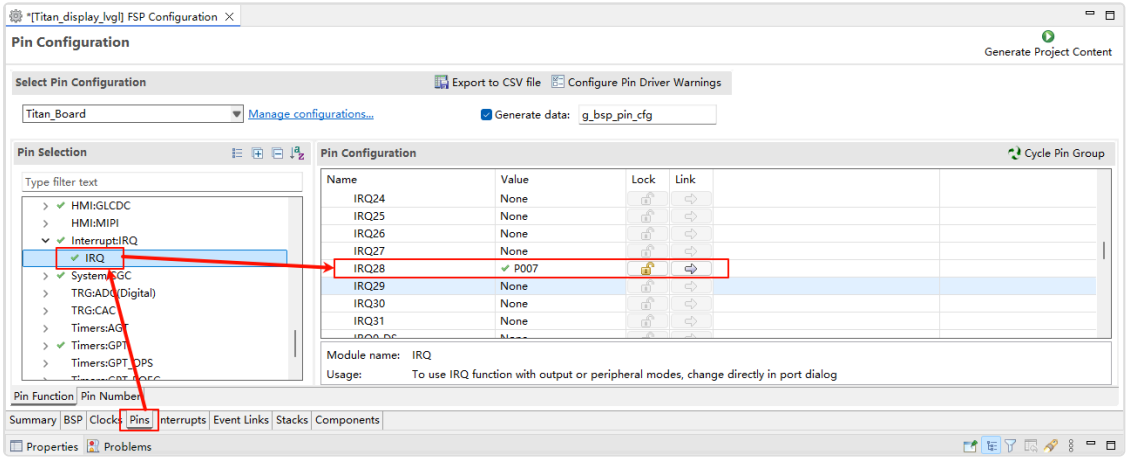
- Create a `r_icu` stack:



- Configure IRQ28:

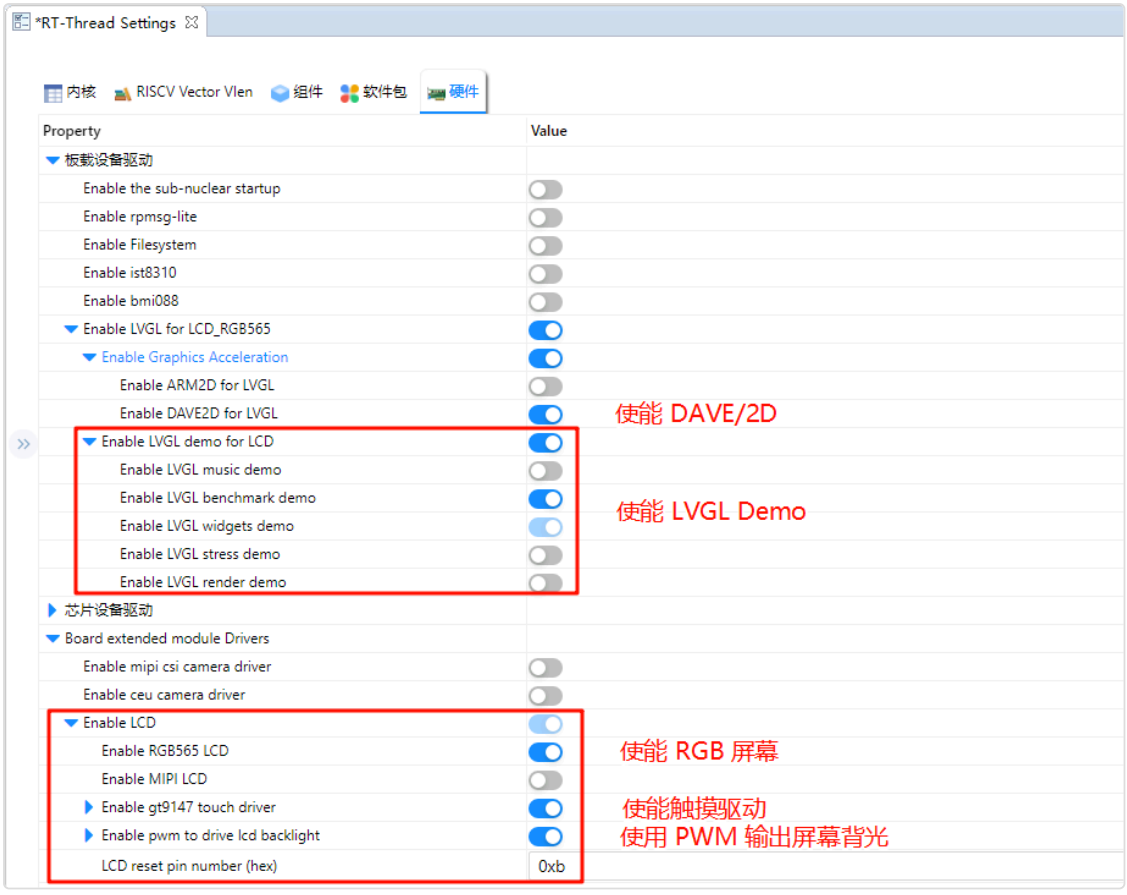


- Configure IRQ28 pins:



RT-Thread Settings Configuration

- Enable the RGB565 LCD, the GT9147 touch driver, and LVGL.



Software Description

The LVGL-related code is located in `./board/lvgl`.

- lv_port_disp.c

```
#include <lvgl.h>
#include <rtthread.h>
#include "ra8/lcd_config.h"
#include "hal_data.h"

static rt_sem_t _SemaphoreVsync = RT_NULL;
static uint8_t lvgl_init_flag = 0;

void DisplayVsyncCallback(display_callback_args_t *p_args)
{
    rt_interrupt_enter();
    if (DISPLAY_EVENT_LINE_DETECTION == p_args->event)
    {
        if (lvgl_init_flag != 0)
            rt_sem_release(_SemaphoreVsync);
    }
    rt_interrupt_leave();
```

```
}

static void vsync_wait_cb(lv_display_t *display)
{
    if (!lv_display_flush_is_last(display)) return;

    // If Vsync semaphore has already been set, clear it then w
    rt_sem_take(_SemaphoreVsync, RT_WAITING_FOREVER);
}

static void disp_flush(lv_display_t *display, const lv_area_t *
{
    if (!lv_display_flush_is_last(display)) return;

#if (BSP_CFG_DCACHE_ENABLED)
    int32_t size;
    /* Invalidate cache - so the HW can access any data written
    size = sizeof(fb_background[0]);

    SCB_CleanInvalidateDCache_by_Addr(px_map, size);
#endif

    R_GLCDC_BufferChange(&g_display0_ctrl,
                         (uint8_t *) px_map,
                         (display_frame_layer_t) 0);
}

void lv_port_disp_init(void)
{
    static rt_device_t device;
    /* LCD Device Init */
    device = rt_device_find("lcd");
    RT_ASSERT(device != RT_NULL);

    _SemaphoreVsync = rt_sem_create("lvgl_sem", 1, RT_IPC_FLAG_

    if (RT_NULL == _SemaphoreVsync)
    {
        rt_kprintf("lvgl semaphore create failed\r\n");
        RT_ASSERT(0);
    }

    /*-----------------------------------
     * Create a display and set a flush_cb
     * ----------------------------------*/
    lv_display_t *disp = lv_display_create(LV_HOR_RES_MAX, LV_V
    lv_display_set_flush_cb(disp, disp_flush);
    lv_display_set_flush_wait_cb(disp, vsync_wait_cb);
    lv_display_set_buffers(disp, &fb_background[0][0], &fb_back
```

```
    lvgl_init_flag = 1;
}
```

- lv_port_indev.c

```
#include <lvgl.h>
#include <rtdevice.h>

#ifdef RT_USING_TOUCH

#include "gt9147.h"
#define GT9147_IRQ_PIN   BSP_IO_PORT_00_PIN_07
#define GT9147_RST_PIN   BSP_IO_PORT_04_PIN_12

#define DBG_TAG "lv_port_indev"
#define DBG_LVL DBG_LOG
#include <rtdbg.h>

#include "hal_data.h"

static rt_device_t touch_dev;
struct rt_touch_data *read_data;
static lv_indev_t *touch_indev;

#if LVGL_VERSION_MAJOR < 9
static void touchpad_read(lv_indev_drv_t *indev, lv_indev_data_
#else
static void touchpad_read(lv_indev_t *indev, lv_indev_data_t *d
#endif
{
    if (RT_NULL != rt_device_read(touch_dev, 0, read_data, 1))
    {
        if (read_data->event == RT_TOUCH_EVENT_NONE)
            return;

        data->point.x = LV_HOR_RES_MAX - read_data->x_coordinat
        data->point.y = LV_VER_RES_MAX - read_data->y_coordinat

        if (read_data->event == RT_TOUCH_EVENT_DOWN)
            data->state = LV_INDEV_STATE_PR;
        if (read_data->event == RT_TOUCH_EVENT_MOVE)
            data->state = LV_INDEV_STATE_PR;
        if (read_data->event == RT_TOUCH_EVENT_UP)
            data->state = LV_INDEV_STATE_REL;
    }
}
```

```
static rt_err_t gt9147_probe(rt_uint16_t x, rt_uint16_t y)
{
    void *id;

    touch_dev = rt_device_find("gt9147");
    if (touch_dev == RT_NULL)
    {
        rt_kprintf("can't find device gt9147\n");
        return -1;
    }

    if (rt_device_open(touch_dev, RT_DEVICE_FLAG_INT_RX) != RT_
    {
        rt_kprintf("open device failed!");
        return -1;
    }

    id = rt_malloc(sizeof(struct rt_touch_info));
    rt_device_control(touch_dev, RT_TOUCH_CTRL_GET_ID, id);
    rt_uint8_t *read_id = (rt_uint8_t *)id;
    rt_kprintf("id = GT%d%d%d \n", read_id[0] - '0', read_id[1]

    rt_device_control(touch_dev, RT_TOUCH_CTRL_SET_X_RANGE, &x)
    rt_device_control(touch_dev, RT_TOUCH_CTRL_SET_Y_RANGE, &y)
    rt_device_control(touch_dev, RT_TOUCH_CTRL_GET_INFO, id);
    rt_kprintf("range_x = %d \n", (*(struct rt_touch_info *)id)
    rt_kprintf("range_y = %d \n", (*(struct rt_touch_info *)id)
    rt_kprintf("point_num = %d \n", (*(struct rt_touch_info *)i
    rt_free(id);

    read_data = (struct rt_touch_data *)rt_calloc(1, sizeof(str
    if (!read_data)
    {
        return -RT_ENOMEM;
    }

    return RT_EOK;
}

rt_err_t rt_hw_gt9147_register(void)
{
    struct rt_touch_config config;
    rt_uint8_t rst;
    rst = GT9147_RST_PIN;
    config.dev_name = "i2c0";
    config.irq_pin.pin = GT9147_IRQ_PIN;
    config.irq_pin.mode = PIN_MODE_INPUT_PULLDOWN;
    config.user_data = &rst;
```

```
    if (rt_hw_gt9147_init("gt9147", &config) != RT_EOK)
    {
        rt_kprintf("touch device gt9147 init failed.\n");
        return -RT_ERROR;
    }

    if (gt9147_probe(LV_HOR_RES_MAX, LV_VER_RES_MAX) != RT_EOK)
    {
        rt_kprintf("probe gt9147 failed.\n");
        return -RT_ERROR;
    }
    return RT_EOK;
}
#endif

void lv_port_indev_init(void)
{
#ifdef RT_USING_TOUCH

#if LVGL_VERSION_MAJOR < 9
    static lv_indev_drv_t indev_drv;         /* Descriptor of a
    lv_indev_drv_init(&indev_drv);           /* Basic initializ
    indev_drv.type = LV_INDEV_TYPE_POINTER;  /* Touch pad is a
    indev_drv.read_cb = touchpad_read;       /* Set your driver

    /* Register the driver in LVGL and save the created input d
    touch_indev = lv_indev_drv_register(&indev_drv);
#else
    /*Register a touchpad input device*/
    touch_indev = lv_indev_create();
    lv_indev_set_type(touch_indev, LV_INDEV_TYPE_POINTER);
    lv_indev_set_read_cb(touch_indev, touchpad_read);
#endif
    /* Register touch device */
    rt_err_t res = rt_hw_gt9147_register();
    RT_ASSERT(res == RT_EOK);

#endif
}
```

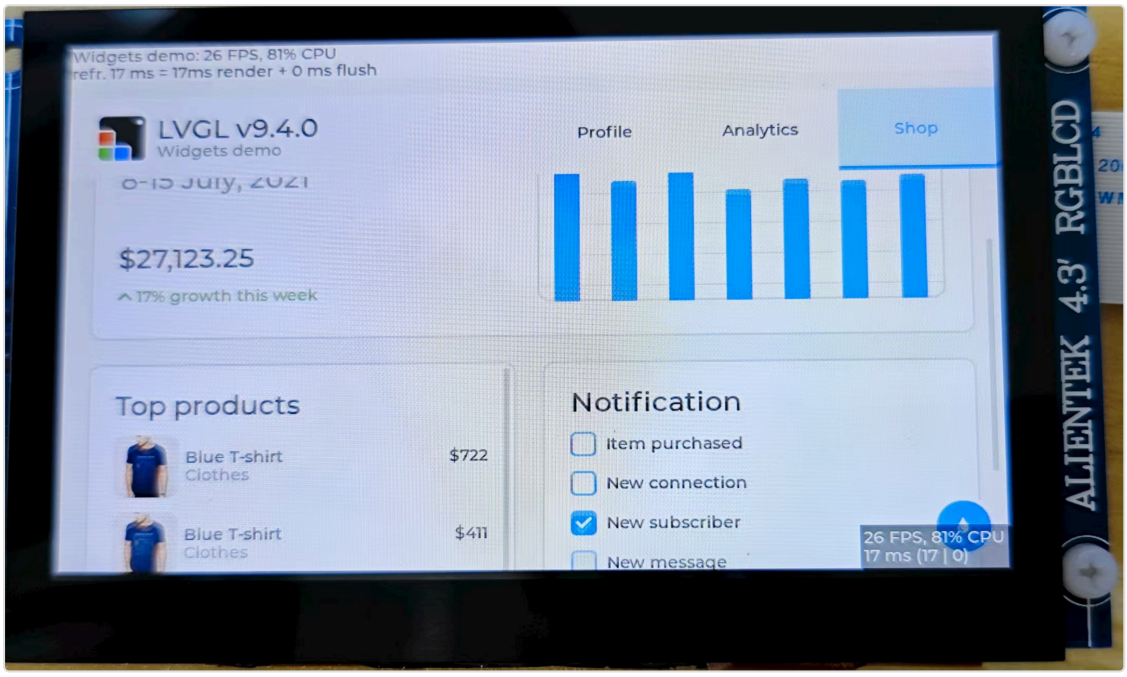
Compilation & Download

- **RT-Thread Studio**: In RT-Thread Studio's package manager, download the Titan Board resource package, create a new project, and compile it.

After compilation, connect the development board's USB-DBG interface to the PC and download the firmware to the development board.

Run Effect

After resetting the development board, you can see the LVGL demo running on the LCD screen.



4.3. MIPI NPU Face Detection

English | 中文

Overview

This example runs real-time face detection on **Titan Board Mini** using:

- **OV5640** camera connected through **MIPI CSI**
- **RA8P1 VIN** for frame capture
- **Arm Ethos-U55 NPU** for YOLO-Fastest inference
- **RGB565 LCD** for live preview and face box overlay

Features

- Captures camera frames from **MIPI CSI + VIN**
- Runs **YOLO-Fastest face detection** on the NPU
- Draws green face boxes on the LCD
- Supports **OV5640 auto focus** by user key
- Supports compile-time control for detection logs

Data Flow

```
OV5640
  -> MIPI CSI
  -> VIN frame buffer
  -> CPU preprocessing (resize + grayscale + quantization)
  -> Ethos-U55 inference
  -> CPU postprocessing (decode + NMS)
  -> D/AVE2D overlay
  -> GLCDC
  -> RGB565 LCD
```

Model

- Model: **YOLO-Fastest (face detection)**
- Framework: **TensorFlow Lite INT8**
- Input size: **192 x 192**
- Camera frame size: **640 x 480**

Runtime Controls

- **Auto focus**
- Press the user key to trigger OV5640 auto focus.
- **Detection log switch**
- Edit `DETECT_RESULT_LOG_ENABLE` in `src/hal_entry.c`
- `0`: disable `detect box num` / `Time elapsed` logs
- `1`: enable logs

Important Notes

- The project uses the framebuffer overlay path for face box rendering on the LCD.
- OV5640 is configured through **I2C0** in the MIPI camera path.

RT-Thread / FSP Configuration

Make sure the project enables:

- **MIPI CSI**
- **VIN**
- **Ethos-U55 / rm_ethosu**
- **OV5640 control through I2C0**
- **RGB565 LCD / GLCDC**

Build and Flash

Build the project with **RT-Thread Studio** or the existing project build flow in this repository, then flash it through the board debug interface.

Expected Result

After boot:

- The LCD shows the live OV5640 preview
- Detected faces are highlighted with green rectangles
- Pressing the user key triggers camera auto focus



5. Multicore

5.1. Titan_Mini_rpmsg_master Inter-Core Communication Example (M85)

English | **Chinese**

Introduction

This project is the Master side of the Titan Board Mini dual-core RPMSG communication example and runs on Cortex-M85 (Core0). It initializes the RPMSG master after power-on and does not send messages automatically. The communication is triggered by a console command, exchanging a bidirectional hello with `Titan_Mini_rpmsg_remote` on Cortex-M33 (Core1).

Companion project: `project/Titan_Mini_rpmsg/Titan_Mini_rpmsg_remote`.

Project Structure

- `src/hal_entry.c`: RPMSG master initialization, receive handling, and the `rpmsg_hello` command
- `board/linker_scripts/fsp.ld`: Core0 linker script, Flash starts at 0x02000000 with length 0xC0000
- `packages/rpmsg-lite-latest`: RPMSG Lite package
- `rt-thread`, `libraries`: shared code linked from the repository root

Communication Flow

1. M85 initializes the RPMSG master and waits for the RPMSG channel with M33 to be ready.
2. Enter `rpmsg_hello` on the M85 console.

3. M85 sends the string `hello` to M33 and prints `M85 TX: hello`.
4. M33 receives it, prints `M33 RX: hello`, and automatically replies with `hello`.
5. M85 receives the reply and prints `M85 RX: hello`.

Usage

1. Build and flash `Titan_Mini_rpmsg_remote` to Core1 Flash (0x020C0000) first.
2. Build and flash this project to Core0 Flash (0x02000000).
3. The M85 debug console uses UART1; the M33 debug console uses UART2.
4. After M85 prints `RPMSG master ready`, enter `rpmsg_hello` on the M85 console.

Expected Output

M85 console:

```
RPMSG master ready
M85 TX: hello
M85 RX: hello
```

M33 console:

```
M33 RX: hello
M33 TX: hello
```

5.2. Titan_Mini_rpmsg_remote Inter-Core Communication Example (M33)

English | **Chinese**

Introduction

This project is the Remote side of the Titan Board Mini dual-core RPMSG communication example and runs on Cortex-M33 (Core1). It is started by M85, initializes the RPMSG remote endpoint, and waits for incoming messages. When it receives `hello` from M85, it automatically replies with `hello`.

Companion project: `project/Titan_Mini_rpmsg/Titan_Mini_rpmsg_master`.

Project Structure

- `src/hal_entry.c`: RPMSG remote initialization, receive handling, and automatic reply logic
- `board/linker_scripts/fsp.ld`: Core1 linker script, Flash starts at 0x020C0000 with length 0x30000
- `packages/rpmsg-lite-latest`: RPMSG Lite package
- `rt-thread`, `libraries`: shared code linked from the repository root

Communication Flow

1. M33 initializes the RPMSG remote and waits for the RPMSG channel with M85 to be ready.
2. After `rpmsg_hello` is entered on the M85 console, M85 sends `hello` to M33.
3. M33 receives it, prints `M33 RX: hello`, and automatically replies with `hello`.
4. M33 prints `M33 TX: hello` after replying, and M85 prints `M85 RX: hello` after receiving it.

Usage

1. Build and flash this project to Core1 Flash (0x020C0000) first.
2. Build and flash `Titan_Mini_rpmsg_master` to Core0 Flash (0x02000000).
3. The M33 debug console uses UART2; the M85 debug console uses UART1.
4. After resetting the board, enter `rpmsg_hello` on the M85 console to verify bidirectional communication.

Expected Output

M33 console:

```
M33 RX: hello
M33 TX: hello
```

M85 console:

```
RPMSG master ready
M85 TX: hello
M85 RX: hello
```

6. Peripherals

6.1. PDM Microphone Real-time Loopback Example Guide

English | **中文**

Introduction

This example demonstrates how to use the **RA8P1 PDM (Pulse Density Modulation) Interface** on **Titan Board Mini** to read audio from a **PDM Microphone** and play it back in real-time through the **ES8156 Audio Codec**, implementing a **Microphone → Speaker** audio loopback function.

Main features include:

- PDM microphone data acquisition using RA8P1 built-in PDM interface
- PDM to PCM format conversion (supports 16-bit and 20-bit modes)
- Mono to stereo audio processing
- Double-buffer mechanism for continuous audio streaming
- Audio output via SSI interface and ES8156 codec

Hardware Introduction

1. PDM Microphone

The **Titan Board Mini** features an onboard **PDM Microphone**:

Parameter	Description
Type	MEMS PDM Microphone
Interface	PDM (Pulse Density Modulation)

Parameter	Description
Output Format	1-bit density stream
Sample Rate	Up to 16kHz (this example)
Channels	Mono

2. RA8P1 PDM Interface

RA8P1 PDM interface features:

- **Built-in PDM demodulator**: Hardware PDM → PCM conversion
- **Configurable filters**: SINC filter, HPF, LPF
- **Multi-channel support**: Supports multiple microphone inputs
- **Configurable output**: Supports 16-bit or 20-bit PCM output
- **DMA support**: DMA transfer support, reducing CPU usage

3. Audio Output

- **Codec**: ES8156 (24-bit DAC, supports 8kHz - 192kHz)
- **Interface**: SSI (I2S protocol)
- **Output**: 3.5mm headphone jack / onboard speaker

Software Architecture

1. Data Flow

```
PDM Microphone
    ↓ (PDM signal)
RA8P1 PDM Interface
    ↓ (Hardware filtering + PDM→PCM)
PDM Buffer (int32_t[])
    ↓ (Software conversion)
DAC Buffer (int16_t[], stereo)
    ↓ (I2S protocol)
ES8156 Codec
```

```
    ↓ (Analog audio)
Speaker/Headphone
```

2. Core Components

PDM Driver

Using Renesas FSP PDM driver:

```
/* Open PDM module */
R_PDM_Open(&g_pdm0_ctrl, &g_pdm0_cfg);

/* Start data acquisition */
R_PDM_Start(&g_pdm0_ctrl, buffer, buf_size.bytes, samples);

/* Stop acquisition */
R_PDM_Stop(&g_pdm0_ctrl);

/* Close PDM module */
R_PDM_Close(&g_pdm0_ctrl);
```

Data Conversion Function

```
/* PDM data to DAC data conversion */
static void convert_pdm_to_dac(int32_t *pdm_data, uint32_t pdm_
{
    for (uint32_t i = 0; i < pdm_samples; i++)
    {
        int16_t sample_16;
        if (g_pcm_bits == PCM_20BITS)
        {
            /* 20-bit -> 16-bit: shift right 4 bits */
            sample_16 = (int16_t)(pdm_data[i] >> 4);
        }
        else
        {
            /* 16-bit mode: shift left 1 bit */
            sample_16 = (int16_t)(pdm_data[i] << 1);
        }

        /* Mono to stereo */
        dac_data[i * 2] = sample_16;      /* Left channel */
        dac_data[i * 2 + 1] = sample_16;  /* Right channel */
    }
}
```

3. Double Buffer Mechanism

To achieve continuous audio streaming, a double-buffer mode is used:

```
Timeline:
    t0      t1      t2      t3      t4
    |-------|-------|-------|-------|
Buffer 0: [Record ] [Play         ]
Buffer 1:                 [Record ] [Play         ]
```

- **Buffer 0**: Record → Play
- **Buffer 1**: Record → Play
- Two buffers alternate for seamless continuous playback

4. Project Structure

```
Titan_Mini_pdm/
├── src/
│   └── hal_entry.c          # Main program entry
├── libraries/
│   └── HAL_Drivers/
│       ├── drv_i2s.c        # SSI/I2S driver
│       └── ports/
│           └── es8156/      # ES8156 codec driver
└── ra_gen/
    └── hal_data.c           # FSP configuration (PDM, SSI, etc
```

Configuration

1. PDM Configuration

Configure PDM parameters in `ra_gen/hal_data.c`:

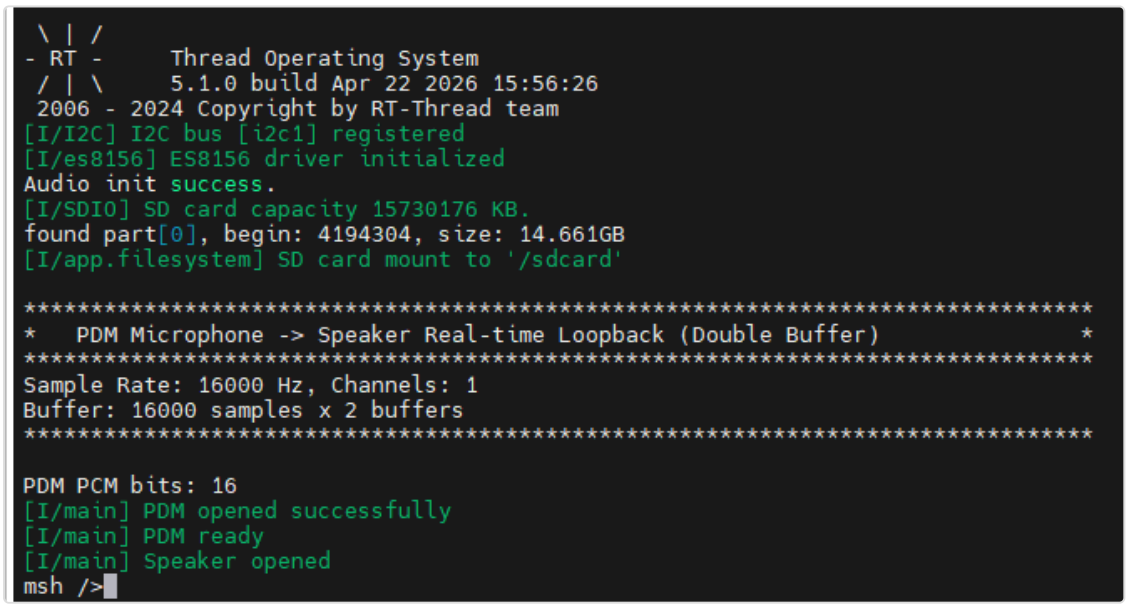
```
const pdm_cfg_t g_pdm0_cfg =
{
    .unit                                = 0,
    .channel                             = 2,        /* PDM chann
    .pcm_width                           = PDM_PCM_WIDTH_16_BITS_
    .pcm_edge                            = PDM_INPUT_DATA_EDGE_RI
    .p_extend                            = &g_pdm0_cfg_extend,
```

```
    // ...
};
```

2. Key Parameters

Parameter	Value	Description
`PDM_FS_HZ`	16000	Output sample rate 16kHz
`PDM_CHANNELS`	1	Mono input
`PDM_PCM_WIDTH`	16_BITS_0_14	16-bit PCM, valid bits [14:0]
`NUM_BUFFERS`	2	Double buffer mode
`PDM_RECORD_DURATION_SEC`	1	1 second audio per buffer

Run Effect



1. After power-on, green LED lights up indicating system ready
2. Blue LED lights up indicating audio loopback is running
3. Speak into the microphone and hear your voice in real-time through headphones/speaker

4. Blue LED blinks every 50 cycles (heartbeat indicator)

Troubleshooting

1. No Audio Output

- Check if headphones/speaker are properly connected
- Check volume settings (default 60%)
- Verify ES8156 initialization was successful

2. Audio Stuttering

- Current uses 1-second buffer, try adjusting `PDM_RECORD_DURATION_SEC`
- Check system load, reduce other tasks

3. Audio Distortion

- Verify `pcm_width` configuration matches hardware
- Adjust shift parameters in data conversion function

Further Reading

- Renesas RA8P1 Hardware Manual
- ES8156 Datasheet
- RT-Thread Audio Framework Documentation

6.2. IMU Sensor Example

中文 | **English**

Overview

This example demonstrates how to use the **LSM6DS3TR-C 6-axis IMU sensor** on the **Titan Board Mini** to implement inertial measurement functionality. It reads **3-axis accelerometer** and **3-axis gyroscope** data through the **I2C interface**, combined with the **RT-Thread sensor framework** to achieve complete sensor data acquisition and processing.

Main features include:

- 6-axis inertial measurement using LSM6DS3TR-C
- Read 3-axis accelerometer data (X/Y/Z)
- Read 3-axis gyroscope data (X/Y/Z)
- Support for multiple range and sampling rate configurations
- Integration with RT-Thread sensor framework

Hardware Introduction

1. LSM6DS3TR-C IMU Sensor

Titan Board Mini supports the **LSM6DS3TR-C** high-performance 6-axis IMU sensor:

Parameter	Description
Model	LSM6DS3TR-C
Manufacturer	STMicroelectronics
Type	6-axis IMU (3-axis accelerometer + 3-axis gyroscope)
Interface	I2C / SPI
Operating Voltage	1.71V - 3.6V

Parameter	Description
Temperature Range	-40°C ~ +85°C
Package	2.5mm x 3mm x 0.83mm LGA-14

Software Architecture

1. Layered Design

The IMU sensor system adopts a layered architecture:

```
Application Layer (user code)
    ↓
RT-Thread Sensor Driver Framework - Sensor device framework
    ↓
LSM6DS3TR-C Driver - IMU driver
```

2. Core Components

Platform Porting Layer Interface

Platform-related interfaces to be implemented (`lsm6ds3tr-c_port.c`):

```
/* I2C read/write interfaces */
int32_t platform_write(void *handle, uint8_t reg, const uint8_t
int32_t platform_read(void *handle, uint8_t reg, uint8_t *bufp,

/* Delay interface */
void platform_delay(uint32_t ms);
```

RT-Thread Sensor Framework

Unified sensor device interface provided by RT-Thread:

```
/* Find sensor device */
rt_device_t rt_device_find(const char *name);

/* Open sensor device */
rt_err_t rt_device_open(rt_device_t dev, rt_uint16_t oflags);
```

```
/* Read sensor data */
rt_size_t rt_device_read(rt_device_t dev, rt_off_t pos, void *b

/* Receive sensor data */
rt_err_t rt_device_set_rx_indicate(rt_device_t dev, rt_err_t (*
```

3. Project Structure

```
Titan_Mini_peripheral_imu/
├── src/
│   └── hal_entry.c          # Main program entry
└── packages/
    └── lsm6ds3tr/           # LSM6DS3TR-C driver package
        ├── lsm6ds3tr-c_reg.h    # Register definitions and dri
        ├── lsm6ds3tr-c_reg.c    # Register-level driver implem
        └── lsm6ds3tr-c_port.c   # Platform porting layer
```

Usage Instructions

```
int lsm6ds3tr_c_read_data_sample(void)
{
    /* Initialize mems driver interface */
    stmdev_ctx_t dev_ctx;
    dev_ctx.write_reg = platform_write;
    dev_ctx.read_reg = platform_read;
    dev_ctx.mdelay = platform_delay;
    /* Init test platform */
    platform_init();
    /* Check device ID */
    whoamI = 0;
    lsm6ds3tr_c_device_id_get(&dev_ctx, &whoamI);

    if (whoamI != LSM6DS3TR_C_ID)
        while (1); /*manage here device not found */

    /* Restore default configuration */
    lsm6ds3tr_c_reset_set(&dev_ctx, PROPERTY_ENABLE);

    do
    {
        lsm6ds3tr_c_reset_get(&dev_ctx, &rst);
    }
    while (rst);
```

```
    /* Enable Block Data Update */
    lsm6ds3tr_c_block_data_update_set(&dev_ctx, PROPERTY_ENABLE
    /* Set Output Data Rate */
    lsm6ds3tr_c_xl_data_rate_set(&dev_ctx, LSM6DS3TR_C_XL_ODR_1
    lsm6ds3tr_c_gy_data_rate_set(&dev_ctx, LSM6DS3TR_C_GY_ODR_1
    /* Set full scale */
    lsm6ds3tr_c_xl_full_scale_set(&dev_ctx, LSM6DS3TR_C_2g);
    lsm6ds3tr_c_gy_full_scale_set(&dev_ctx, LSM6DS3TR_C_2000dps
    /* Configure filtering chain(No aux interface) */
    /* Accelerometer - analog filter */
    lsm6ds3tr_c_xl_filter_analog_set(&dev_ctx,
                                     LSM6DS3TR_C_XL_ANA_BW_400H
    /* Accelerometer - LPF1 path ( LPF2 not used )*/
    /* Accelerometer - LPF1 + LPF2 path */
    lsm6ds3tr_c_xl_lp2_bandwidth_set(&dev_ctx,
                                     LSM6DS3TR_C_XL_LOW_NOISE_L
    /* Accelerometer - High Pass / Slope path */
    /* Gyroscope - filtering chain */
    lsm6ds3tr_c_gy_band_pass_set(&dev_ctx,
                                 LSM6DS3TR_C_HP_260mHz_LP1_STRO

    /* Read samples in polling mode (no int) */
    while (1)
    {
        /* Read output only if new value is available */
        lsm6ds3tr_c_reg_t reg;
        lsm6ds3tr_c_status_reg_get(&dev_ctx, &reg.status_reg);
        if (reg.status_reg.xlda)
        {
            /* Read magnetic field data */
            rt_memset(data_raw_acceleration, 0x00, 3 * sizeof(i
            lsm6ds3tr_c_acceleration_raw_get(&dev_ctx,
                                             data_raw_accelerat
            acceleration_mg[0] = lsm6ds3tr_c_from_fs2g_to_mg(
                                     data_raw_acceleration[0]);
            acceleration_mg[1] = lsm6ds3tr_c_from_fs2g_to_mg(
                                     data_raw_acceleration[1]);
            acceleration_mg[2] = lsm6ds3tr_c_from_fs2g_to_mg(
                                     data_raw_acceleration[2]);
            rt_kprintf("Acceleration [mg]:%4.2f\t%4.2f\t%4.2f\r
                       acceleration_mg[0], acceleration_mg[1],

        }

        if (reg.status_reg.gda)
        {
            /* Read magnetic field data */
            rt_memset(data_raw_angular_rate, 0x00, 3 * sizeof(i
```

```
            lsm6ds3tr_c_angular_rate_raw_get(&dev_ctx,
                                             data_raw_angular_r
            angular_rate_mdps[0] = lsm6ds3tr_c_from_fs2000dps_t
                                           data_raw_angular_rate[0]
            angular_rate_mdps[1] = lsm6ds3tr_c_from_fs2000dps_t
                                           data_raw_angular_rate[1]
            angular_rate_mdps[2] = lsm6ds3tr_c_from_fs2000dps_t
                                           data_raw_angular_rate[2]
            rt_kprintf("Angular rate [mdps]:%4.2f\t%4.2f\t%4.2f
                       angular_rate_mdps[0], angular_rate_mdps[
        }

        if (reg.status_reg.tda)
        {
            /* Read temperature data */
            rt_memset(&data_raw_temperature, 0x00, sizeof(int16
            lsm6ds3tr_c_temperature_raw_get(&dev_ctx, &data_raw
            temperature_degC = lsm6ds3tr_c_from_lsb_to_celsius(
                                     data_raw_temperature);
            rt_kprintf("Temperature [degC]:%6.2f\r\n",
                       temperature_degC);
        }
        rt_thread_mdelay(500);
    }
}
INIT_APP_EXPORT(lsm6ds3tr_c_read_data_sample);
```

After the program is powered on, it will automatically run this function. Simply connect a serial terminal to view the printed IMU data.

Configuration Instructions

1. Kconfig Configuration

In `libraries/M85_Config/Kconfig`, IMU options are included:

```
config BSP_USING_LSM6DS3
    bool "Enable LSM6DS3 6-axis IMU"
    default n
```

2. RT-Thread Settings

In RT-Thread Studio, the following components need to be enabled:

1. **Device Drivers**

- Enable I2C device driver
- Configure I2C1 interface

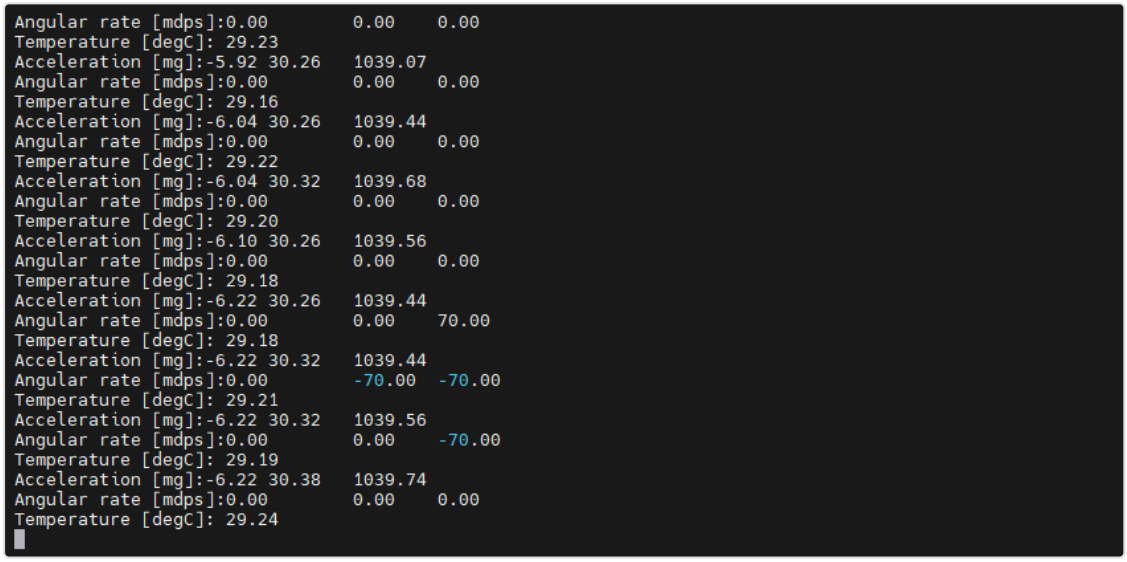
2. **Sensors**

- Enable LSM6DS3 6-axis IMU

Running Results

1. Terminal Output

After resetting Titan Board Mini, the terminal will output the following information:



Related Resources

- RT-Thread Sensor Framework Documentation

6.3. RGB LCD Example Guide

English | **Chinese**

Introduction

This example demonstrates how to use the **GLCDC module of the RA8 series MCU** together with **RT-Thread's LCD driver framework** on the **Titan Board** to drive an RGB LCD screen, enabling image display and interface updates. This document provides a detailed overview of **RA8 GLCDC peripheral features** and the **RT-Thread LCD driver framework**, along with example configurations and usage methods.

RA8 Series GLCDC Module

1. Overview

The **GLCDC (Graphics LCD Controller)** is a high-performance graphics controller integrated in RA8 series MCUs, specifically designed to drive TFT/RGB LCD screens. It supports various resolutions, color formats, and graphics processing functions. Combined with RT-Thread's **LCD driver framework**, it provides a unified interface for screen initialization, refresh, graphics rendering, and DMA-accelerated operations.

The RA8 **GLCDC** enables image output from either internal MCU memory or external frame buffers to RGB/LCD displays. Key features include:

- **Frame buffer control**: Supports multiple frame buffers for page switching or double-buffered display
- **Color format support**: RGB565, RGB888, ARGB8888, etc.
- **Graphics processing**: Background layers, text/graphics composition, alpha blending, palette mapping
- **Synchronization signal generation**: HSYNC, VSYNC, DE (Data Enable)
- **DMA support**: High-speed data transfer, reducing CPU load
- **Interrupt functionality**: Frame-end and line-end interrupts

2. Module Architecture

The RA8 GLCDC module mainly consists of the following submodules:

1. **Layer Composition Unit**

- Supports multiple layer stacking
- Provides alpha blending, transparency control, and color keying
- Allows rotation and flipping of layers

2. **Frame Buffer Interface**

- Supports access to MCU internal SRAM or external memory
- Provides single or double buffering to ensure continuous display
- Works with DMA to automatically read image data

3. **DMA Controller**

- Automatically transfers pixel data to the RGB output port
- Configurable burst length to improve bandwidth utilization
- Supports circular transfer, suitable for video or animation scenarios

4. **Timing Generator**

- Automatically generates HSYNC, VSYNC, and DE signals
- Supports TTL interface RGB timing
- Polarity, sync width, and front/back porch timings are configurable

5. **Interrupt and Event Controller**

- Provides frame-end and line-end interrupts
- Can be used for page switching, dynamic drawing, or scrolling display
- Supports interrupts triggered by DMA transfer completion

3. GLCDC Working Principle

1. **Frame buffer read**

- GLCDC uses DMA to fetch image data from memory, supporting single or double buffering for continuous display.

2. **Layer composition**

- Supports multiple layers such as background + foreground + icons/text

- Provides alpha blending and palette mapping

3. **Pixel timing output**

- Generates HSYNC/VSYNC/DE signals according to LCD interface requirements
- Supports RGB parallel interface, TTL interface, or LVDS (depending on board implementation)

4. **Interrupts and events**

- **Frame-end interrupt (VBlank)**: Can be used to update the next frame
- **Line-end interrupt**: Useful for scrolling displays or dynamic rendering

4. GLCDC Supported Features

Feature Category	Description
Resolution	Up to 800×480 (depending on MCU model and clock)
Color formats	RGB565, RGB888, ARGB8888, etc.
Multi-layer	Background + foreground + icon layers, supports blending
Frame buffer	Single/double buffer mode, DMA improves performance
Palette	8/16-bit palette mapping for color conversion
Sync signals	HSYNC, VSYNC, DE, configurable polarity and timing
DMA support	Automatic memory transfer, CPU-free
Interrupts	Frame-end, line-end interrupts for synchronized refresh
Rotation/Flip	Supports 90°/180°/270° rotation and X/Y flipping

---

RT-Thread LCD Driver Framework

RT-Thread provides a **LCD driver framework**, which abstracts the underlying controller and supports different screen types and MCU peripherals. Key

interfaces include:

Main Interfaces

Function/Macro	Purpose
`rt_device_find("lcd")`	Find the LCD device handle
`rt_device_open(dev, flags)`	Open the device and initialize the LCD hardware
`rt_device_control(dev, cmd, args)`	Control the LCD, e.g., update image, set backlight, read screen info
`rt_device_write(dev, pos, buffer, size)`	Write pixel data to the LCD
`rt_device_read(dev, pos, buffer, size)`	Read pixel data from the LCD (supported on some screens)
`rt_device_close(dev)`	Close the LCD device

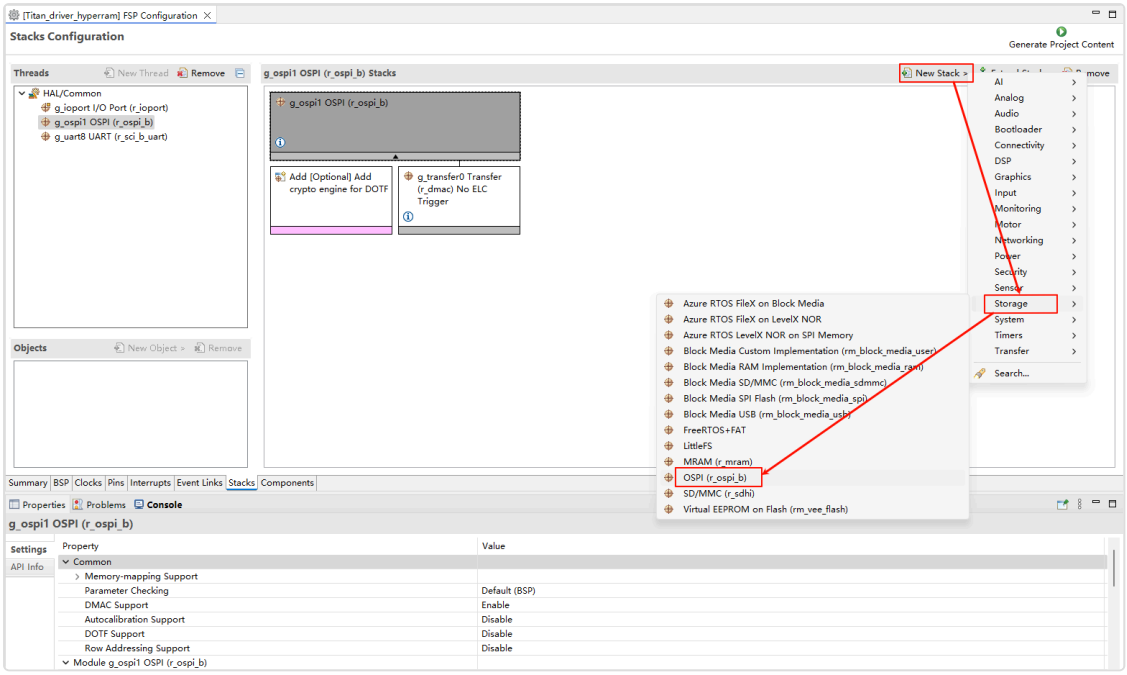
Hardware Description



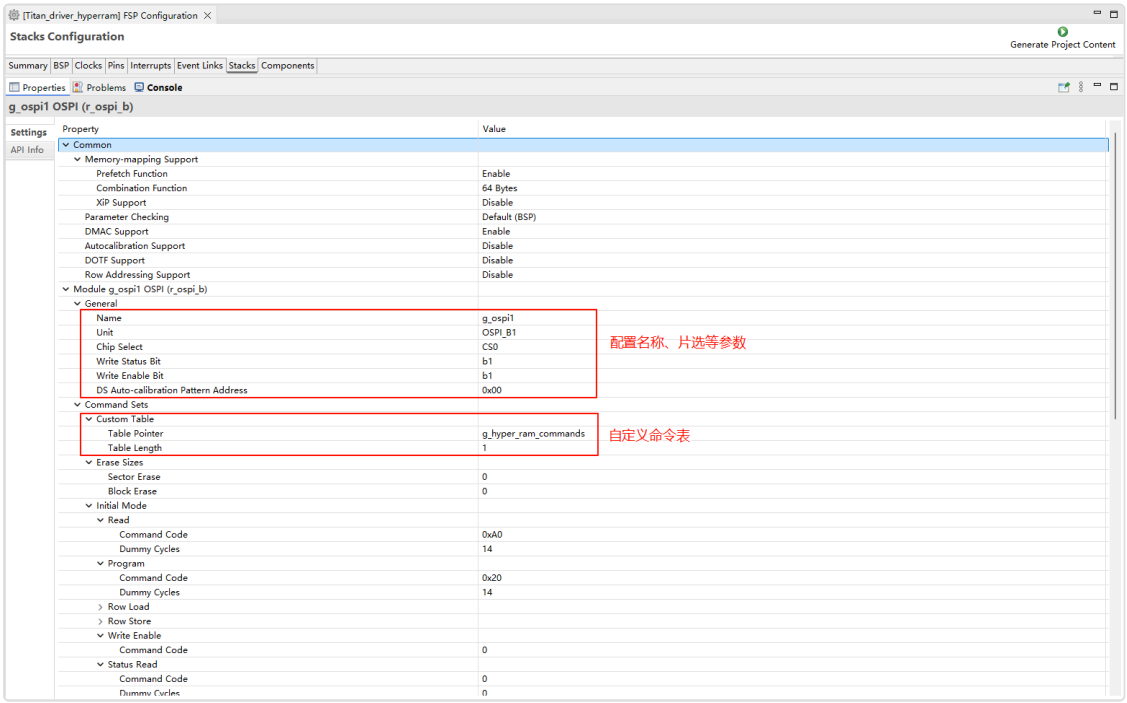
FSP Configuration

HyperRAM Configuration

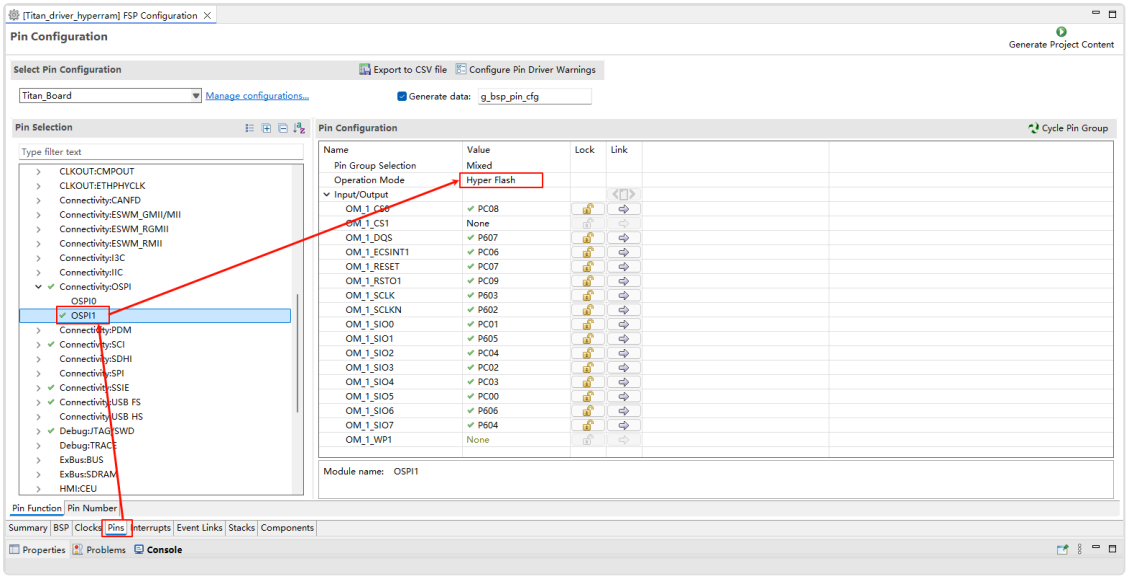
- Create a `r_ospi_b` stack：



- Configuration `r_ospi_b` stack：

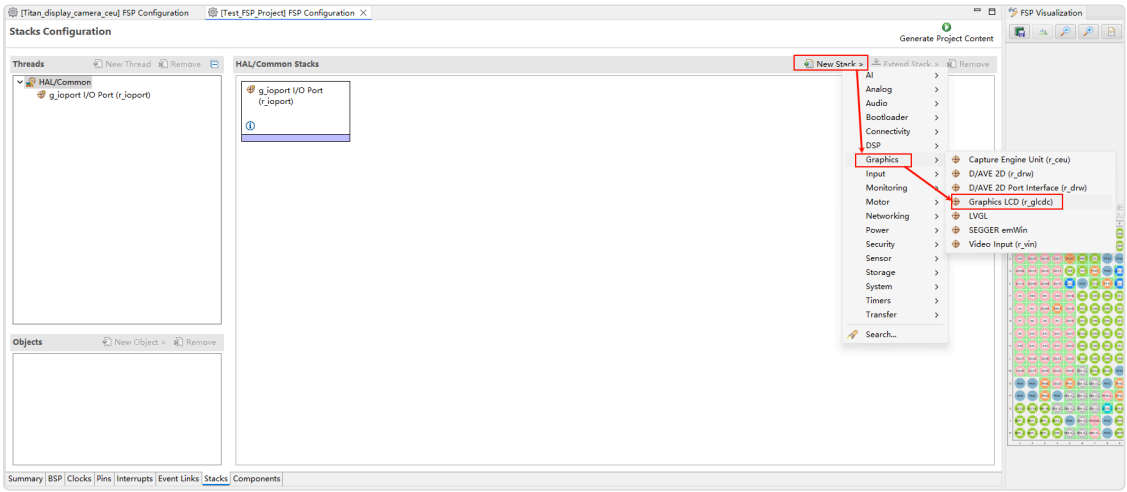


- Configuration HyperRAM pins：

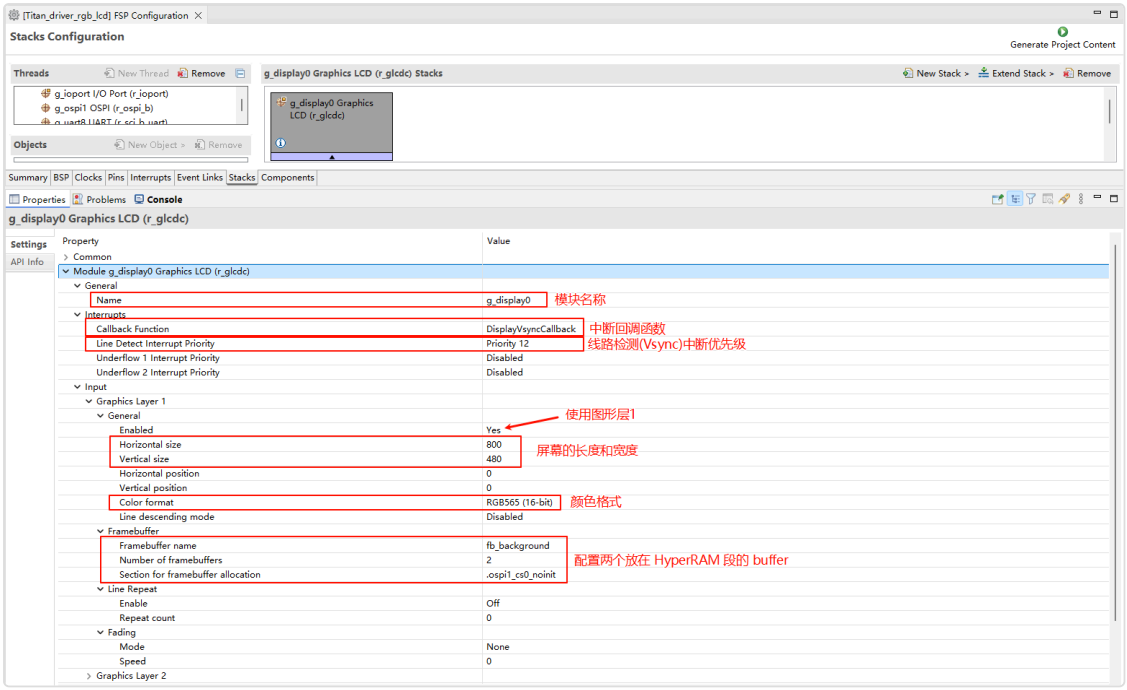


RGB LCD Configuration

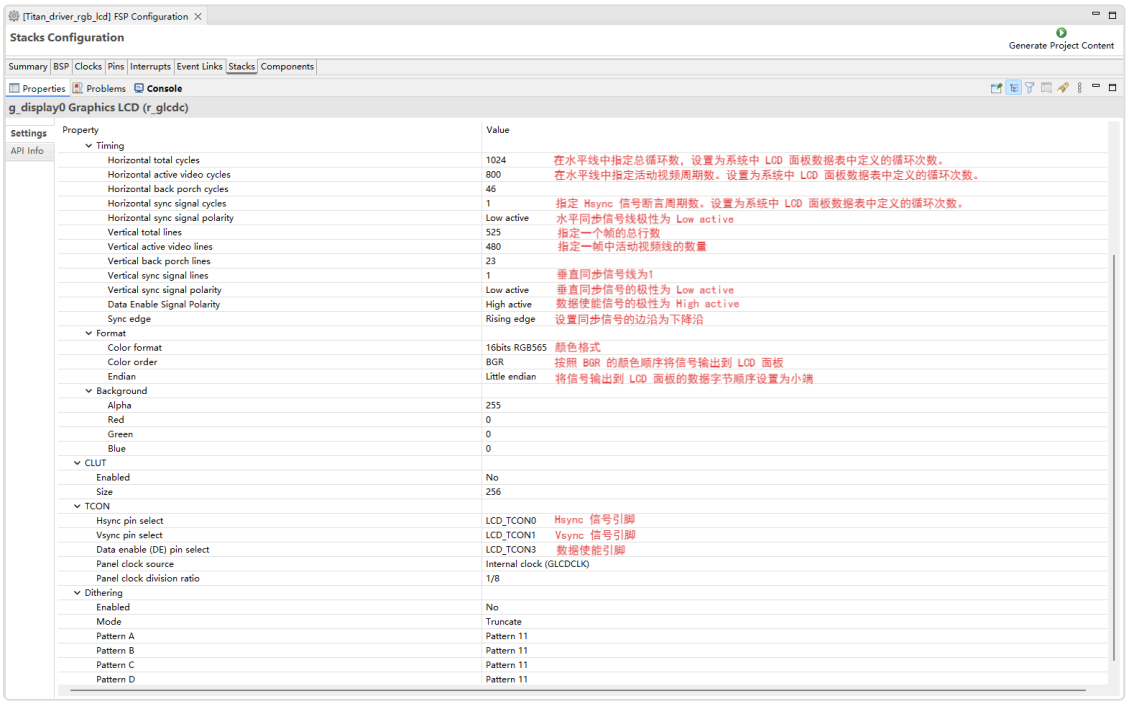
- Create a `r_glcdc` stack:



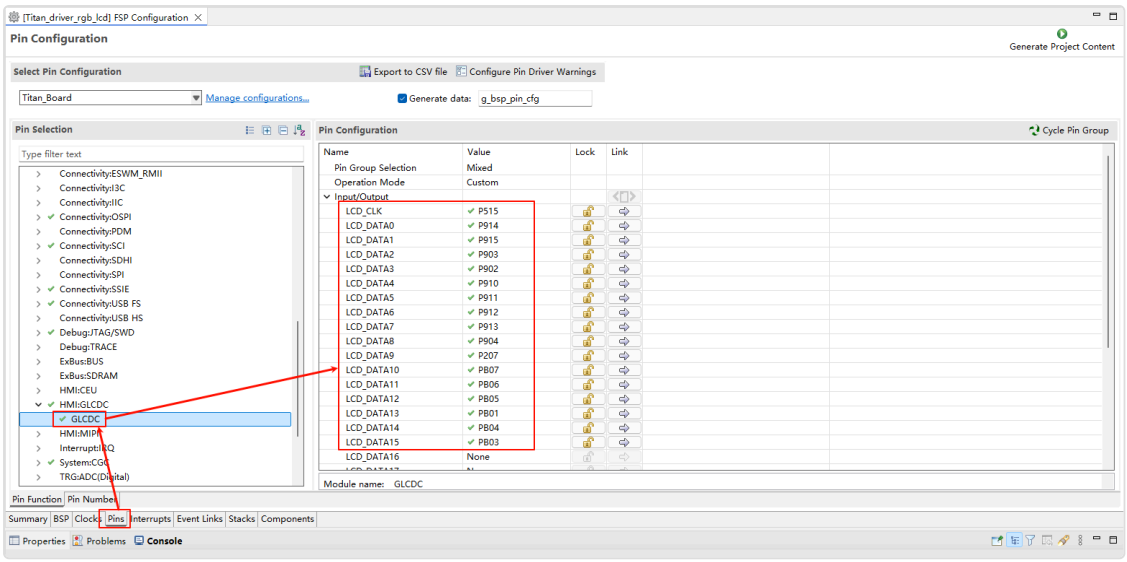
- Configure interrupt callback and graph layer 1:

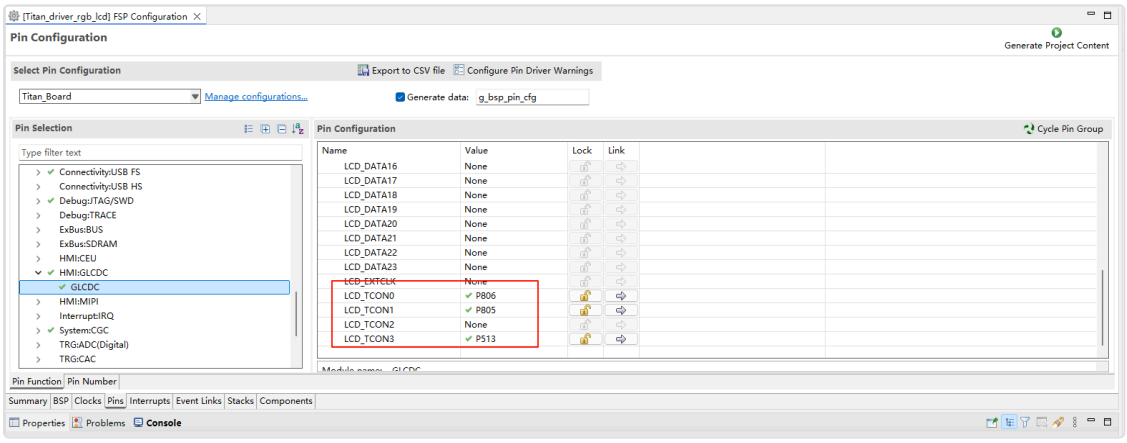


- Configure Output、CLUT、TCON and Dithering:



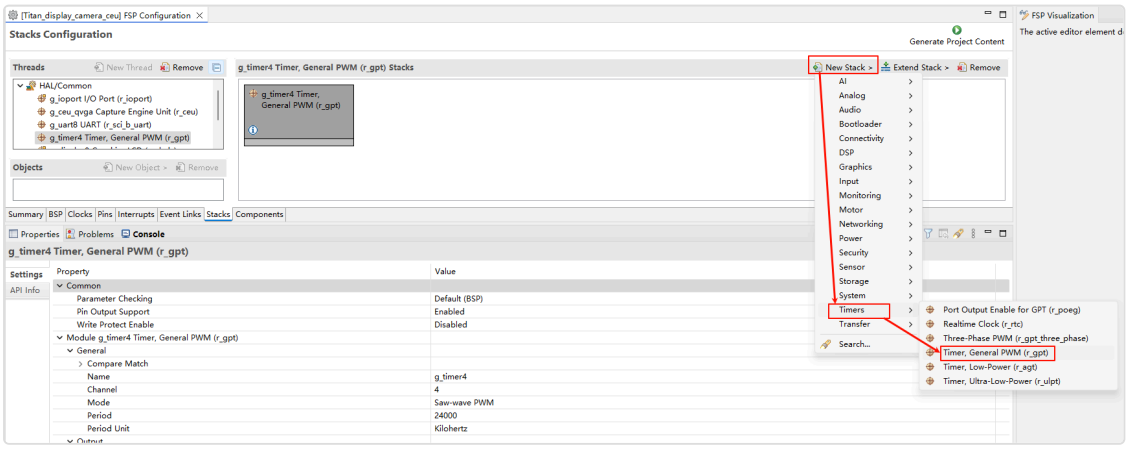
- Configure GLCDC pins:



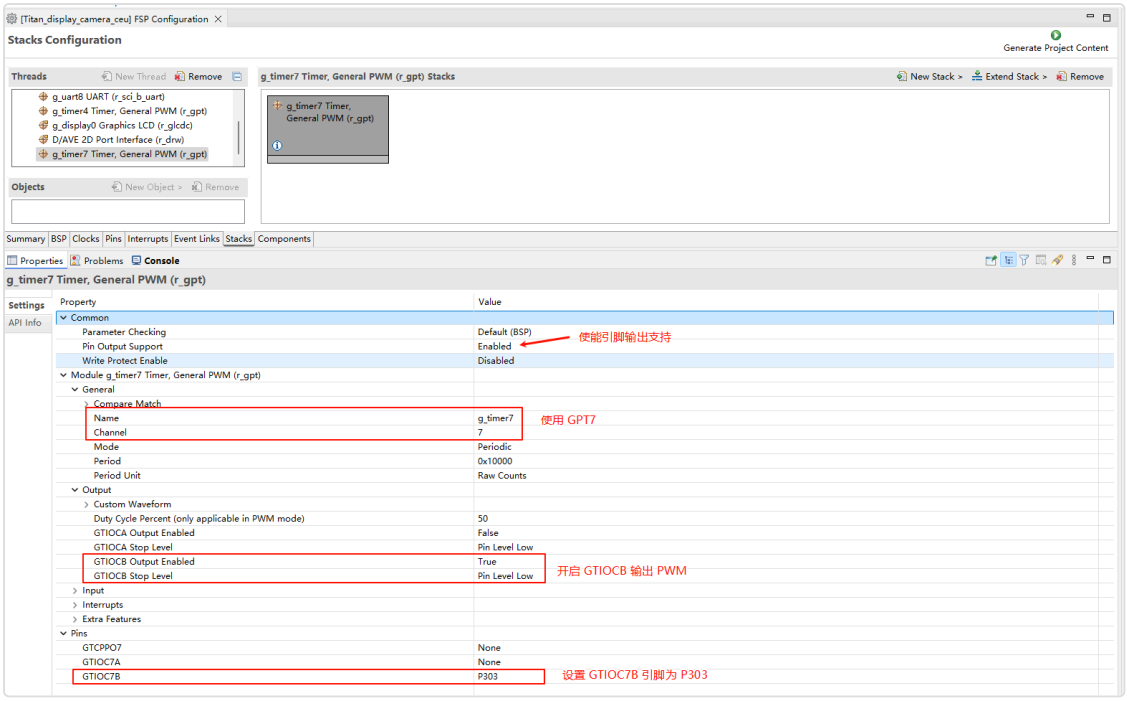


LCD Backlight Configuration

- Create a `r_gpt` stack:



- Configure back light PWM output:



D/AVE 2D Configuration

- Create a `r_drw` stack:



RT-Thread Settings Configuration

- Enable RGB565 LCD in RT-Thread Settings, using pwm7 output screen backlight.



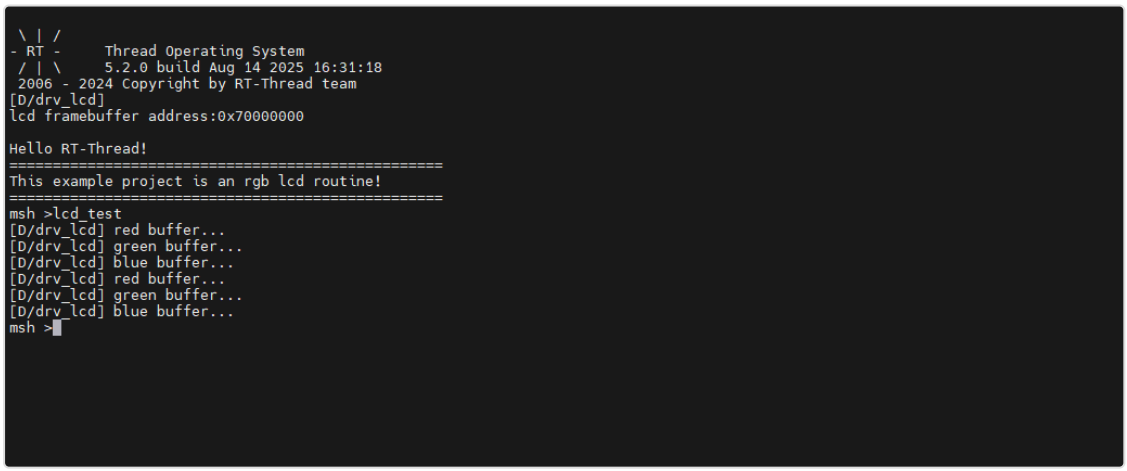
Compilation & Download

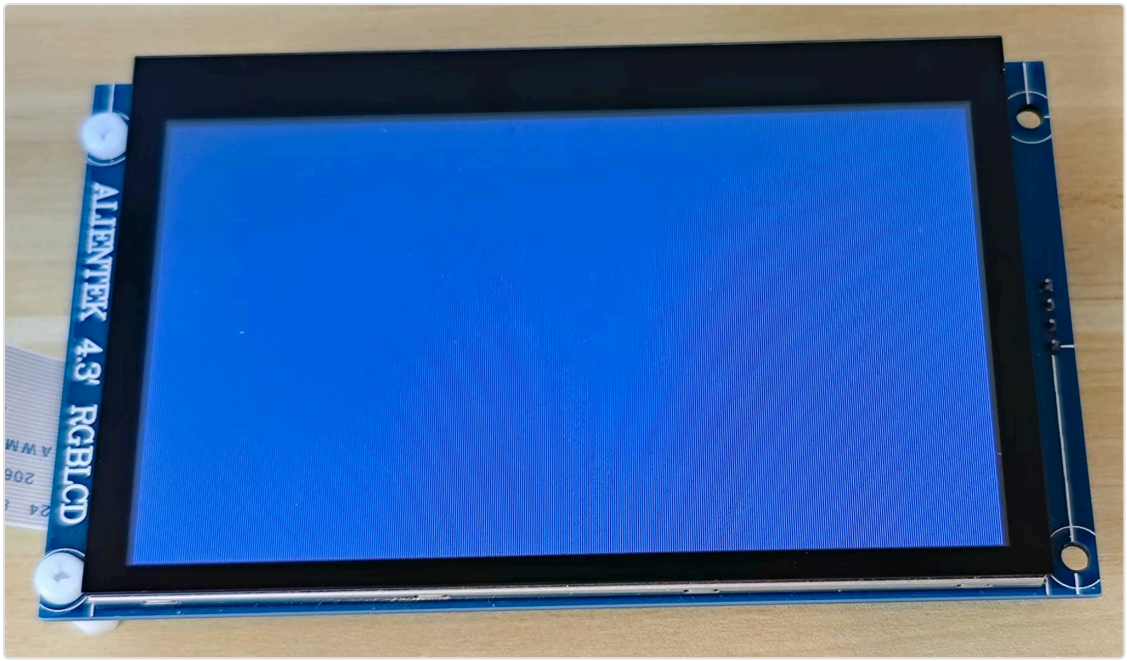
- **RT-Thread Studio**: In RT-Thread Studio's package manager, download the Titan Board resource package, create a new project, and compile it.

After compilation, connect the development board's USB-DBG interface to the PC and download the firmware to the development board.

Run Effect

After reset the development board, type `lcd_test` command in the terminal to run the brush program.




ALIENTEK 4.3' RGBLCD

6.4. USB PCDC Driver Example Guide

English | **Chinese**

Introduction

This example demonstrates how to use the **RA8 Series MCU USB PCDC module (r_usb_pcdc)** on the **Titan Board** to enumerate the device as a **USB virtual serial port (CDC)**, allowing serial communication with the board on a PC as a “COM port/ttyACM”.

USB PCDC Instructions

1. Overview

USB PCDC (USB Peripheral Communications Device Class) is a specific implementation of the **CDC (Communications Device Class)** defined in the USB protocol.

It primarily allows **embedded devices to appear as a virtual serial port (VCOM) over USB**, enabling the host (PC or embedded host) to interact with the USB device just like a traditional UART serial port.

Common application scenarios:

- MCU as a USB device, recognized by the PC as a virtual serial port (COMx).
- Embedded devices communicating with a host computer for data transfer or debugging.
- Replacing physical RS-232/RS-485 interfaces to reduce hardware cost.

2. PCDC Protocol Features

1. **CDC Class Protocol**

- USB-defined communication device class, uses standard class drivers (no extra driver installation needed; automatically recognized on Windows/Linux/macOS).

2. **Data Transfer Methods**

- **Control Endpoint (Endpoint 0)**: Used for device enumeration and descriptor transfer.
- **Interrupt Endpoint**: Used for status and control signals (e.g., DTR, RTS).
- **Bulk Endpoint**: For high-efficiency, reliable serial data transfer.

3. **Virtual Serial Port Features**

- Appears as a standard serial port (COM port or ttyUSB) on the host.
- Supports UART configurations such as baud rate, data bits, parity, and stop bits.

3. Typical Application Architecture

- **USB Device (MCU side)**: Implements the PCDC protocol stack, connecting USB data to the MCU's internal UART/application layer.
- **USB Host (PC/embedded system)**: Maps a virtual serial port through the OS-provided CDC driver.
- **Application Layer**: Host software (PuTTY, Tera Term, serial monitor tools) can communicate directly.

4. Advantages

- **No additional driver needed**: Most OSs include built-in CDC drivers.
- **Replaces traditional serial ports**: Enables debugging/communication even without a physical UART.
- **Higher data rates**: Full-Speed (12 Mbps) or High-Speed (480 Mbps) USB exceeds typical UART bandwidth.
- **High compatibility**: Works with various host applications (LabVIEW, Python pySerial, C# serial libraries).

RA8 Series USB PCDC Module (r_usb_pcdc) Overview

The RA8 Series MCU integrates a **USB PCDC (Peripheral CDC) module**, designed to enumerate the MCU as a **USB virtual serial port (CDC device)**, enabling bidirectional data communication with a PC or other USB hosts. It

works together with the RA8 **basic USB module (r_usb_basic)**, and the **r_usb_pcdc class driver** provides full CDC functionality.

1. Module Functionality

- **Device Mode Support**: MCU acts as a USB device connected to a host (Full-Speed / High-Speed, depending on MCU specification).
- **Virtual Serial Port (CDC) Features**:
- Handles class requests: `SET_LINE_CODING`, `GET_LINE_CODING`, `SET_CONTROL_LINE_STATE`, `SEND_BREAK`.
- Supports standard Bulk IN/OUT endpoints and an Interrupt IN endpoint.
- Generates serial port status notifications (e.g., DTR/RTS, line state).
- **Data Transfer Management**: Supports high-efficiency transfer with DMA/DTC or FIFO buffers.
- **Event Callback Mechanism**: For connection, disconnection, configuration completion, data transfer completion, class request handling, and error events.
- **Hot-plug and Low Power**: Supports VBUS detection, suspend/resume, and remote wakeup.
- **Application Layer Interface**: Provides API for read/write buffers, send-complete notifications, and event callbacks.

2. Supported Features

- **Standard USB Enumeration**: Supports Device, Configuration, and Interface Descriptors.
- **Endpoint Management**: Configures Bulk IN/OUT and Interrupt IN endpoints.
- **Class Request Handling**: Parses line coding and control line state according to CDC protocol.
- **Reliable Transfers**: Supports CRC, NAK, STALL, and timeout handling.
- **Multiple Device & Composite Device Support**: Can dynamically manage multiple CDC devices via hubs.
- **DMA Collaboration**: Optimizes data throughput and reduces CPU load.

3. Module Architecture

```
[ Application Layer Task/Thread ]
         ↑     Callback/Event notifications (connect, configure
[ r_usb_pcdc Class Driver ]
         ↕     Class request parsing, data transfer management,
[ r_usb_basic Basic Driver ]
         ↕     USB device enumeration, endpoint control, SOF in
[ USB Device Controller/PHY ]
         ↕     USB D+/D- signals (FS) or ULPI/HS PHY (HS)
[ USB Cable ]  <–> Host (PC)
```

Key Submodules:

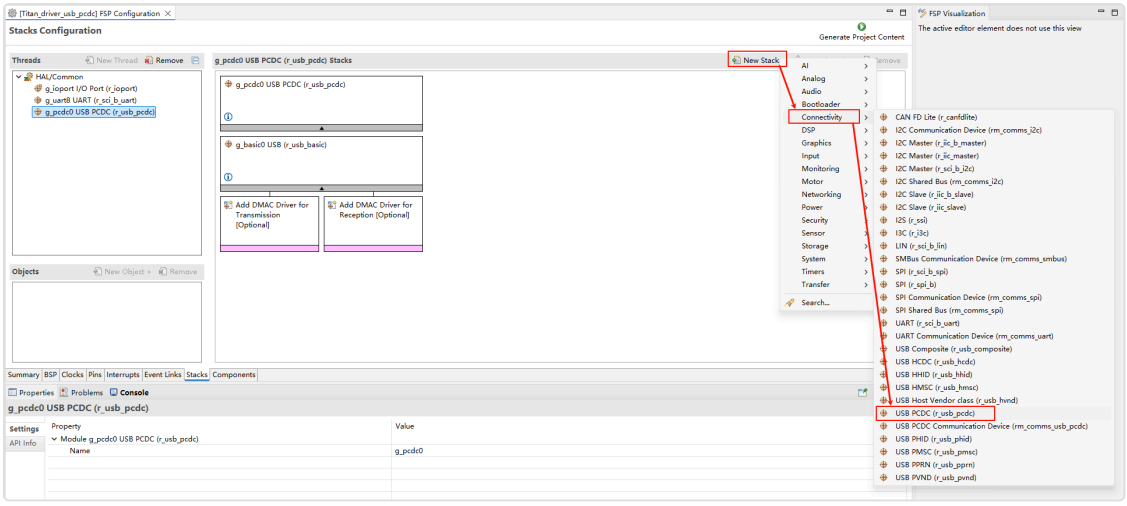
1. **Class Request Handler**: Parses CDC class requests and callbacks the application.
2. **Endpoint Control and Buffers**: Manages Bulk/Interrupt endpoints and supports DMA/FIFO.
3. **Event Management**: Handles connection/disconnection, configuration, suspend/resume, transfer complete, error events.
4. **Power and Clock Control**: USB PHY, VBUS detection, suspend/wake management.

4. Workflow

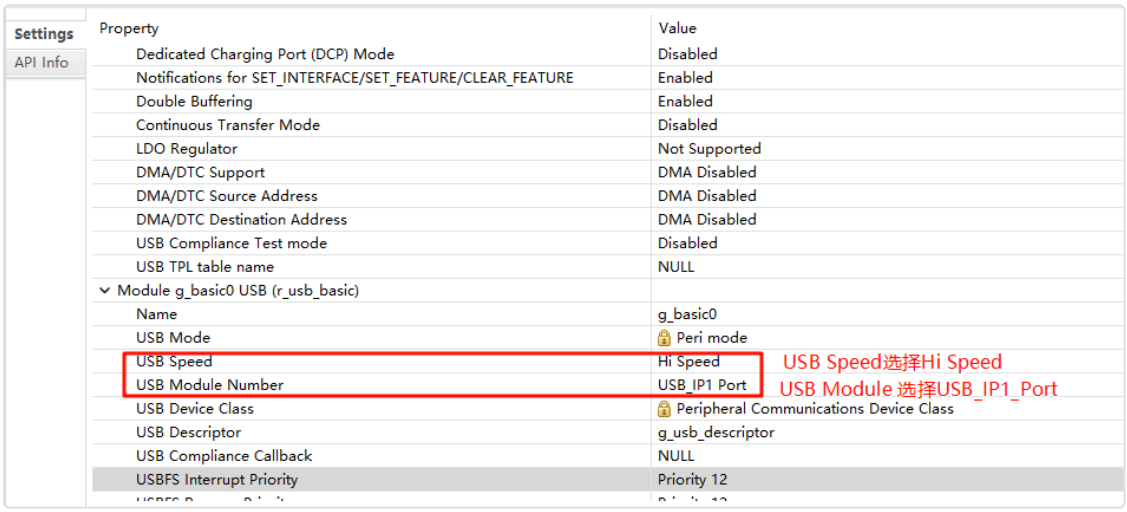
1. **USB Initialization**: Initialize USB controller, PHY, and class driver.
2. **Device Connection Detection**: Detect VBUS/ID signals to trigger enumeration.
3. **Enumeration and Configuration**: Upload descriptors and configure interface/endpoints.
4. **Endpoint Data Transfer**: Application exchanges data with host via Bulk IN/OUT endpoints.
5. **Class Request Handling**: Process CDC control requests from host.
6. **Event Callback Notification**: Notify application of connection, configuration, and transfer completion.
7. **Disconnection Handling**: Stop endpoint transfers, release buffers, and handle device removal events.

FSP Configuration Configuration

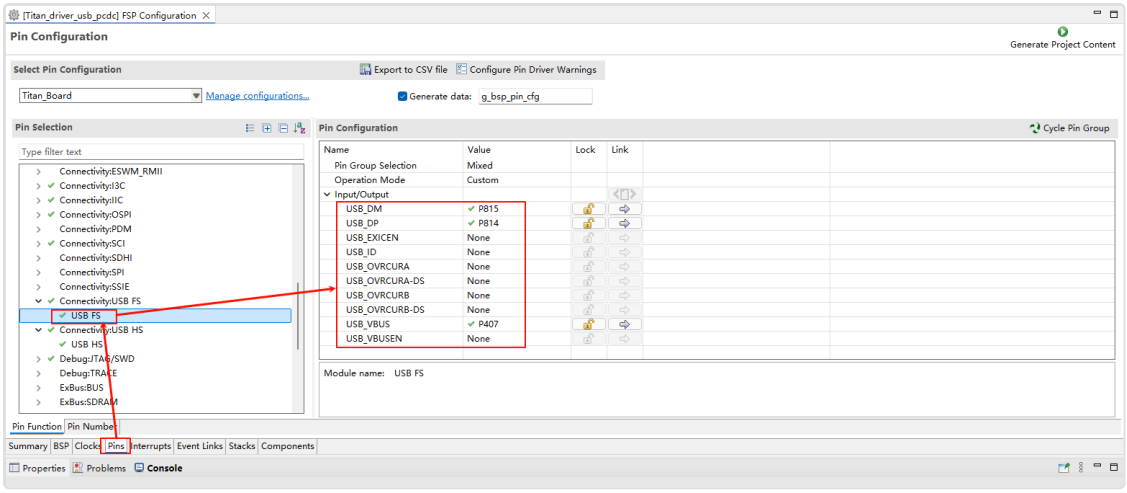
- Create a `r_usb_pcdc stack` :



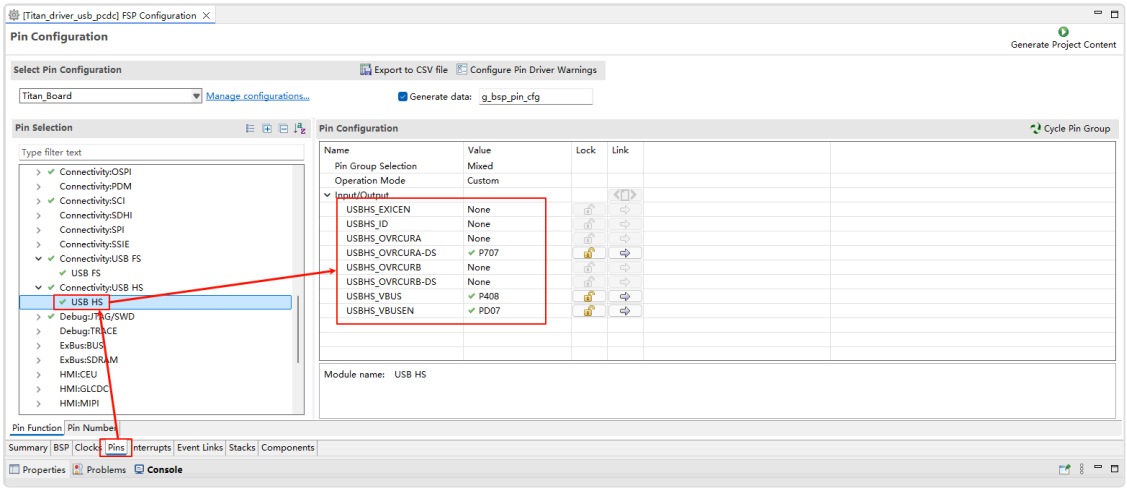
- Configure `r_usb_basic` :



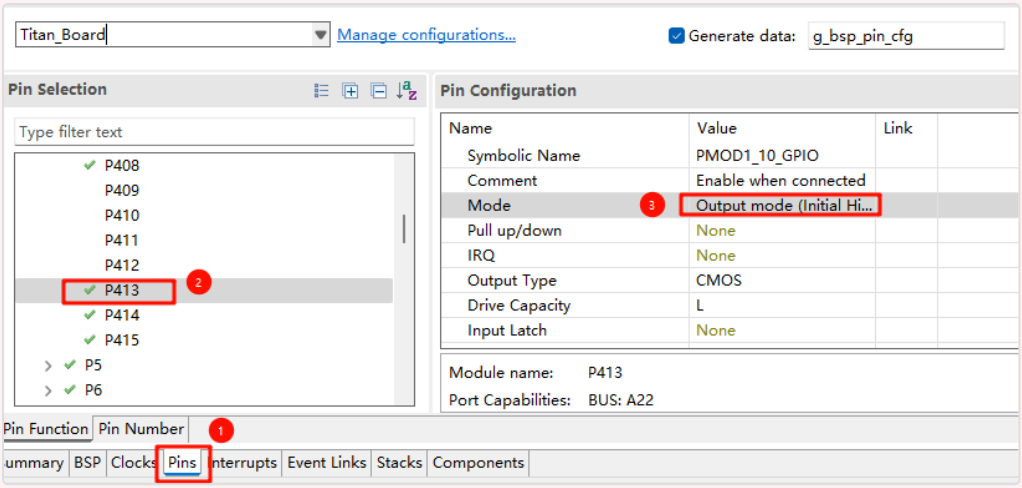
- Configure USB FS pin :



- Configure USB HS pin：

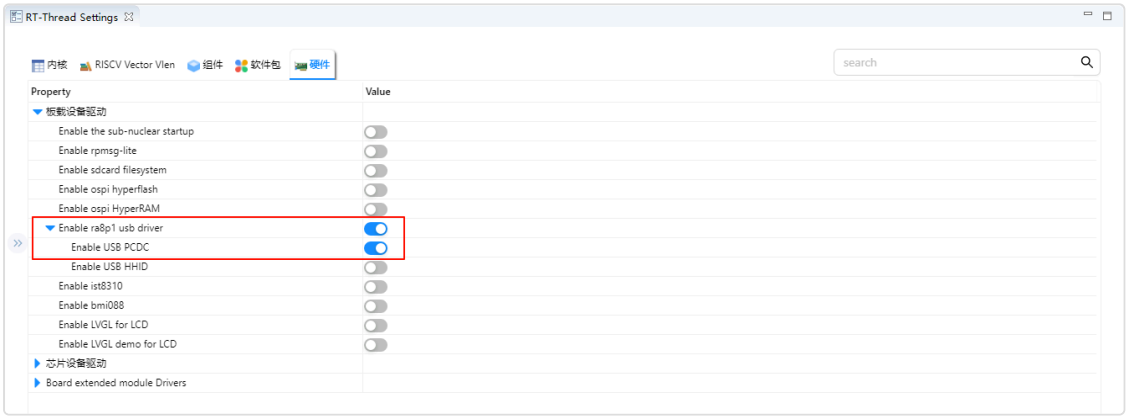


> ***PS:*** *Note that when switching to HS mode, the* ***P413 pin*** *needs to be pulled high, as shown in the figure below:*



RT-Thread Settings Configuration

Enable USB PCDC。



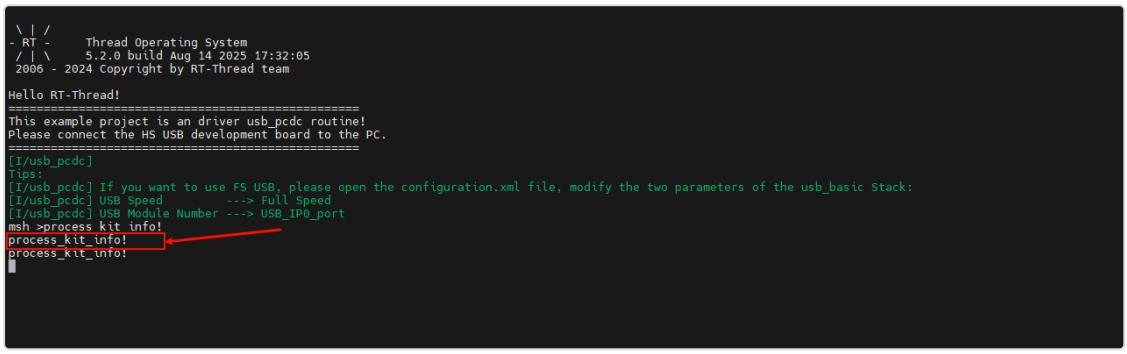
Compilation & Download

- **RT-Thread Studio**: Download the Titan Board resource package from the RT-Thread Studio package manager, then create a new project and compile.

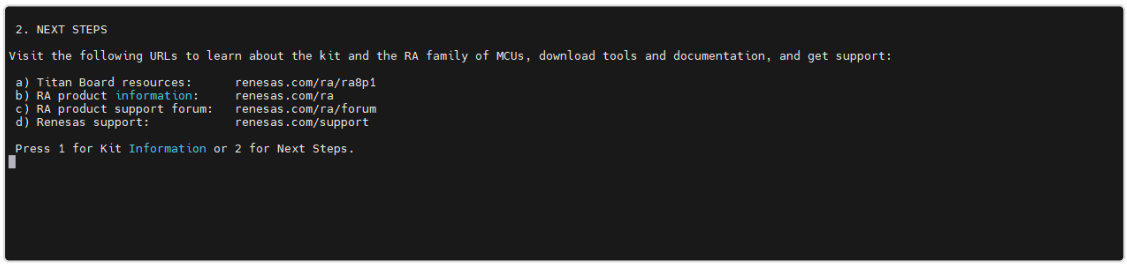
After compiling, connect the USB-DBG interface of the development board to the PC and download the firmware to the development board.

Running Effect

Use USB cable to connect the USB-DEV interface of Titan Board with the computer, open the serial port terminal with the serial port assistant, input 1 into the serial port terminal, and see the output in the debugging terminal of the development board.



Enter 2 into the USB-PCDC serial terminal, and you will see the following output in the USB-PCDC serial terminal:



6.5. WAV Audio Player Example Guide

English | 中文

Introduction

This example demonstrates how to play **WAV format audio files** on the **Titan Board Mini** using the **ES8156 Audio Codec** via the **SSI (Synchronous Serial Interface)**, combined with the **RT-Thread Audio Framework** for complete audio playback functionality. This project supports **16 sample-rate options, mono/stereo channels, and 16-bit PCM WAV** files.

Main features include:

- High-quality audio playback using ES8156 codec
- WAV format audio file playback support (16 sample-rate options, 16-bit PCM, mono/stereo)
- I2S audio data transmission via SSI interface
- Volume control, pause/resume/stop playback control
- Audio file reading from SD card or Flash file system

Hardware Introduction

1. ES8156 Audio Codec

The **Titan Board Mini** features the **ES8156** high-performance audio codec:

Parameter	Description
Model	ES8156
Manufacturer	Everest Semiconductor
Type	Stereo DAC (Digital-to-Analog Converter)
Resolution	24-bit
Sample Rate	8kHz - 192kHz

Parameter	Description
SNR	> 100dB
Output Power	30mW + 30mW @ 16Ohm
Interface	I2S / PCM
Control Interface	I2C
Operating Voltage	1.8V - 3.6V

2. ES8156 Key Features

Audio Performance

- **High-fidelity audio**: 24-bit DAC, supporting high-resolution audio
- **Wide sample rate range**: 8kHz - 192kHz, covering various audio scenarios
- **Low noise**: > 100dB SNR for clear audio quality
- **Low distortion**: THD+N < -85dB
- **Multi-format support**: I2S, Left-justified, Right-justified, PCM

Output Characteristics

- **Stereo output**: Left and right channels independent output
- **Headphone driver**: Built-in headphone amplifier, direct drive for 16Ohm - 32Ohm headphones
- **Line output**: Line-level output support
- **Volume control**: Digital volume control, range 0-255, +/-0.5dB/Step

3. SSI (Synchronous Serial Interface)

The SSI interface on RA8P1 is used for audio data transmission:

- **I2S protocol**: Standard I2S audio protocol
- **Master/Slave mode**: Configurable as master or slave
- **DMA support**: DMA transfer support, reducing CPU usage

- **Multi-channel**: Support for mono/stereo

Software Architecture

1. Layered Design

The audio playback system uses a layered architecture:

```
Application Layer (User Code)
    ↓
WAV Player Package - WAV Player
    ↓
Audio Device Framework - RT-Thread Audio Device Framework
    ↓
ES8156 Driver - ES8156 Driver
    ↓
SSI/I2S Driver - SSI/I2S Driver
    ↓
FSP SSI HAL - Hardware Abstraction Layer
```

2. Core Components

WAV Player Package

RT-Thread's WAV player package provides complete audio playback functionality:

- **WAV file parsing**: Parse WAV file header, extract audio parameters
- **Playback control**: Play, pause, resume, stop
- **Volume control**: 0-99 level volume adjustment
- **State management**: Playback state machine management
- **Multi-file support**: Support playback from different storage media

Main interfaces (`wavplayer.h`):

```
/* Play WAV file */
int wavplayer_play(char *uri);

/* Stop playback */
int wavplayer_stop(void);

/* Pause playback */
```

```
int wavplayer_pause(void);

/* Resume playback */
int wavplayer_resume(void);

/* Set volume (0-99) */
int wavplayer_volume_set(int volume);

/* Get volume */
int wavplayer_volume_get(void);

/* Get playback state */
int wavplayer_state_get(void);
```

ES8156 Driver

The ES8156 driver provides codec control interfaces:

```
/* Initialize ES8156 */
rt_err_t es8156_device_init(void);

/* Get device handle */
struct es8156_device *es8156_get_device(void);

/* Set volume (0-255) */
void es8156_set_volume(rt_uint8_t volume);

/* Mute/Unmute */
void es8156_mute(rt_bool_t mute);

/* Power down */
void es8156_powerdown(void);
```

Driver configuration:

- **I2C interface**: I2C1 (for register configuration)
- **I2C address**: 0x08 (ADDR pin connected to GND)
- **Analog voltage**: 3.3V (configurable 1.8V/3.3V)
- **Default volume**: 191 (0dB)

3. Project Structure

```
Titan_Mini_wavplayer/
├── src/
│   └── hal_entry.c          # Main program entry
```

```
├── libraries/
│   └── HAL_Drivers/ports/
│       └── es8156/
│           ├── es8156.c     # ES8156 driver implementation
│           └── es8156.h     # ES8156 driver header
└── packages/
    └── wavplayer-latest/    # WAV player package
        ├── inc/
        │   ├── wavplayer.h  # Player interface
        │   ├── wavrecorder.h # Recorder interface
        │   └── wavhdr.h     # WAV file header definitions
        └── src/
            ├── wavplayer.c      # Player implementation
            ├── wavplayer_cmd.c  # Player commands
            ├── wavhdr.c         # WAV file parsing
            └── wavrecorder.c    # Recorder implementation
```

Usage

1. Initialization Flow

The system needs to initialize ES8156 and audio device during startup:

```
#include <rtthread.h>
#include "es8156.h"

void hal_entry(void)
{
    /* Initialize ES8156 codec */
    if (es8156_device_init() != RT_EOK)
    {
        rt_kprintf("ES8156 initialization failed!\n");
        return;
    }

    rt_kprintf("ES8156 audio codec initialized successfully!\n"
    rt_kprintf("WAV Player is ready!\n");

    /* Main loop */
    while (1)
    {
        /* Application logic */
        rt_thread_mdelay(1000);
    }
}
```

2. Playing WAV Files

Use msh command line to play audio files:

```
/* Play WAV file from SD card */
msh >wavplay -s /sdcard/test.wav
```

3. WAV File Format Requirements

WAV files must meet the following requirements:

- **Format**: Standard WAV (RIFF) format
- **Encoding**: PCM encoding
- **Sample rate**: One of the 16 supported sample-rate options
- **Bit depth**: 16-bit
- **Channels**: Mono or stereo
- **File extension**: .wav

Configuration

1. Kconfig Configuration

Configure audio options in `libraries/M85_Config/Kconfig`:

```
menuconfig BSP_USING_AUDIO
    bool "Enable Audio"
    select BSP_USING_SSI0
    select BSP_USING_ES8156
    default n
    if BSP_USING_AUDIO
        config BSP_USING_AUDIO_VOLUME
            int "Default volume (0-99)"
            default 50
    endif
```

2. RT-Thread Settings

In RT-Thread Studio, enable the following components:

1. **Device drivers**

- Enable SSI/I2C device drivers
- Enable Audio device

2. **Packages**

- Add wavplayer-latest package

3. **Components**

- Enable DFS file system
- Enable SD card file system

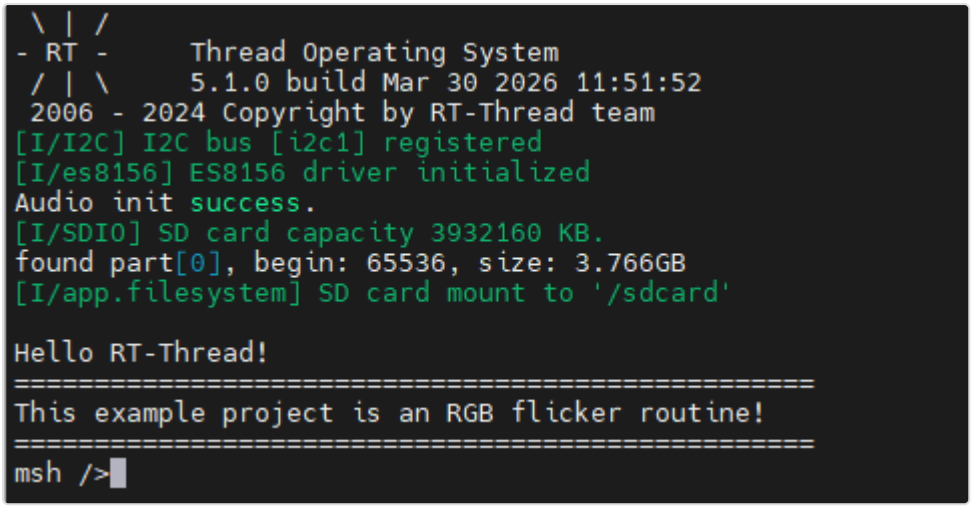
3. Hardware Connections

ES8156 connections to Titan Board Mini:

- **I2C control interface**:
- I2C1_SDA / I2C1_SCL
- Used for ES8156 register configuration
- **I2S audio interface**:
- SSI_TX (audio data output)
- SSI_BCK (bit clock)
- SSI_LRCK (left/right channel clock)
- SSI_MCK (master clock, optional)

Run Effect

Convert the audio file to **PCM, 16-bit, mono/stereo WAV** format with one of the 16 supported sample-rate options, save it to the SD card, insert the SD card into the development board. After powering on, the system will automatically mount the SD card. Then enter the command `wavplay -s <filename>` to play.



Further Reading

- RT-Thread Audio Framework Documentation